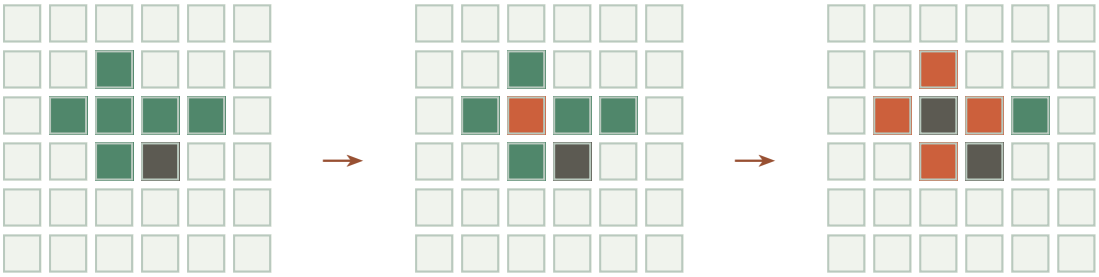
一棵树 · 一个位 · 一组线程

森林中的一位火

HPCGame「fire」源码讲解书

同一棵树，在第一版里占一个整数，在第七版里分住两张位图。跟着它经过一拍火势，看懂八次改写。



① 原始森林　② 惊雷落下　③ 同步转移

空地：浅白　树：绿　火：橘　灰：深灰

SOURCE EDITION / 2026.11

作者的 v1 – v8 ＋ 同学的活动块实现
以随包源码真实行号为准；第七版 L1 – 261 完整精读。

目录

右列是本版 PDF 的纸面页码。在 HTML 中点击章名直接跳转。

00	同一场火，两种时间	4
01	从两张纸到三行火	7
02	v7 逐行精读：一片火如何跨过第 64 列	13
03	把 v1–v6 摊在 v7 旁边	52
04	v8：评测没有打开 OpenMP，怎么办	58
05	另一位作者：只把可能变化的块排进队列	63
A	术语第一次出现在哪里	74
B	复杂度与性能账本	85
C	按拼法查语法与设施	87
D	哪些地方还不能照注释下结论	95

定位约定：L 指源码物理行。1–8 属于同一作者；br 属于另一位作者。

BEFORE READING

阅读导引　森林中的一位火

这一章你会得到什么：按自己卡住的地方找到入口。纸面上的 L191 是源文件第 191 行，和 PDF 的第 191 页没有关系。

你想弄明白什么	翻到哪里	读完自己验一下
为什么同一拍只烧一格远	第 0 章的六行小图	解释 (2,4) 为何没被点燃
每次改版到底省了什么	第 1 章与第 3 章	不看表说出 v5 与 v6 的派工区别
位运算看不下去	第 2 章 L56–84、189–221	画出列 63 的火怎样点亮列 64
线程为何互相等待	第 2 章 L209–218、第 4 章	指出三道门分别保护哪份数据
为何同学的两位公式不同	第 5 章开头与活动块手推	用一棵旧火分别代入两套公式
只想查一个符号或耗时说法	附录 A、B、C、D	说出 8.11 的分母与单位

> 一句话记住：先在六行纸上算对，再把同一个问题换成六十四位去问。

CHAPTER 00

第 0 章　同一场火，两种时间

这一章你会得到什么：拿一支笔，在六行数字里找出下一步所有的火。能解释为什么新点燃的树不能立刻再烧一格，你就已经抓住了后面八个程序都不能改的规则。

0.1　从一棵树开始

这是一场格子游戏。时间步（time step，模拟的一轮）有两个时刻要分清：魔法刚执行完，和火势刚更新完。格子上的数字是状态（state，某一刻格子的取值），只有四种：0 是空地，1 是活树，2 是火，3 是灰。惊雷点一棵树，别的状态不动；回春把一个闭矩形里的灰变成树，别的状态不动。闭矩形就是上下左右端点都算在内，两端恰好相等也能选中一个格子。

魔法完成后，给森林拍一张“照片”。逐格看照片：树旁边的上、下、左、右只要有火，就在新纸上写 2；旧火一律写 3；其余原样抄。斜着挨到火没有用，地图左端也不和右端相接。惊雷点出的火已经在照片里，所以它这一拍就变成灰，也能点着邻树。刚被邻居传染的火却只出现在新纸上，它要等下一拍。源码里两张轮换的图就是这两张纸，叫双缓冲（double buffering，分别存放当前和下一状态）。

> 一句话记住：一轮可以先回春再着火，但火不能在同一次转移里连跳两格。

0.2　亲手做一张 6 × 6 的照片

下面是一个完整的单步输入。第一行依次为边长、事件数、总步数；接下来六行是地图；末行是“第 1 步、类型 1、行 2、列 2”。坐标（coordinate，行列位置）从 0 数，所以这是从上往下第三行、从左往右第三格。横着看列，竖着看行。

```
6 1 1
0 0 0 0 0 0
0 0 1 0 0 0
0 1 1 1 1 0
0 0 1 3 0 0
0 0 0 0 0 0
0 0 0 0 0 0
1 1 2 2
```

魔法后的第三行变成 `0 1 2 1 1 0`；这是本轮唯一的一处就地改写。照片上的火只有 (2,2)。在新纸上划掉它写 3，再圈出 (1,2)、(2,1)、(2,3)、(3,2)，四棵树都写 2。特别看 (2,4)：它和 (2,3) 相邻，但照片里的 (2,3) 是 1，所以它仍是 1。(3,3) 既是灰又只斜邻旧火，原样保留。

```
0 0 0 0 0 0
0 0 2 0 0 0
0 2 3 2 1 0
0 0 2 3 0 0
0 0 0 0 0 0
0 0 0 0 0 0
```

换一种魔法再想一次：假设上面这张输出就是下一步的输入，第 2 步回春矩形只有 (2,2) 一个格子。旧灰先变树，四邻又恰好都有火，于是本轮输出该格为火，并非树。这不是回春失效，而是你看的时刻不同。官方参考程序的事件循环、边界交换、生成新图也是这个先后关系。

> 一句话记住：一格算错时，先问你比较的是魔法前、魔法后还是转移后。

0.3　文件长什么样，什么叫相同

`forest.in` 和 `forest.out` 是文本文件（text file，以字符记录数据的文件）；字符 '2' 和内存中数值 2 不能直接互换。输入的事件按步号递增，同一步最多一个。回春行在类型 2 后另有四个整数：起始行、起始列、结束行、结束列。输出不再写尺寸和事件，只写最终的 n 行地图。程序诊断可以写到标准错误流（standard error，单独的诊断通道），不能混进森林数字。

`context/gencase.py` 第 20–37、71–85 行规定了生成器：初始按权重 15:83:2 随机产生空、树、火，没有初始灰；这不是说每个 100 格的小图必有 2 棵着火。总步数为 7 时，取整后回春 3 次、惊雷 2 次，其余两步无事。矩形两边长度分别随机取样并限制范围，所以成本要看实际矩形，不能把“有一个事件”当作恒定的一次写入。

实际比对工具 `context/compare.py` 第 7–29 行同时读两行，去掉首尾空白，再按连续空白分出字段（token，分隔后的一段文字）。每行字段数和对应字段字符串都要相等；比如 `1 2` 与 `1 2` 可以相同，`01 2` 与 `1 2` 不相同。它并没有逐字节比较，也没有验证每个字段一定是 0–3。逐行并行读取两个文件的写法还有一处尾行漏检可能，见附录 D。写程序仍按题目的 n 行、每行 n 项，不依赖脚本的宽容。

> 一句话记住：数值、写出来的字符和比对脚本眼里的字段，是三层不同的东西。

0.4　68 秒怎样变成 8.1 纳秒

题面给 4 个节点（node，一台独立机器），每节点 16 个核（core，可执行指令的处理单元），启动 64 个进程（process，拥有独立地址空间的程序实例）。同一个进程里的线程（thread，可共享内存的一条执行路径）不是另一台机器。消息传递接口（Message Passing Interface，进程间显式交换数据的接口，简称 MPI）让不同进程交换边沿；共享内存并行接口（Open Multi-Processing，共享内存多线程编程接口，简称 OpenMP）让同一进程把循环分给线程。

满图 16,384 × 16,384 = 268,435,456 格，2,000 步，共 536,870,912,000 次逻辑格更新。这是拿不同程序作比较的共同分母，不等于每个程序都访问了这么多次内存。68 秒须合计完成每秒约 78.95 亿格。墙钟（wall-clock time，从开始到结束的实际经过时间）除以全部更新是 0.127 纳秒/格；如果理想地让 64 核平均分工，每核可花的时间才是 $68 \times 64 \times 10^9 / 536{,}870{,}912{,}000 = 8.11$ 纳秒/格。不要把两个数字都叫“每格耗时”然后直接相减。若实际上只用一个节点的 16 核，同一 68 秒只给每核约 2.03 纳秒/格；读写文件和等待还会占掉其中一部分。

正确且 110 秒时 40 分，正确且 68 秒时 150 分。在线性区内，每省一秒约增 $(150-40)/(110-68) = 2.62$ 分；89 秒恰在中点，对应 95 分。以上都是从题面推算，没有跑比

赛，也没有把 89 秒记到任何一个版本名下。

> 一句话记住：8.1 是把 64 核全部用起来才有的预算，不是运行记录。

0.5 读源码前的一张地图

位（bit，一个只能为 0 或 1 的位置）可以组成一个 64 位字（word，这里是装 64 个位的无符号整数）。位平面（bit-plane，把很多格子的同一属性并排存成一张位图）让一条逐位逻辑运算同时作用于 64 格。行窗口（rolling window，只保留眼下需要的相邻几行）和分块（tile，可独立调度的小片地图）都利用相邻关系，但解决的不是同一个问题。

版本	一句话定位	比上一版改了什么
v1	直接翻译规则	二维整数、格式化读写、每步复制旧图
v2	不再每步造副本	预分配新图并交换，输入整块读、输出攒块写
v3	让相邻地址对应相邻格	把每行独立的整数容器改成连续字节，检查短读
v4	一次算一字	两张位平面表示一代，另做一整张火图
v5	把两个大循环分给线程	两个循环分别进入 OpenMP 并行区，火图仍在
v6	一队线程贯穿全部时间步	分阶段派循环，单人做事件和交换，仍存整图火情
v7	每个线程只带三行火	按行段静态分工，自己滚动邻火窗口
v8	用标准线程落实分工	显式队伍和栅栏，回春按行分；可选 MPI 分支只让 0 号算

同学的 `br-fire-pool.cpp` 不在这条编号线上。它换了一套两位编码，再用活动分块队列决定“哪片需要算”；还保留了交换分区边缘的辅助函数。读这种代码不能只看文件顶部引用了哪个库，得从入口追到它真正调用了谁。第 5 章专门画清这条界线。

> 一句话记住：读 1→8 看“同一张地图怎么扫”，读 br 再问“哪些块这一步可以不扫”。

CHAPTER 01

第 1 章　从两张纸到三行火

这一章你会得到什么：能指出每次改版拿走的具体成本，还能分清哪些麻烦是新表示带来的。定位中的 L 是源文件的物理行，空行也占一个号码。 代码框保留关键原行，行号不连续时，中间行见标注的源文件区间。

1.1　v1　让每个判断都能用手指着读

读这个版本会用到三种基础写法。int（integer，整数类型）存格子状态；if（conditional statement，条件判断）表示条件成立才执行；for（for loop，计数循环）控制行列。std::cin（standard input stream，标准输入流）的 >> 从字符里解析一个整数；std::cout（standard output stream，标准输出流）的 << 向外写。这里的双尖号不是位移。方括号 [i] 以从零起的下标访问容器，访问者自己负责范围。

这一版想干什么：先把第 0 章那两张纸写成程序。二维数组（two-dimensional array，用行列两个下标取格子）读起来像方格纸。这里实际用嵌套向量（nested vector，外层容器保存一排内层容器）：sta[r][c] 要先找到第 r 行，再找到那一行的第 c 格。地图四周多一圈初值为零的格子，靠边的树也能照常检查四个方向。

卡在哪：题目规模上有约 2.68 亿格。按常见四字节整数（int，通常用来存放普通整数）估算，一张实心地图已经约 1 GiB；额外两圈和每行容器还没计入。一轮要把旧图完整复制给临时图，算完又把临时图赋回去。即便这一步一棵树都没烧，这两处依然要触碰整片内存。

这一版改了什么：第 26、31 行用 freopen 把标准输入输出重新接到文件；它接收路径、模式、原来的流，失败返回空指针。第 41 行多配两行两列，第 81 – 95 行把外部坐标各加 1。第 106 – 120 行只从 sta 读、向 temp 写；把第 114 行改成写 sta，就可能让一行连续的树一拍全烧起来。

选行 • 1-fire_pr.cpp L39 – 45

```
39      int n, m, t;
40      std::cin >> n >> m >> t;
41      std::vector<std::vector<int>> sta(n + 2, std::vector<int>(n + 2));
42      for(int i = 1; i <= n; i++) {
43          for(int j = 1; j <=n; j++) {
44              std::cin >> sta[i][j];
45          }
```

选行 • 1-fire_pr.cpp L104 – 120

```
106         std::vector<std::vector<int>> temp = sta;
110                     (sta[r - 1][c] == 2
112                     || sta[r][c - 1] == 2
116                 else if(sta[r][c] == 2) temp[r][c] = 3;
120         sta = temp;
```

结果与下一版的动机：这版的价值是把对错说清，后面每种新的状态表示都必须还原出同一结果。下一版可以拿走两次复制，前提是连“没变的格子”也在新图里写一次。

> 一句话记住：正确性来自只读旧图。

1.2 v2 把复制改成交换，把字符攒成块

先分清地址和内容。指针（pointer，保存地址的值）写成 char* p；*p 取该地址的一个字符，++p 改的是地址。只读限定 const（const qualifier，通过这个名字不可改所指值）不妨碍 p 移动。fopen、fread、fclose 分别负责开流、读若干项、关流；一次读取返回的‘项数’只有在每项一字节时才等于字节数。while（while loop，条件成立就反复执行）适合数量未知的连续空白。

这一版想干什么：算法还按格子逐个判断。换的是数据进出机器的方法。缓冲区（buffer，临时存放一批数据的一段空间）把读文件和辨认数字拆成了两步，程序可以顺着内存往前走，不必给每个格子做一次通用格式转换。

卡在哪：上一版每步才创建临时图；把它提到循环外后，新图里可能留下上一次使用时的旧值。比如这里本轮应保留空地，作者不能只是跳过空地不写。第 131 – 132 行就是这个保证。另一处隐患是第 32 行发出整文件读取请求，却没检查究竟读进多少字节。

这一版改了什么：第 26 – 44 行一次读取并扫描字符。第 92 行在步循环外分配 temp；第 121 – 136 行给所有真格赋值再 swap：交换（swap，互换两个容器管理的存储）不需要逐格复制。输出第 148 – 165 行分配 2^{22} 字节，每个格子产生一枚数字字符和一个分隔符。读计时从打开文件前开始，含文件读取，不能和上一版的起点混算。

选行 • 2-fire_pr.cpp L26 – 43

```
27      if (!fp) { std::fprintf(stderr, "open in failed: %s\n", argv[1]); return 1; }
31      std::vector<char> buf(fsz + 1);
32      std::fread(buf.data(), 1, fsz, fp);
34      buf[fsz] = '\0';
43          return v;
```

选行 • 2-fire_pr.cpp L119 – 137

```
123                 if(sta[r][c] == 1 &&
125                     || sta[r][c - 1] == 2 || sta[r][c + 1] == 2)) {
128                 else if(sta[r][c] == 2) {
129                     temp[r][c] = 3;
136         std::swap(sta, temp);
```

结果与下一版的动机：第 98 行容许消费当前步以前的事件，但并没有给乱序输入排序；合法输入本来就按步号排列。数据布局仍是二维 int，每次更新要看一整棵树的状态才用得上两个比特。v3 接着改这个账。

> 一句话记住：两张图都要把每个真格写全。

1.3 v3 让一个格子占一个字节

uint8_t（unsigned eight-bit integer，八位无符号整数）表示 0–255；std::size_t（size type，对象大小类型）适合装内存中的元素数或下标。static_cast<T>(x)（explicit type conversion，显式类型转换）要求将 x 转换成 T，并不能代替对 x 是否在允许范围内的检查。

这一版想干什么：字节（byte，通常由八个位组成）装得下 0 到 3。把很多独立的行改成一条长带子，行跨度（stride，从本行同一列走到下一行同一列要跨多少元素）直接参加下标计算。这样相邻行也位于同一片连续分配的空间里。

卡在哪：把第 r 行误算为 r×n 会跳错地方：作者沿用了上下左右各一格的空边框，真正行宽是 n+2。以 n=6 为例，第二行第二个内部格子用内部坐标 (2,2) 算得 18；把步长写成 6 就变成 14，仍可能落在合法内存中，所以只看程序有没有崩是不够的。

这一版改了什么：第 35–41 行检查实际读入字节；第 60–63 行让 stride 成为适合表示内存大小的无符号类型，并用两个连续的 uint8_t 容器存两代。第 134–149 行的上下邻分别在 idx-stride 和 idx+stride，左右只差 1。static_cast 是显式类型转换，让编译器检查这类转换是否合法；它并不会自动检查所有乘法是否超界。

选行 • 3-fire_pr.cpp L57–72

```
60    size_t stride = static_cast<size_t>(n) + 2;
62    std::vector<uint8_t> cur(stride * stride, 0);
63    std::vector<uint8_t> nxt(stride * stride, 0);
66        size_t row_offset = static_cast<size_t>(i) * stride;
69            if (p >= end) { std::fprintf(stderr, "input truncated\n"); return 1; }
```

选行 • 3-fire_pr.cpp L131–149

```
132           size_t row_offset = r * stride;
138                   (cur[idx - stride] == 2 ||
139                    cur[idx + stride] == 2 ||
```

结果与下一版的动机：以本题图幅忽略边框估，双代从约 2 GiB 减到 512 MiB。每次内层循环还是只更新一个格子。v4 的主意比换一种更小的整数更进一步：让相邻 64 棵树同时回答同一条真假问题。

> 一句话记住：换布局可以省字节，不会自动省格子数。

1.4 v4 并排放同一件事的答案

三个运算先在一位上试：按位与（bitwise AND，逐位取交集）在两边皆为 1 时得 1；按位或（bitwise OR，逐位取并集）在任一边为 1 时得 1；按位取反（bitwise complement，把每位 0 和 1 对调）写作 ~。左移、右移（bit shifts，把无符号数中的位向高位或低位搬）这里才用 <<、>>。uint64_t（unsigned 64-bit integer，六十四位无符号整数）把这套演算放大到一排 64 格。

这一版想干什么：位运算（bitwise operation，对一个字中每一个二进制位置独立做逻辑运算）在这里有一个直观用途：一行 64 格的问题“哪些格是火”只要一对字就能回答。作者不给每格写两位相邻的编码，而是把同一种属性的 64 个位放在同一字中。

卡在哪：旧图和新图各两张位面，另有第五张火图。它免去四邻取火时反复解码旧状态，却引入整图的写入和再读取。位宽 64 也带来两个新接缝：每行未必刚好用完一个字，左右邻火还能跨字。同一行第 63 列烧到第 64 列合法，某行末列直接烧到下一行首列则不合法。

这一版改了什么：第 62 – 68 行分配四张状态面和一张 fire。第 146 – 155 行用掩码（mask，只挑选指定位的二进制模式）限制回春范围，第 168 行取出全图旧火。第 179 – 185 行把同一字左右移并借相邻字的一位，再用树与邻火的交集筛出新火。没有哪条指令在做一次完整的森林更新；是一小组逻辑式共同完成。

选行 • 4-fire_bit.cpp L58 – 68

```
62      std::vector<uint64_t> b1(total_words, 0), b0(total_words, 0);
63      std::vector<uint64_t> nb1(total_words, 0), nb0(total_words, 0);
64      std::vector<uint64_t> fire(total_words, 0);
67      const uint64_t b1bit[4] = {0, 1, 1, 0};
68      const uint64_t b0bit[4] = {0, 0, 1, 1};
```

选行 • 4-fire_bit.cpp L166 – 192

```
168             fire[k] = b1[k] & b0[k];
179                 uint64_t L = (f << 1) | (j > 0 ? fire[idx - 1] >> 63 : 0ULL);
184                 uint64_t tree = b1[idx] & ~b0[idx];
188                 nb0[idx] = b0[idx] | burn;
192         std::swap(b0, nb0);
```

结果与下一版的动机：现在状态图缩小，单线程仍要把所有行从头扫到尾。v5 保留位面数学，把没有互相写入依赖的循环迭代交给多条线程。

> 一句话记住：把一条规则同时问 64 个位置。

1.5 v5 两次派工，先提取火，再更新树

并行指令（compiler directive，写给编译器的特殊指示）#pragma omp parallel for 把后面的循环交给线程团队分摊；schedule(static) 指静态调度（static scheduling，预先分配迭代范围）。这两个词决定谁算，不重新定义算什么。

这一版想干什么：共享内存允许几条线程读同一张旧图。写入却必须先划界：同一地址不能让两个线程各自随意改。这里按字生成火图，再按行更新新图，两段各自的输出下标天然不同，适合分工。

卡在哪：第一个循环的一段火图可能会被第二个循环的另一个线程读取。你不能仅凭‘我负责的行写好了’就开始第二段：边界那一行还需要别人写的火。每步需要两次进入并行区（parallel region，由一个线程团队共同执行的代码区域），循环之间也必须有先后。

这一版改了什么：第 159、166 行的 parallel for 是‘建立团队并分配循环’的组合指令。schedule(static) 是静态调度（static scheduling，预先分配迭代范围）；不用每处理一个字就去向全局队列取下一个字。第 184 – 185 行在两段计算之后交换两张位面。

选行 • 5-fire_bit.cpp L156 – 174

```
159         #pragma omp parallel for schedule(static)
161             fire[k] = b1[k] & b0[k];
166         #pragma omp parallel for schedule(static)
173                 uint64_t L = (f << 1) | (j > 0 ? fire[idx - 1] >> 63 : 0ULL);
174                 uint64_t R = (f >> 1) | (j + 1 < W ? fire[idx + 1] << 63 : 0ULL);
```

选行 • 5-fire_bit.cpp L175 – 186

```
175                 uint64_t nbr = L | R | fire[idx - W] | fire[idx + W];
178                 uint64_t burn = tree & nbr;
180                 nb1[idx] = tree;
181                 nb0[idx] = b0[idx] | burn;
185         std::swap(b0, nb0);
```

结果与下一版的动机：假如每次进出团队都计一次，2000 步对应 4000 次并行区进入；这不是操作系统一定创建了 4000 套线程，运行库可以复用线程。v6 要省的是反复进出并行区的组织工作。

> 一句话记住：先看清依赖，再给循环派人。

1.6 v6 让团队包住时间，而不是让时间包住团队

single（single construct，单线程执行构造）让团队任选一个成员执行结构块；它在这里末端自带会合。团队是一起到达同步点的所有参与者，和‘某时刻恰好在计算的人’不一定一样。

这一版想干什么：同一队人完成所有时间步，但没有取消工作之间的依赖。栅栏（barrier，一队线程全部到齐才能进入下个阶段的同步点）让快线程不能先交换旧图。分清‘一人做’和‘别人等到它做完’是读这一版的钥匙。

卡在哪：事件阶段不能让每人把同一条事件再消费一次；只让一人处理，但没做事的人必须看到事件改过的地图。火平面的生产和消费同样隔着一次同步。全图 fire 仍需约 32 MiB，大图每步至少要写完它，下一阶段又从它读。

这一版改了什么：第 116 – 122 行把团队移到步循环外；single 表示团队任选一人进入，不保证永远是编号 0 的人。第 171、178 行在已有团队内分派两个 for，没有再建立两个新团队；第 199 – 205 行派一人交换。第 162、166 行已经把事件耗时和转移计时起点分开，不能把这项改动记在 v7 名下。

选行 • 6-fire_bit.cpp L116 – 126

```
116     #pragma omp parallel
122             #pragma omp single
```

选行 • 6-fire_bit.cpp L162 – 182

```
171             #pragma omp for schedule(static)
173                 fire[k] = b1[k] & b0[k];
178             #pragma omp for schedule(static)
```

结果与下一版的动机：v7 问的不是能否把同一循环再加一个并行指令，而是能否根本不建大家共享的火图。每个线程只需要自家行段的上一行、当前行、下一行。

> 一句话记住：团队可以常驻，阶段之间仍得等。

1.7 v7 只把三行火带到手边

这一版想干什么：把第 r 行想象成书桌中间的纸：上面摊 r−1，下面摊 r+1。算完中间这行，就把中间移到上方、下面移到中间，再取一张新纸。这是火的临时视图，并不意味着整张旧森林已经被覆盖。

卡在哪：线程分界不能当森林边界。若一个线程从第 9 行起算，第 8 行也许属于隔壁线程，但它的旧火仍能点燃第 9 行。它可以直接读共享旧图，不必请邻线程把自己的私有窗口借出来。对方只写新图，因此不会在你读到一半时改掉第 8 行。

这一版改了什么：第 113 – 122 行按线程编号分行并准备三个私有容器。第 169 – 185 行在行段开始时装好窗口。第 191 – 200 行算四邻和两张新面，第 203 – 204 行轮换三个行容器，第 209 – 218 行等所有人写完，再由一人交换全图。题面编译命令没有显式启用 OpenMP；算法能分行，不等于评测程序真的会并行。

选行 • 7-fire_final.cpp L111 – 124

```
111     #pragma omp parallel
120         std::vector<uint64_t> f_prev(W, 0);
121         std::vector<uint64_t> f_curr(W, 0);
122         std::vector<uint64_t> f_next(W, 0);
```

选行 • 7-fire_final.cpp L188 – 205

```
190                         uint64_t f = f_curr[j];
192                         uint64_t R = (f >> 1) | (j + 1 < W ? f_curr[j + 1] << 63 : 0ULL);
196                         uint64_t tree = b1[idx] & ~b0[idx];
200                         nb0[idx] = b0[idx] | burn;
204                     std::swap(f_curr, f_next);
```

结果与下一版的动机：一整行宽 256 字，即 2048 字节。每线程的窗口约 6 KiB；16 个线程约 96 KiB，和一整张约 32 MiB 的火图不可同日而语。这是容量推算，不是缓存命中率或秒数。下一章沿真实的 261 行把这个结论拆开。

> 一句话记住：私有窗口可以越过分工边界读旧图，不能越界写新图。

CHAPTER 02

第 2 章　v7 逐行精读：一片火如何跨过第 64 列

这一章你会得到什么：能从一枚字符追到一个字中的一位，再追到下一拍、下一行和输出文件。代码框保留原文件物理行号；每条解释里的短代码是定位标记，整行以代码框为准。看到空行、括号时也标明它分隔哪两件事。

第 1–25 行 · 先把时钟和入口接好

这段在干什么：这段只建立计时方法并检查两个文件名，不碰森林。程序启动后拿到的是一组字符串，两个路径由启动者给，不能假定运行目录就是输入所在处。

先认写法：先认三种标点。#include（header inclusion，包含头文件）让后面的声明可见；::（scope resolution，作用域限定）告诉编译器去哪个命名空间或类型找名字；{...}（lambda expression，匿名函数表达式）做出一个可调用的对象。它们分别回答‘声明从哪来’‘名字属于谁’‘这段操作如何被反复调用’。

代码块：

```
#include <iostream>
#include <vector>
#include <chrono>
#include <cstdio>
#include <cstdint>
#include <cstddef>
#include <omp.h>
#include <algorithm>

namespace {
    using Clock = std::chrono::steady_clock;
}
```

```
auto ms_since = [](Clock::time_point t0) {
    return std::chrono::duration<double, std::milli>(Clock::now() - t0).count();
};

int main(int argc, char** argv) {
    auto t_all = Clock::now();
    auto t0 = Clock::now();

    if (argc < 3) {
        std::cerr << "用法: " << argv[0] << " <输入文件> <输出文件>\n";
        return 1;
    }
```

```

```

逐行讲解

第 1 行 · `#include <iostream>`

输入输出流（input/output stream，按顺序收发字符的对象）来自 iostream。这里后续只需要错误流 cerr；头文件名里的 io 不表示地图一定会用 cin 读。没有直接声明就依赖其他头偶然包含它，不稳妥。

第 2 行 · `#include <vector>`

向量（vector，能管理一段连续元素的容器）由这个头文件提供。后面的 `vector<char>` 和 `vector<uint64_t>` 是同一模板配两种元素，前者数的是字符，后者数的是字，不可把容量单位混用。

第 3 行 · `#include <chrono>`

计时库（chrono，表示时刻和时长的标准库）给出后面所需的时钟与单位。只有两个时刻做差才是时长；直接拿某个时刻当毫秒数没有这里需要的含义。

第 4 行 · `#include <cstdio>`

标准 C 文件接口（C standard I/O，以文件流指针读写的接口）在 C++ 里通过 cstdio 声明。它让读写操作明确给出元素大小和个数，后面的短读检查就靠这个返回约定。

第 5 行 · `#include <cstdint>`

固定位宽整数头文件 cstdint 声明 uint64_t 等类型（在支持精确位宽的目标上）。后面的 63、64 都是根据整字恰好 64 位设计的，随手改成 unsigned int 会破坏跨字逻辑。

第 6 行 · `#include <cstddef>`

大小类型（size type，能表示对象大小的无符号整数类型）由 cstddef 等头提供，标准名字是 std::size_t；本文件用未限定的 size_t。不能从‘无符号’推导出所有输入都安全，负数转换后可能变成很大的正数。

第 7 行 · `#include <omp.h>`

共享内存运行库的声明来自 omp.h。第 113、114 行普通函数会查团队大小和编号；仅能找到头文件不保证产生并行代码，也不保证链接器能找到函数定义。

第 8 行 · `#include <algorithm>`

算法头 algorithm 声明取小值 min 和区间填充 fill。后者在第 174、185 行专门清除边沿的旧火，和默认构造时只清零一次不是一回事。

第 9 行

空行把头文件与本文件自己的名称隔开；不参与编译逻辑，删掉不改变结果。

第 10 行 · `namespace {`

命名空间（namespace，给名字划出作用范围）在这里没有名字，内部声明只供这个翻译单元使用。它不是函数，不制造线程，不需要像定义类那样在闭括号后放分号。

第 11 行 · `using Clock = std::chrono::steady_clock;`

类型别名（type alias，给现有类型换短名）用 using 创建。steady_clock 是单调时钟，适合算经过时长，不能把它的读数当作今天几点；后面改用 Clock 时指的仍是同一类型。

第 12 行 · `}`

右花括号终止匿名命名空间；短名仍可在本文件后续使用。若把范围一直包到 main，入口的放置就变了，不能把括号当随意缩进。

第 13 行 · `auto ms_since = [](Clock::time_point t0) {`

匿名函数（lambda expression，现场定义的小函数对象）写作 {...}；空方括号表示不捕获外部局部变量。auto 让编译器推导对象类型，t0 是按值传入的时刻，外部起点不会被改。

第 14 行 · `return std::chrono::duration<double, std::milli>(Clock::now() - t0).count();`

Clock::now() 取当前时刻，减起点得时长；duration<double,std::milli> 将计数的单位定为毫秒，count() 取数值。若直接取原时长的 count，单位未必是毫秒，标签相同也不能比较。

拿数字验一下：手边有秒表时可以对照：相差 0.125 秒的两个时刻，经这个 duration 转换后 count() 应是 125，而不是 0 或 125000。double 允许表达不足一毫秒的部分，后面 %.1f 只是显示时再舍入。

第 15 行 · `};`

花括号结束函数体，分号结束第 13 行的变量定义。不要照普通函数定义的写法把分号省掉；这里保存的是可以在后面多次调用的对象。

第 16 行

空行把计时小工具与程序入口隔开。它不清空时钟，也不让任何变量改变。

第 17 行 · `int main(int argc, char** argv) {`

main 是进程入口；argc 是参数个数，argv 是指向字符串指针的指针（pointer，保存地址的值）。例如 forest in out 一共三个参数，argv[0] 是程序名，两个路径从下标 1 取。

第 18 行 · `auto t_all = Clock::now();`

用自动类型推导保存总起点。这个时刻后面不会重置，第 258 行用它算整次经过时间；若拿可重置的局部起点代替，total 只剩末阶段。

第 19 行 · `auto t0 = Clock::now();`

再取一个独立起点，读文件从这里算。两个调用很接近但职责不同：第 225 行会复用 t0，不会影响总时钟。

第 20 行

空行将初始化与参数检查隔开；两个时钟继续计时，不存在隐式暂停。

第 21 行 · `if (argc < 3) {`

条件语句 if 只在参数数目不足时进入。条件用小于而非等于，因为缺一个或两个路径都不允许；参数多于所需时源码并未拒绝。

拿数字验一下：把 argv 看成三格：格 0 是 forest，格 1 是 forest.in，格 2 是 forest.out。能访问下标 2 的条件是个数至少 3，因为编号从零开始。这与地图的内部行号从 1 开始完全是两套约定。

第 22 行 · `std::cerr << "用法: " << argv[0] << " <输入文件> <输出文件>\n";`

标准错误流 cerr 的 << 是流插入，把用法、实际程序名、换行依次写给人看。这里的 << 不移动二进制位；输出目的地也不是 forest.out。

第 23 行 · `return 1;`

return 1 从 main 提前返回非零状态，启动者可据此判断未成功。只打印提示不返回，下面仍会取不存在的路径。

第 24 行 · `}`

此括号只结束参数不足分支。正常运行从这里往下走；不是所有路径都会在上面返回。

第 25 行

空行标志‘已具备访问两个文件名的条件’；它并不表示这两个路径一定可读可写。

读代码前先排一条时间线。 用三张纸分别写“名字”“起点”“时长”。在 forest.exe forest.in forest.out 里，名字那张纸上有三个字符串。程序读 t_all 的时候连输入能否打开都不知道，这时不能填写“读了几毫秒”。等到第 102 行，会发生的是拿新时刻减起点，不是从机器那里取一个叫 read 的硬件计数器。

看到的拼法	在这个例子中问什么	不应据此推断什么
Clock::time_point	某个时刻在单调时钟上的位置	它是日历几点钟
Clock::now()-t0	两个时刻相隔多久	计数的一个单位一定是毫秒
duration<double, std::milli>(差).count()	把间隔以毫秒为单位读出	比显示的小数位更多就一定测得更准
return 1	这条启动请求因参数不足结束	任意一种参量错误都一定被这份程序发现

从零重写时怎么想：先写出正常和异常两条路。正常：三个字符串齐全，去打开 argv[1]。异常：只有两个字符串，写用法并从 main 返回。让手指从两条路分别走一次，只有第一条能走到输入段。再把时钟放在两条路分开之前，你就知道为什么总起点早于真正文件读取。

这段结束后的状态：有效路径名已经可以取用，两个时刻已保存；输入文件尚未打开。

写错会怎样：参数数量把程序名也算进去。若用 argc<2，只有输入路径时仍会试图把缺失的输出路径当字符串。

和前后版本的关系：v1 也分阶段计时；v7 仍用普通进程入口，不会自动识别 MPI 进程编号。

> 一句话记住：看时钟先找起点，看参数先数程序名。

第 26–55 行 · 从文件读到一对指针

这段在干什么：把整份文本读进可连续访问的内存，再写一个按需取整数的小工具。设文件只有五个字符‘12 3\n’，游标指向 1 时第一次读取的结果应为 12，并停在空格上。

代码块：

```
26      // ========================================
27      // 1. 快速读入
28      // ========================================
29      FILE* fp = std::fopen(argv[1], "rb");
30      if (!fp) { std::fprintf(stderr, "open in failed: %s\n", argv[1]); return 1; }
31
32      std::fseek(fp, 0, SEEK_END);
33      long fsz = std::ftell(fp);
34      std::fseek(fp, 0, SEEK_SET);
35
36      std::vector<char> buf(fsz + 1);
37      size_t got = std::fread(buf.data(), 1, fsz, fp);
```

```
38      if (got != size_t(fsz)) {
39          std::fprintf(stderr, "read failed: expected %ld bytes, got %zu\n", fsz, got);
40          std::fclose(fp);
41          return 1;
42      }
43      std::fclose(fp);
44      buf[fsz] = '\0';
45
46      const char* p = buf.data();
47      const char* end = buf.data() + fsz;
48
49      auto read_int = [&]() {
```

```
50          while (p < end && (*p < '0' || *p > '9')) ++p;
51          int v = 0;
52          while (p < end && *p >= '0' && *p <= '9') { v = v * 10 + (*p - '0'); ++p; }
53          return v;
54      };
55
```

逐行讲解

第 26–28 行

三行纯注释用双斜杠标出输入阶段。它们帮助找位置，不会替程序打开文件或自动开始新的计时。

第 29 行 · `FILE* fp = std::fopen(argv[1], "rb");`

文件指针 FILE* 接收 fopen(路径,模式) 的结果。rb 表示读已有文件并按二进制字节处理，在 Windows 上避免换行转换干扰文件长度。失败会返回空指针，不能接着读。

第 30 行 · `if (!fp) { std::fprintf(stderr, "open in failed: %s\n", argv[1]); return 1; }`

逻辑非 !fp 检查打开失败。fprintf(stderr,格式,参数) 将路径代入 %s 并换行；随后退出让读者知道哪个输入路径有问题，不会误报成短读。

第 31 行

空行分开打开文件与测量长度；句柄 fp 在两段之间保持打开。

第 32 行 · `std::fseek(fp, 0, SEEK_END);`

fseek 的三个参数是文件、偏移、基准；以 SEEK_END 为基准偏移零，定位到文件末端。这里只移动位置而不读数据；返回状态没有被检查。

拿数字验一下：位置图可画成‘开头 0 → 中间 1、2、3 → 末端 4’。四字节文件末端的位置是 4，不是末尾那个字节的编号 3。正因为这个差一，后面可以直接申请 fsz 个有效字符，而不用再给实际长度加一。

第 33 行 · `long fsz = std::ftell(fp);`

ftell(fp) 查询当前文件位置并以 long 返回。此刻在末端，普通可定位二进制文件的位置才可用作字节长度。失败能返回负值，源码没有在分配前拦截。

第 34 行 · `std::fseek(fp, 0, SEEK_SET);`

用 SEEK_SET 作基准、偏移 0，把位置移回开头；不回去会从末尾开始读，读不到刚量出的那些字节。

第 35 行

空行把测量位置与申请内存分离。fsz 的单位仍是字节，文件位置已经归零。

第 36 行 · `std::vector<char> buf(fsz + 1);`

vector<char> 按 fsz+1 个字符分配连续空间。多出的一个位置用于第 44 行的零字符；不是说文件里还应该多读一个字节。

第 37 行 · `size_t got = std::fread(buf.data(), 1, fsz, fp);`

fread(目的地址,每项字节数,项数,文件) 返回读到的完整项数。这里每项是 1 字节，所以 got 和文件长度可直接比较；buf.data() 是首元素地址，不是容器对象本身。

拿数字验一下：试读 5 项，结果返回 3，就只允许把前 3 项称为文件提供的数据。这也是为什么此处先比较 got，再向 buf[fsz] 写哨兵；写了哨兵不能让缺失的两个文件字节重新出现。

第 38 行 · `if (got != size_t(fsz)) {`

把长度转成 size_t 再与实读量比较，意在发现短读。函数式转换 size_t(fsz) 不检查负数；测长本身仍有前提，不能拿类型统一当作输入检查。

第 39 行 · `std::fprintf(stderr, "read failed: expected %ld bytes, got %zu\n", fsz, got);`

格式串的 %ld 匹配 long，%zu 匹配 size_t。把预期和实收的两个数都打印，能区分‘文件打不开’和‘打开后没读齐’。

第 40 行 · `std::fclose(fp);`

短读路径在退出前 fclose(fp)，释放这个文件流。不能指望后面的正常关闭替它办事，因为下一行已经返回。

第 41 行 · `return 1;`

此处返回失败，不拿部分文本当完整森林。少读的部分即便暂时碰巧是零，也不是用户提供的地图。

第 42 行 · `}`

结束短读分支。成功路径从未走第 40 行，所以它还欠一次关闭文件的动作。

第 43 行 · `std::fclose(fp);`

成功时关闭磁盘文件，之后扫描的是内存里的副本。关闭文件不会使 buf.data() 失效，因为容器拥有那段内存。

第 44 行 · `buf[fsz] = '\0';`

写入空字符 '\0' 作尾部哨兵（sentinel，标记边界的特殊值），它与数字字符 '0' 不同。真实扫描仍用 p<end 保护，不能把这行误当成读取了一个空地格。

第 45 行

空行隔开“把文件搬进来”与“开始解释字符”。这以后不需要通过 fp 找格子了。

第 46 行 · `const char* p = buf.data();`

const char* 是指向只读字符的指针：可移动游标 p，不能经 p 覆写字符。起点来自 data()；如果后来让容器重新分配，旧指针可能不再有效。

第 47 行 · `const char* end = buf.data() + fsz;`

end 是首地址加真实字节长度，表示半开区间 [p,end) 的终点；此处刚好是哨兵位置。把它误设为末尾那个有效字符的地址，可能丢掉没有尾换行的末位数字。

拿数字验一下：若文本是 6 空格 1，内存有四个位置：0 是 '6'、1 是空格、2 是 '1'、3 是哨兵。数字 1 的地址小于 end，读完它正好等于 end。此时再读一次，返回零同时不前进，所以非法的缺字段需要调用者另行发现。

第 48 行

空行隔开游标和捕获这两个地址的小函数；不新建作用域。

第 49 行 · `auto read_int = [&]() {`

[&] 按引用捕获（reference capture，通过原变量而非副本访问），无参数调用 read_int() 可移动外面的 p。若每次调用都从旧副本起读，n、m、t 无法依次消费。

第 50 行 · `while (p < end && (*p < '0' || *p > '9')) ++p;`

while 只要条件为真就重复；&& 先验证范围再读 *p，|| 连接“小于 0 字符或大于 9 字符”。++p 前进一个字节。它跳过所有非数字，不能用来正确解析负数。

第 51 行 · `int v = 0;`

每调用一次就从 v=0 开始构造新整数。不能把上一个数字的结果留在 v 里，否则输入 12 空格 3 可能拼成 123。

第 52 行 · `while (p < end && *p >= '0' && *p <= '9') { v = v * 10 + (*p - '0'); ++p; }`

只在范围内且属于数字时做 v=v×10+当前位，再移动指针。例如先见 1 再见 2 得 12。字符相减得到数值；没有检查过大的数字是否超出 int 可表示范围。

拿数字验一下：手算读入“105 ”：v 从 0 变 1，再变 10、105；p 落在空格。中间数字 0 也必须使旧值乘 10，不是看见 0 就略过。文件里的空地字符和作为十进制数字的一位 0，是否独立成项取决于解析位置。

第 53 行 · `return v;`

return v 把本次的数返回给调用者，p 则停在数字后的字符处。若本来就到了末尾，也会返回 0，调用者不能仅凭这个值区分缺字段和合法零。

第 54 行 · `};`

结束 lambda 并用分号完成变量定义。此刻只是定义了解析器，真正的首次调用在第 59 行。

第 55 行

空行留给下一阶段标题。解析器及其引用仍在 main 作用域中有效。

把游标的每一步画出来。 把换行用“换”代替，纸上写五格：位置 0='1'、1='2'、2=空格、3='3'、4=换。位置 5 是哨兵而非文件内容。下表模拟两次 read_int；注意游标移动发生在函数体里，却能被下次调用看见，因为捕获的是原变量。

动作	入口 p	v 的变化	退出 p
第一次跳过非数字	0	还没用 v；第一个字已是数字	0
第一次吃数字	0	0→1→12；看到空格停下	2
第二次跳过非数字	2	跨过一个空格，停在字符 3	3
第二次吃数字	3	0→3；看到换行停下	4
再调用一次	4	跨到 5，没有数字，仍得到 0	5

断点该放哪。 若 n 正确而 m 也被读成 n，不要先怀疑森林逻辑。检查同一地址 p 在连续两次调用之间是否前进；若指针从未越过首位，会重复读，若一口气越过分隔符后的首位，则会吞掉下个字段的一部分。工程里一个“读数函数”其实同时间接返回两样东西：明面上的数，以及留给下一次调用的位置。

这段结束后的状态：文件已关闭；buf 仍然拥有文本。p 从首字节起步，end 永远停在有效区间之外的第一个位置。

写错会怎样：重复读文件并不修复游标错误；如果读了一个数却没移动 p，后面每次都会拿到同一份输入。

和前后版本的关系：v2 已经整块读取，但 v3 才检查 fread 的实际返回量。v8 在解析完事件后尝试归还这块内存。

> 一句话记住：读进多少字节，就只在多少字节的范围内解释字符。

第 56–84 行 · 把一格找回到两个位里

这段在干什么：输入把每格写成 0–3 的字符，这里改用两张位图回答‘树或火吗’和‘火或灰吗’。字有 64 位，位 0 对应该字里列号最小的格子。

代码块：

```
56      // ========================================
57      // 2. 初始化位平面
58      // ========================================
59      int n = read_int(), m = read_int(), t = read_int();
60      int W = (n + 63) >> 6;
61      size_t total_words = size_t(n + 2) * W;
62
63      std::vector<uint64_t> b1(total_words, 0), b0(total_words, 0);
64      std::vector<uint64_t> nb1(total_words, 0), nb0(total_words, 0);
65
66      const uint64_t b1bit[4] = {0, 1, 1, 0};
67      const uint64_t b0bit[4] = {0, 0, 1, 1};
```

```
68
69      for (int i = 1; i <= n; ++i) {
70          size_t row_offset = size_t(i) * W;
71          for (int j = 0; j < n; ++j) {
72              while (p < end && *p < '0') ++p;
73              if (p >= end) { std::fprintf(stderr, "input truncated\n"); return 1; }
74              int v = *p - '0';
75              if (v > 0) {
76                  size_t idx = row_offset + (size_t(j) >> 6);
77                  uint64_t bit = 1ULL << (j & 63);
78                  if (b1bit[v]) b1[idx] |= bit;
79                  if (b0bit[v]) b0[idx] |= bit;
```

```
80              }
81              ++p;
82          }
83      }
84
```

逐行讲解

第 56–58 行

三行注释切到位面初始化。它们没有改变 p，因此标题后的第一次取整数仍从文件头开始。

第 59 行 · `int n = read_int(), m = read_int(), t = read_int();`

顺序读三个整数：n 管行列、m 管事件、t 管模拟步数。三者不能互换；m=1、t=4 就是一张事件单推动四轮，而不是四张单只算一轮。

第 60 行 · `int W = (n + 63) >> 6;`

W 按每行算字数。(n+63)>>6 在本题非负且加法可表示时等于向上取整的 n/64；65 列要两个字，不能只给整除得到的一个。

拿数字验一下：各看三个邻近尺寸：n=63 时 W=1，n=64 时 W=1，n=65 时 W=2。若先做 n/64 再无条件加 1，在 n=64 时白留整整一字；若只做 n/64，在 63 和 65 时又会少分配。

第 61 行 · `size_t total_words = size_t(n + 2) * W;`

分配 (n+2) 行，每行 W 字，真实行在 1..n。先把行数转成 size_t 再做乘法，减少 int 中间乘积过小的风险；n+2 自身仍按 int 算。

第 62 行

空行分开尺寸推算与容器分配。total_words 数的是字，不是字节；下一行的每个元素有 8 字节。

第 63 行 · `std::vector<uint64_t> b1(total_words, 0), b0(total_words, 0);`

每代两张位面：b1 标‘树或火’，b0 标‘火或灰’。构造参数（个数,初值）给每一字填 0。零既编码空地，也让从未写过的边界和填充位保持无火。

拿数字验一下：拿八个输入格 0 1 2 3 1 0 2 0 缩小演算。把第 0 列写在纸最右端，b1 是 01010110，b0 是 01001100。把这两串与起来得到 01000100，只有位 2 和 6 是火，和输入中两个 2 对上。

第 64 行 · `std::vector<uint64_t> nb1(total_words, 0), nb0(total_words, 0);`

nb1、nb0 与当前面同尺寸，但属于下一代。若直接把结果写回 b1、b0，后算的行可能偷看到本步刚生成的新火，破坏同时更新。

第 65 行

空行隔开储存空间和编码规则。名字 nb1 中的 n 指下一代，不是地图边长。

第 66 行 · `const uint64_t b1bit[4] = {0, 1, 1, 0};`

常量查找表（lookup table，用整数下标直接取得预先写好的答案）列出四种输入的高位。原始值 1、2 查得 1，树与火同属 b1，不能简单拿输入值的二进制高位代替。

拿数字验一下：一张查表写在纸上：输入值 → 高面回答：0→否、1→是、2→是、3→否。‘高面’只是我在这张卡上给 b1 的位置，不能推出题面数字 1 的高二进制位也是 1。要先约定编码，再对外输出。

第 67 行 · `const uint64_t b0bit[4] = {0, 0, 1, 1};`

第二张表只给输入 2、3 返回 1。和上一行合起来按 b1b0 顺序是 00、10、11、01；这个顺序与 br 的数值二进制并不相同。

第 68 行

空行隔开两张编码表与实际格子读取。此时四张图上仍然都是零。

第 69 行 · `for (int i = 1; i <= n; ++i) {`

外层循环从内部行号 1 走到 n（含 n）。上下各留一行；如果照列那样从 0 写，第一行会落在原本预留的边界位置。

第 70 行 · `size_t row_offset = size_t(i) * W;`

行首偏移是 i×W 字。把行号转换后再乘，以 65 列为例 W=2，内部第 2 行首字下标为 4。这里不是 i×n 个格子。

第 71 行 · `for (int j = 0; j < n; ++j) {`

列号 j 从 0 到 n−1，一开始就对应实际的位编号。存储没有给每一行的左边预留一个位；和第 69 行同样加 1 就会把第 63 列塞进下一字。

第 72 行 · `while (p < end && *p < '0') ++p;`

对合法地图字符只需跳过小于 '0' 的空格或换行。条件先检查 end 才解引用；它不是全字符合法性验证，字母和 '9' 也不会在此被筛掉。

第 73 行 · `if (p >= end) { std::fprintf(stderr, "input truncated\n"); return 1; }`

在解引用前检查 p 是否已经到末端；若格子还没数完，报告文本截断并退出。这个检查不判断字符是不是题目允许的四种状态。

第 74 行 · `int v = *p - '0';`

把一个数字字符与 '0' 相减得到状态。例如 '3'−'0'=3。这里一次仅取一个字符，不能照第 52 行的十进制多位解析方式跨格消费。

第 75 行 · `if (v > 0) {`

非空状态才需设位，因为容器初始为零。这个分支省了空地的两次查表，不能把 v>0 误读成‘v 已在 1 到 3 之间’。

第 76 行 · `size_t idx = row_offset + (size_t(j) >> 6);`

全局字下标＝行首＋列号整除 64。列 63 仍在本行第一字，列 64 才在第二字；同一字中不同格靠下一行的 bit 区分。

拿数字验一下：给 n=65、i=2、j=64，row_offset=4，idx=5，bit=1。两张面都在各自数组的第 5 号字取最低位。换成 j=63，idx=4，bit=1ULL<<63；格子只差一列，字地址和位编号却同时改变。

第 77 行 · `uint64_t bit = 1ULL << (j & 63);`

1ULL 是无符号长长整数字面量，左移 j&63 位形成单一置位掩码。列 64 的低六位是 0，不会出现移位量 64；若用窄的整数 1，高位列便可能出错。

第 78 行 · `if (b1bit[v]) b1[idx] |= bit;`

若高位表在 v 项为真，就对目标字做按位或赋值 |=。只置 bit 对应的一格，不会擦除同一字里先前读过的 63 格；普通赋值 = 则会。

第 79 行 · `if (b0bit[v]) b0[idx] |= bit;`

低位独立查询并置位。火要同时设置两面，所以两个 if 不能接成 else if，不然输入 2 被误码成树。

第 80 行 · `}`

结束‘非空才写’分支。不要把移动 p 的语句也一起包进去：遇到空地时同样必须读掉这个字符。

第 81 行 · `++p;`

前进一步的是字符指针 p 而非列号 j。哪怕本格是空地也要消耗字符；否则后面的循环虽按时结束，却会反复读取同一空地，把后续输入整体错位。

第 82 行 · `}`

结束一行的列循环。列号在下一次外循环会从零重新开始，指针 p 却继续沿整个文件向前，不需要人为跳过固定长度的一行。

第 83 行 · `}`

结束输入地图的行循环。这里恰好完成 n^2 次格子读取，新图没有被写过；接下来的数字属于事件，不应再当状态处理。

第 84 行

空行标出从固定二维形状的格子区切到可变长度的事件区。游标会由通用整数解析器继续使用。

把地图格刻成地址卡。 取 n=65，每行 W=2。设输入零基行 0 的第 0 列是树、63 列是火、64 列是灰，其他都空。请分别列出键（内部行、字下标、位号、b1、b0），再看表。

输入坐标	内部行	全局字下标	位号	这一位写成 b1b0
(0,0)	1	2	0	10
(0,63)	1	2	63	11
(0,64)	1	3	0	01
(1,0)，假设为空	2	4	0	00

下标 2 的高面有位 0 与位 63，低面只有位 63；下标 3 的低面仅位 0。其余 63 个未用位虽然分配了，却不对应真格。初始为零，转移里的 burn 必须由 tree 筛选，而填充位不可能是 tree，所以水平移位偶尔把候选邻火移进去也不会凭空长树。这个推理比直接说“补齐总没事”更可靠。

逆向自查：若只知道一个全局字下标 3，你无法确定唯一一棵树；还需要字内位。反之拿到位号 0，也可能是每行的第 0、64、128 列。程序里 idx 与 bit 总成对出现，正是因为地址里有两个层次。

这段结束后的状态：b1/b0 是初态；nb1/nb0 和两端哨兵行仍全零。p 位于格子区末格之后，可以接着读事件。

写错会怎样：全图一起向上取整不等于每行单独取整。以 n=65 为例，每行需要两个字，第二个字只用一位；下一行必须从新的两个字起步。

和前后版本的关系：v3 一格占一个字节、四侧留边；v4 以后行列的边界处理不再对称。br 的两位含义另外定义。

> 一句话记住：行找字，列找位，编码表决定该置哪一面。

第 85–110 行 · 事件表是一张按时间排好的票夹

这段在干什么：把两种不同长度的事件收齐。一次惊雷有两个坐标，一次回春有四个端点；一张事件单同时留两套位置，但只按类型填写其中一套。

代码块：

```
85      struct magic_event {
86          int time, type;
87          int para1[2]{};
88          int para2[4]{};
89      };
90      std::vector<magic_event> magic(m);
91      for(int i = 0; i < m; i++) {
92          magic[i].time = read_int();
93          magic[i].type = read_int();
94          if(magic[i].type == 1) {
95              for(int j = 0; j < 2; j++) magic[i].para1[j] = read_int();
96          }
```

```
97          if(magic[i].type == 2) {
98              for(int j = 0; j < 4; j++) magic[i].para2[j] = read_int();
99          }
100     }
101
102     double ms_read = ms_since(t0);
103
104     // ============================================
105     // 3. 主循环计算
106     // ============================================
107     double ms_event = 0, ms_move = 0;
108     int cnt_event = 0;
```

```
109     auto t_step = Clock::now();
110
```

逐行讲解

第 85 行 · `struct magic_event {`

结构体（structure，把有关字段组成一种新类型）在 main 内定义，名字为 magic_event。单纯定义结构不会创建 m 个对象，实际存储在第 90 行才申请。

第 86 行 · `int time, type;`

两个 int 分别表示时间与类型。它们虽然同类型却回答不同问题：时间决定何时消费，类型决定本行还要取几个参数。

第 87 行 · `int para1[2]{};`

固定长度数组 int para1[2]{} 给惊雷预留行列两个整数，花括号把元素初始化为零。数组本身不记当前事件种类，必须配合 type 读取。

拿数字验一下：惊雷在文件里为‘4 1 4 5’，存后 time=4、type=1、para1={4,5}，para2 仍为四个零。后续读取哪组参数必须看 type，而不能试图以‘这一组有没有零’辨别，因为 (0,0) 完全是合法坐标。

第 88 行 · `int para2[4]{};`

给矩形预留四个端点，之后按起始行、起始列、终止行、终止列解释。把四个数当成起点和宽高会多选或少选一整片。

第 89 行 · `};`

花括号合上类型定义，分号不可像普通控制块那样省略。这行后面可以用 magic_event 声明对象，但还没填过任何一次事件。

第 90 行 · `std::vector<magic_event> magic(m);`

vector<magic_event> magic(m) 建立 m 个可下标访问的事件对象。用 reserve(m) 只留容量而不产生元素，不能替代此处的按下标填写。

第 91 行 · `for(int i = 0; i < m; i++) {`

for 的三段为初始化、继续条件、每轮更新。i++ 在循环尾递增，不在此处使用旧值；有效下标从 0 到 m−1，m 为 0 时整个循环不进入。

第 92 行 · `magic[i].time = read_int();`

以 magic[i].time 访问第 i 张事件单的时间成员。read_int 会越过前一次留下的空白；若重复使用旧时间而不读，游标和事件内容都会错。

第 93 行 · `magic[i].type = read_int();`

下一整数是 type，和前一个时间必须分两次读取。比如输入‘4 1 4 5’，两个 4 的含义分别是时间、行，不能只靠数值认字段。

第 94 行 · `if(magic[i].type == 1) {`

类型等于 1 时才把本条余下两字段当点坐标。== 作相等判断；赋值运算 = 会改变字段，不能替换这个比较。

第 95 行 · `for(int j = 0; j < 2; j++) magic[i].para1[j] = read_int();`

不带花括号的 for 只反复执行紧跟的赋值语句，j 取 0、1，依次保存 x、y。读一个就停会把 y 留给下一张事件单。

第 96 行 · `}`

结束惊雷的读取支路。类型 2 并不进入它，因此游标仍指向矩形的第一个整数，下一支才能读全四个端点。

第 97 行 · `if(magic[i].type == 2) {`

第二个独立 if 检查回春。合法类型只有 1、2，两次判断只会命中一次；程序未另设非法类型的报错路径。

第 98 行 · `for(int j = 0; j < 4; j++) magic[i].para2[j] = read_int();`

j=0..3，读四次整数写进 para2。这里的 4 是每个矩形所需坐标的个数，和事件总数 m 无关；端点顺序决定后面循环的方向。

第 99 行 · `}`

退出类型 2 的参数读取块。此刻这张事件单的游标已经越过全部有效数字，下一个空格或换行留给 read_int 跳过。

第 100 行 · `}`

结束 m 次装入。事件仍按文件顺序保存，代码没有排序；第 131 行以后能够只带一个游标，依赖题面已给的升序保证。

第 101 行

空行把读取和统计分开。文件实际上早在第 43 行关闭，但读阶段还包含了内存里的解析和格式转换。

第 102 行 · `double ms_read = ms_since(t0);`

ms_since(t0) 的 double 返回值记录读取阶段毫秒数。它包含读取、分配和编解码，不是磁盘带宽的纯测量；以后重用 t0 不改变这个已存的值。

拿数字验一下：要估 IO 是否值得优化，先记下这段包含的项目：打开与测量、整段读入、向量分配、逐格编码、读事件。换硬盘只能直接影响其中一部分；读段耗时减半也不能反推出磁盘读速恰好翻倍。

第 103 行

空行切换到主模拟；也提醒读者 ms_read 不会在每个时间步重新累计。

第 104–106 行

三行注释给主循环命名；它描述接下来一整段，不是‘从此刻起自动并行’。真正的并行指令在第 111 行。

第 107 行 · `double ms_event = 0, ms_move = 0;`

两个 double 累计器初始化为 0。一个累计事件阶段，一个累计转移阶段；写成只声明不赋初值再用 += 就无法从可预期的计数起跑。

第 108 行 · `int cnt_event = 0;`

cnt_event 是共享的‘下一张待消费事件单’的下标，不是当前时间。没有事件的时间步也要做转移，却不应让这个下标加一。

拿数字验一下：事件时间若是 2、5、9，ts=3 时游标为 1，指向时间 5；ts=4 时仍为 1。ts 每拍走一步，cnt_event 只在消费时走一步。两者碰巧数值相等没有任何特殊含义。

第 109 行 · `auto t_step = Clock::now();`

把 t_step 定成时钟的时间点，在团队外声明使后面两个 single 看的是同一个对象。第 129、164 行才设置各阶段使用的真正起点。

第 110 行

空行隔开共享量和团队内部私有量。变量属于哪里由声明位置和并行规则决定，不由空行或名字决定。

用三张事件单验证输入没有串栏。 假设头信息的 m=3、t=5，事件依次是下面三行。先不碰地图，只读记录，把每个数字写到指定成员中；表里不填的字段是另一类事件用的预留空间。

输入事件行	time、type	生效参数	读完后指向
1 1 0 3	1、1	para1=(0,3)	下一张单的时间 3 之前的空白
3 2 0 1 2 4	3、2	para2=(0,1,2,4)	下一张单的时间 5 之前的空白
5 1 2 0	5、1	para1=(2,0)	有效数据末端或尾部空白

现在遮住输入只用事件表来走五轮。第 1 步消费第 0 张单，游标变 1；第 2 步不消费，游标仍 1；第 3 步消费第 1 张，变 2；第 4 步仍 2；第 5 步变 3。末轮再进 while 时，cnt_event<m 为假，代码不应访问 magic[3]。同一个例子能分别检查读阶段的字段长度和算阶段的游标边界。

从零重写时怎么想：先决定数据在什么时候才完整。末尾那个矩形参数未读取之前，不能把读时钟停掉；否则 v1 与 v7 同样的 read 标签就可能一个包含所有事件、另一个只包含格子。

这段结束后的状态：m 张事件单就绪，cnt_event 指向第一张尚未消费的单，读取耗时已有记录。

写错会怎样：如果矩形只取两个参数，两个剩余数字会被误认作下张单的时间和类型。程序未必报错，模拟却会从那一刻开始跑偏。

和前后版本的关系：v1 – v7 使用相似事件结构；v8 把端点取了名字。br 选择按步号直接查表。

> 一句话记住：事件条数、坐标个数和模拟步数分开数。

第 111–140 行 · 按行分工，派一个人放惊雷

这段在干什么：一队线程各有私人的行段和窗口，却共享同一份森林和事件进度。例：n=10，三个线程，分块长度 4，真行分别是 1 – 4、5 – 8、9 – 10。

代码块：

```
111     #pragma omp parallel
112     {
113         int num_threads = omp_get_num_threads();
114         int tid = omp_get_thread_num();
115 
116         int chunk = (n + num_threads - 1) / num_threads;
117         int start_r = tid * chunk + 1;
118         int end_r = std::min(n, (tid + 1) * chunk);
119 
120         std::vector<uint64_t> f_prev(W, 0);
121         std::vector<uint64_t> f_curr(W, 0);
122         std::vector<uint64_t> f_next(W, 0);
```

```
123 
124         for (int ts = 1; ts <= t; ++ts) {
125 
126             // --- 阶段一：事件处理（串行） ---
127             #pragma omp single
128             {
129                 t_step = Clock::now(); // 计时开始
130 
131                 while (cnt_event < m && magic[cnt_event].time <= ts) {
132                     if(magic[cnt_event].type == 1) {
133                         int x = magic[cnt_event].para1[0] + 1;
134                         int y = magic[cnt_event].para1[1] + 1;
```

```
135                         int c = y - 1;
136                         size_t idx = size_t(x) * W + (size_t(c) >> 6);
137                         uint64_t bit = 1ULL << (c & 63);
138                         if ((b1[idx] & bit) && !(b0[idx] & bit)) b0[idx] |= bit;
139                     }
140                     else if(magic[cnt_event].type == 2) {
```

逐行讲解

第 111 行 · `#pragma omp parallel`

预处理指令 #pragma 给编译器附加指示；omp parallel 让团队中的各线程执行随后的结构块。若未启用，对普通循环不能凭这行自动获得多线程；显式运行库调用另有链接要求。

第 112 行 · `{`

左花括号给并行指令划定作用范围。每个参与者都进入同样的代码，下一行读到的线程数相同，编号则各不相同。

第 113 行 · `int num_threads = omp_get_num_threads();`

omp_get_num_threads() 无参数，返回当前团队真实线程数，不是整机核数。算行段要用实际参与者：按设想的 16 人分，实际 4 人到场就会有行没做。

第 114 行 · `int tid = omp_get_thread_num();`

omp_get_thread_num() 返回团队内从 0 开始的编号。它用于分配行段，不是操作系统进程号，也不等于这次 single 的执行者。

第 115 行

空行分开‘知道我是谁’和‘计算我负责哪些行’。这些局部变量在线程间各有一份，不需要上锁读取编号。

第 116 行 · `int chunk = (n + num_threads - 1) / num_threads;`

对正数做向上取整除法。n=10、团队=3，则 (10+2)/3 为 4，每人最多四行。改为直接整除得 3，三人总计只能覆盖九行。

拿数字验一下：n=17、T=4 时 chunk=5，各段为 1..5、6..10、11..15、16..17。一段行数差三行，这是向上取整统一块长的结果，不保证行数差最多一。若末段太短，前面线程仍必须在发布前做完。

第 117 行 · `int start_r = tid * chunk + 1;`

起点 tid×chunk+1 把分段编号转成内部一基行号。tid=0 从 1 起，tid=1 从 5 起；省略 +1 会把第 0 行哨兵当真行。

第 118 行 · `int end_r = std::min(n, (tid + 1) * chunk);`

std::min(a,b) 返回较小者，把末段截在真行 n。三人分十行时第三段理论到 12，必须截到 10。人比行多时，起点可能大于终点，第 169 行处理。

第 119 行

空行把行段上下界和私有的三行缓冲分开。end_r 是闭区间端点，循环会处理编号等于它的那一行。

第 120 行 · `std::vector<uint64_t> f_prev(W, 0);`

每个线程建立长 W 的前一行火图。这里 W 的单位为 64 位字，不是格子数；一条 16384 列的行只用 256 个字。

第 121 行 · `std::vector<uint64_t> f_curr(W, 0);`

f_curr 保存当前行哪些格为火，而不是该行所有状态。只拿它无法区分树与灰，判断树时仍必须回到 b1/b0。

第 122 行 · `std::vector<uint64_t> f_next(W, 0);`

f_next 是下一行的旧火缓冲，不是下一代的森林。字母 next 在 f_next 里指空间方向，在 nb1 里指时间方向，混淆会引入提前传播。

第 123 行

空行结束团队的一次性准备。从下一行起反复使用已分配的三个容器，不是每个时间步都重新申请三次。

第 124 行 · `for (int ts = 1; ts <= t; ++ts) {`

时间步从 1 到 t 且两端都含。每个线程都要经历同样的步数与栅栏顺序；不能让空行段线程自行提前返回，否则别人可能一直等待。

第 125 行

空行分隔循环头与事件阶段。没有事件的时间步也要进入下面的同步区，不能跳过整轮。

第 126 行

纯注释说本阶段串行，意思是一次由团队中一人改共享状态，而不是整个程序终止并行区、销毁所有线程。

第 127 行 · `#pragma omp single`

single 指令从团队中选一人执行接下来的块。没有 nowait 时，末尾有隐式栅栏；它不规定进入块前大家已到齐，也不指定是哪位线程执行。

拿数字验一下：把团队想成三人共看一张事件单：抽到的人执行，另两人从块尾等待。不能推断下一时间步还会抽到同一个人，因此不能把共享事件进度放进某个线程的私有局部变量里，然后指望所有单都被它依次消费。

第 128 行 · `{`

这对花括号的左端圈出单人工作的范围，直到第 165 行才结束。若只把计时语句圈进去，后面的事件循环就会被每个线程执行。

第 129 行 · `t_step = Clock::now();`

选中的线程用 Clock::now() 重置事件阶段起点。计时点属于共享变量；只有一位写，其他人在这段不会读写它。

第 130 行

空行分开计时起点和事件消费。已有的等待或进并行区的成本不自动计入此次事件循环。

第 131 行 · `while (cnt_event < m && magic[cnt_event].time <= ts) {`

先用 cnt_event<m 排除没有单子，再读其 time。&& 的短路性保护数组尾；用 <= 会消费已经到期的单，但看不到当前未来单子后面被错排的早单。

拿数字验一下：拿合法时间 1、4、8 作例，ts=4 时循环消费时间 4，下一次检查时间 8 便停。若文件错排为 8、4，ts=4 时游标面对 8，不会主动越过它去取后面的 4。所以‘小于等于’只是一种容忍，不能替代排序。

第 132 行 · `if(magic[cnt_event].type == 1) {`

检查本张单是不是惊雷。魔法只会对旧图做就地修改；若写到下一代，稍后的转移会盖掉魔法，邻居也看不到刚点燃的火。

第 133 行 · `int x = magic[cnt_event].para1[0] + 1;`

取点的原始行并加一，得到内部真行号 x。输入 (0,0) 对应内部第 1 行；这一次平移绝不是‘向下一个邻居传播’。

第 134 行 · `int y = magic[cnt_event].para1[1] + 1;`

列先也被写成一基 y，这是沿用早期版本的写法。留意下一行随即减回来；位列真正用的是零基下标。

第 135 行 · `int c = y - 1;`

c=y−1 把列还原到 0..n−1。拿 y 直接算掩码，会把每一列的火向右挪一格；最右端可能落入补齐位或邻行。

第 136 行 · `size_t idx = size_t(x) * W + (size_t(c) >> 6);`

把一基行号乘 W，加上零基列号除以 64 得到字下标。例：n=65 时 W=2，原始第 0 行第 64 列在内部下标 3，不是 64。

第 137 行 · `uint64_t bit = 1ULL << (c & 63);`

用 c&63 取字内位置并形成一个仅一位为 1 的掩码。如果整字都置位，一次惊雷就可能同时点到一整排的树。

第 138 行 · `if ((b1[idx] & bit) && !(b0[idx] & bit)) b0[idx] |= bit;`

两次按位与测试同一位，高面有而低面无才是树。逻辑 && 与 ！把两次测试化成一个真假结论；对低面做或赋值使 10→11，灰和空都不变。

拿数字验一下：玩具字的 b1=00101100、b0=00001000。惊雷点位 2：高面该位 1、低面 0，低面变成 00001100；若点位 3，高低原本皆为 1，属于已着火，不能改变。不要对整字做 b1!=0 && b0==0 来代替逐位测试。

第 139 行 · `}`

结束惊雷分支。下一行的 else if 和第 132 行的 if 配对；不能把它接到内层单行的树判定上，否则事件分类就变了。

第 140 行 · `else if(magic[cnt_event].type == 2) {`

若前一类型判断未命中，才检查是否为回春。接下来要处理的是闭矩形，不再把两个坐标当一个点；不合法的类型没有实现第三种语义。

每个变量是谁的。 把一支队伍看作多人围着同一张桌子。桌面放整图和事件进度，每人手边放自己的行号和草稿。‘只读’和‘私有’不是同义：同一张旧图可以同时被许多人读取，仍是共享对象。

变量	有几份	这个阶段允许谁改	如果放错地方
b1、b0	团队一份	事件 single 中被选的一人	每人私有会看见不同森林
cnt_event	团队一份	事件 single 的执行者	若私有且 single 换人，事件进度发生分歧
tid、start_r、end_r	每人一份	各自初始化	若共享，线程可能覆盖别人的行段
f_prev、f_curr、f_next	每人三行	本线程	若共享，人走不同行会覆盖窗口
nb1、nb0	团队一份	每人只写自己的真行	按行独占写才能在发布后组成全图

独立推算一个极端：n=2、团队=5，chunk=1。tid=0、1 有效；其余三个起点为 3、4、5，终点都被截成 2。可以猜到它们不参加行循环，但不能猜它们不进入 single 或 barrier。某个没有真行

的人同样可能被选去放惊雷；放惊雷使用的是全图坐标，与它自己的 start_r 无关。

这段结束后的状态：分工已定；进入类型 1 分支时，目标树按两位编码被点燃。若类型是 2，控制流进入下一段。

写错会怎样：线程编号不是行号。每人拿不到整除时剩下的行就会漏算；反过来把事件也让每人做一遍，既会争用写入又会移乱游标。

和前后版本的关系：v6 使用运行库调度循环；v7 自己算固定行段。v8 的惊雷明确交给线程 0，v7 的 single 不指定编号。

> 一句话记住：谁分到哪些行，与谁执行一次事件是两件事。

第 141–168 行 · 只让矩形里的灰复活

这段在干什么：复活一个区间不是把整个字都设成树。先取出本字与矩形的交集，再从其中挑出旧灰。跨字例：列 62–65 分别选中字 0 的 62、63 和字 1 的 0、1。

代码块：

```
                        int sx = magic[cnt_event].para2[0] + 1, sy =
    magic[cnt_event].para2[1] + 1;
                        int ex = magic[cnt_event].para2[2] + 1, ey =
    magic[cnt_event].para2[3] + 1;
                        int c_sy = sy - 1, c_ey = ey - 1;
                        int j_start = c_sy >> 6, j_end = c_ey >> 6;

                        for (int r = sx; r <= ex; ++r) {
                            size_t row_offset = size_t(r) * W;
                            for (int j = j_start; j <= j_end; ++j) {
                                int lo = (j == j_start) ? (c_sy & 63) : 0;
                                int hi = (j == j_end) ? (c_ey & 63) : 63;
                                uint64_t mask = (lo == 0 && hi == 63) ? ~0ULL : (((1ULL <<
    (hi - lo + 1)) - 1) << lo);
                                size_t idx = row_offset + j;
```

```
                                uint64_t ash = mask & b0[idx] & ~b1[idx];
                                b1[idx] |= ash;
                                b0[idx] &= ~ash;
                            }
                        }
                    }
                    cnt_event++;
                }
                ms_since += ms_since(t_step);

                // 【修正】重新设置计时点，让 move 从事件结束后开始计算
                t_step = Clock::now();
```

```
            }
            // 隐式 barrier

            // --- 阶段二：滚动窗口与转移计算（并行） ---
```

逐行讲解

第 141 行 · `int sx = magic[cnt_event].para2[0] + 1, sy = magic[cnt_event].para2[1] + 1;`

同时取开始行 sx 与开始列 sy，两者先加一。逗号分隔的是同一 int 声明中的两个变量，不是一个坐标对象；矩形坐标顺序与输入一致。

第 142 行 · `int ex = magic[cnt_event].para2[2] + 1, ey = magic[cnt_event].para2[3] + 1;`

同样得到终止行和终止列。终点是包括在内的，不是数组半开区间的尾指针；面积为 $(ex-sx+1)\times(ey-sy+1)$。

第 143 行 · `int c_sy = sy - 1, c_ey = ey - 1;`

两个列端点减一还原零基位列，行端点保持一基。以后处理跨字区间，只有这两个零基列适合用低六位取余。

第 144 行 · `int j_start = c_sy >> 6, j_end = c_ey >> 6;`

两端各自右移六位，得到首字和末字的编号。列 62..65 的结果为 0..1，两个字都要动；用长度除 64 会错过这个四格跨字例。

第 145 行

空行分开算一次的范围与逐行执行的部分。行数不同不会改变这两个列端点所在的字编号。

第 146 行 · `for (int r = sx; r <= ex; ++r) {`

逐行 r=sx..ex，等号保证仅一行高的矩形也执行一次。这里位于 single 内，不会由多个线程同时去改同一矩形。

第 147 行 · `size_t row_offset = size_t(r) * W;`

每换一行重新算行首字偏移。矩形覆盖同样列号但不同行时，绝对下标要相差 W 字；忽略这一项只会反复改同一行。

第 148 行 · `for (int j = j_start; j <= j_end; ++j) {`

在这一行里逐字走到末字，j 在这里是字编号而非列号。每次可以同时决定最多 64 格中哪些灰复活。

第 149 行 · `int lo = (j == j_start) ? (c_sy & 63) : 0;`

条件运算符 ?: 按真假二选一：首字左端从 c_sy 对 64 的余数开始，后续字从位 0 开始。若所有字都用原始左端余数，中间字会漏掉前缀。

拿数字验一下：同一行受选列 62、63、64、65。走到 j=0，lo=62，hi=63，低 62 位全要原样保留；走到 j=1，lo=0，hi=1，剩余高 62 位保持。这是一个矩形，不是要对两个完整字都执行回春。

第 150 行 · `int hi = (j == j_end) ? (c_ey & 63) : 63;`

若当前是末字，右端是 c_ey 的余数，否则可以到位 63。一字内的短矩形同时是首字和末字，两行判断都得生效。

第 151 行 · `uint64_t mask = (lo == 0 && hi == 63) ? ~0ULL : (((1ULL << (hi - lo + 1)) - 1) << lo);`

覆盖 0..63 时取无符号零逐位取反 ~0ULL，得到全 1。其余长度 k=hi−lo+1 小于 64，(1ULL<<k)−1 先造 k 个低位 1，再左移 lo。比如位 2..4 得 00011100；两端不能少算一个。

拿数字验一下：单点区间 lo=hi=3 时，宽度是 1，低位掩码为 1，移到位 3 得 00001000。若把 hi−lo+1 漏写 +1，单点矩形的掩码为零，肉眼看大矩形未必发现这个错误；单格例最好记。

第 152 行 · `size_t idx = row_offset + j;`

将本行起点与当前字编号相加。这里 j 已经是字编号，不应该像第 76 行的格子列那样再除一次 64。

第 153 行 · `uint64_t ash = mask & b0[idx] & ~b1[idx];`

按位与依次保留矩形掩码内、低面为 1、高面为 0 的格子。这才是旧灰位集合 ash；若少了 ~b1，旧火也会被误当成灰复活。

第 154 行 · `b1[idx] |= ash;`

按位或赋值给高面，只把 ash 选中的位置从 0 变为 1。注意要沿用上一行保存的 ash，不能在改完高面后再以新高面重新选灰。

第 155 行 · `b0[idx] &= ~ash;`

低面与 ash 的按位补集，清掉同一组位置的低位，别的位置不动。与前一行合起来实现 01→10；只做前一行会把灰变为 11 即火。

拿数字验一下：玩具掩码已筛出 ash=00010010。这两个位置原本高 0、低 1；高面 |=ash 先变成两位都是 1，然后低面 &=~ash 只在这些位置清零。中间一瞬间的 11 不是一个新的时间步，其他线程到第 165 行后才利用完成的魔法。

第 156 行 · `}`

结束当前行的字循环。到了这一点，矩形这一行的首尾残字和中间整字都已处理；行循环仍可能有下一行。

第 157 行 · `}`

结束矩形的行循环。此时不必再扫描矩形外的区域：逐字合并已经保存了掩码外原有两位。

第 158 行 · `}`

结束类型 2 的处理块。接下来的事件进度要在类型判断之外更新，否则只遇到惊雷时可能忘记消费。

第 159 行 · `cnt_event++;`

后置 ++ 把共享游标移向下一张单，这里的旧值没有被使用。若遗漏，while 条件一直盯同一条到期事件，步号就推进不了。

第 160 行 · `}`

退出所有到期单子的 while 循环。要么单子已经用完，要么下一张是未来的；不能把循环结束误当成所有时间步都结束了。

第 161 行 · `ms_event += ms_since(t_step);`

毫秒累加器 += 把本次事件段加到旧总量中。无事件时也会有检查与计时开销。它不表示所有核在魔法阶段花费的处理器时间之和。

第 162 行

空行分隔累计事件耗时与重启转移计时。两个阶段共用 t_step 变量，但记录的起点并不相同。

第 163 行

这行注释提示为什么要重设起点：若从处理魔法之前开始算 move，同一段工作会同时进入两个指标。v6 第 166 行已有相同操作。

第 164 行 · `t_step = Clock::now();`

给共享时刻重新赋值。转移的读数会包进离开 single、等待其他线程、交换结果等成本，不是只计内层几个位运算的纯耗时。

第 165 行 · `}`

离开 single 的结构块。这里没有 nowait，因此参与团队的线程都必须等这一单人阶段完成后再向前。

第 166 行
这是一条注释而非另一道栅栏指令。真实的等待来自第 127 行 single 的语义，删注释不影响同步，删指令则完全不同。

第 167 行
空行从已完成的魔法切换到邻火计算。所有人接下来读取的都是同一份魔法后快照。

第 168 行
纯注释指明进入并行转移。不是每一行都有独立线程；下面每名线程自行走其固定行段，行段之间可以同时进行。

一张矩形掩码的账。 为了在纸上画满一字，用八位玩具字。假设当前 b1=01011000，b0=00111010，回春只覆盖位 1..4，即 mask=00011110。从右往左数位 0..7。不要先改图再找灰，那会把证据改掉。

纸上量	八位结果	读成一句话
~b1（只看这八位）	10100111	旧高面为零的候选
b0 与 ~b1	00100010	旧灰在位 5 与位 1
再与 mask	00000010	只复活位 1；位 5 在矩形外
新 b1	01011010	在位 1 设高位
新 b0	00111000	在位 1 清低位

如果把第 154、155 行分别看成两条独立魔法，很容易误以为中间新出现了火。实际上 ash 取自同一份旧状态，两次写共同实现一次状态修改；其间没有任何线程应该启动状态转移。回到 64 位版本再检查一个整字矩形：lo=0、hi=63，计算长度 64 的普通分支不执行；取全 1 不是微优化，而是避免无效移位的语义必要条件。

遮住源码重写：先写出“只改变谁”的布尔条件，再写“该状态的两位由几变几”。不先求出共同的 ash，后面想分别修改两面时很难保住‘选的还是同一批格子’。

这段结束后的状态：本步魔法的全部写入已结束。工作组通过 single 末端同步后才读邻居；t_step 指向转移阶段的起点。

写错会怎样：把 1 移 64 位不是得到一个‘第 65 位’；对 64 位操作数，这种移位未定义。整字覆盖必须另走全 1 掩码分支。

和前后版本的关系：v4 已处理整字特例；v6 的计时起点也已经重置。v8 的主要变化是矩形的各行可由不同线程负责。

> 一句话记住：先挑出矩形里的旧灰，再同时改它的两面。

第 169–188 行 • 启动窗口：分界不是森林边缘

这段在干什么：线程开始一段时没有可继承的上一行窗口，要从共同的旧状态恢复。假设我负责 5 - 8 行，准备 5 时，要读旧的第 4、5、6 行，而不是把第 4 行当成地图外。

代码块：

```
169             if (start_r <= end_r) {
170                 if (start_r > 1) {
171                     size_t offset = size_t(start_r - 1) * W;
172                     for(int j = 0; j < W; ++j) f_prev[j] = b1[offset + j] & b0[offset + j];
173                 } else {
174                     std::fill(f_prev.begin(), f_prev.end(), 0);
175                 }
176 
177                 size_t offset = size_t(start_r) * W;
178                 for(int j = 0; j < W; ++j) f_curr[j] = b1[offset + j] & b0[offset + j];
179 
180                 for (int r = start_r; r <= end_r; ++r) {
```

```
181                     if (r < n) {
182                         size_t next_offset = size_t(r + 1) * W;
183                         for(int j = 0; j < W; ++j) f_next[j] = b1[next_offset + j] &
    b0[next_offset + j];
184                     } else {
185                         std::fill(f_next.begin(), f_next.end(), 0);
186                     }
187 
188                     size_t curr_offset = size_t(r) * W;
```

逐行讲解

第 169 行 · `if (start_r <= end_r) {`

只让确有真行的线程初始化和计算窗口。若 n 小于线程数，start_r 可能大于 end_r；空闲者跳过计算，却不能跳过第 209 行的共同等待。

第 170 行 · `if (start_r > 1) {`

如果自己的首行大于 1，则有真实上邻。这里比较的是全图首行，不是问‘我是不是自己行段的第一行’，后者对每个线程总为真。

第 171 行 · `size_t offset = size_t(start_r - 1) * W;`

取 start_r−1 的行首字偏移，读取的是别的行段也可能负责的旧图区域。只要大家都不改旧图，这种共享只读访问不需要给每个字加锁。

第 172 行 · `for(int j = 0; j < W; ++j) f_prev[j] = b1[offset + j] & b0[offset + j];`

遍历这一行的 W 个字，对高、低面逐位取交集。11 才是火，所以 f_prev[j] 存的每一位是在回答上一行该列‘是否有火’，而不是原来的状态数字。

拿数字验一下：若第 4 行的两面分别是 01010110、01001100，则 f_prev=01000100。对第 5 行来说，它表示第 2 列和第 6 列各有一个来自正上方的火源。行方向不用移位，因为同一列的位编号没有变化。

第 173 行 · `} else {`

else 接住全图第一行的情况。即使有人调整了线程数，这个分支仍只表示没有上邻，不表示没有上一位线程。

第 174 行 · `std::fill(f_prev.begin(), f_prev.end(), 0);`

fill(起迭代器,尾迭代器,值) 用 0 覆盖整个前行缓冲；迭代器是用来走容器位置的对象，end 不包含在区间内。复用窗口时不能只依赖初始为零。

第 175 行 · `}`

合上初始化上一行的分支。无论走哪条路，f_prev 的每一字都已经定义，后面的无条件四邻或运算才有同一含义。

第 176 行

空行转向当前行的初始化。上边缘可填零，但当前行一定是真行，因为第 169 行已过滤掉空行段。

第 177 行 · `size_t offset = size_t(start_r) * W;`

计算本线程第一条真行的旧图偏移。这里的 offset 与第 171 行的同名局部变量不冲突，前者所在的 if 块已结束。

第 178 行 · `for(int j = 0; j < W; ++j) f_curr[j] = b1[offset + j] & b0[offset + j];`

将本段第一行解码为火位填入 f_curr。若错误地直接把 b1 放进去，所有树也会变成邻火来源，森林一拍就会异常扩散。

第 179 行

空行切开只在行段开头做一次的装载和即将重复的滚动。每行都重建三个数组虽可能算对，却失去窗口的复用意义。

第 180 行 · `for (int r = start_r; r <= end_r; ++r) {`

行 r 从段首走到段末，每次向下移动一行。上下界包括在内；其他线程也在做同样的循环，但它们写的是不相交的行。

第 181 行 · `if (r < n) {`

只要不是全图末行，就有真实下邻。假设本段到 8、全图到 10，算 8 时仍得读 9；用 end_r 作判断会在第 8 行造出假边界。

第 182 行 · `size_t next_offset = size_t(r + 1) * W;`

用 r+1 乘 W 得到下一行在旧面中的位置。若移到 r−1，此处会把上邻装两遍并永久漏掉下邻。

第 183 行 · `for(int j = 0; j < W; ++j) f_next[j] = b1[next_offset + j] & b0[next_offset + j];`

逐字计算下一行 fire 并覆盖 f_next。这个容器可能刚从上一轮的 f_prev 换回来，所以每一字都要赋值，不能将新旧火用 |= 合并。

拿数字验一下：设出发时 f_prev 装第 4 行、f_curr 装第 5 行，算第 5 行前在这里读入第 6 行。算完交换后，第 6 行已经是 f_curr，算它时只需把第 7 行读进 f_next；这就是为什么循环每次只新解码一行。

第 184 行 · `} else {`

else 只在真实底行执行，因为地图外没有下一行可以传播火。当前线程末行不一定是全图底行。

第 185 行 · `std::fill(f_next.begin(), f_next.end(), 0);`

把 f_next 整行清成无火。若末行还沿用上一轮留下的位，底边会像接了一行看不见的着火森林。

第 186 行 · `}`

合上‘取真下邻或补零’的分支。从这里起，三个窗口均就绪，内层计算不必对每个格子的上下方向判断越界。

第 187 行

空行区分装载邻行与写本行。只有前一个阶段使用 r+1 作为下邻，写回目标一直属于 r。

第 188 行 · `size_t curr_offset = size_t(r) * W;`

算当前行的全图字偏移，稍后定位目标图与当前两面。它与 f_curr 不同：后者本来只是一行，按 j 取即可。

把窗口贴成三张卡，再移动两次。 假设 n=8，线程专管真行 3..5。开头 f_prev 来自第 2 行，f_curr 来自第 3 行。沿代码读出下面每一轮窗口的内容；表中数字是旧图的行号，不是火的数量。

准备更新的行	prev	curr	新装入 next	刚写的是谁
3	2	3	4	新图第 3 行
4	3	4	5	新图第 4 行
5	4	5	6	新图第 5 行

走到第 5 行时，这个线程手里已经有第 6 行的旧火，但它不应顺便写第 6 行。否则和其他线程的任务重叠。读权限的范围是负责行段再向上下各伸一行，写权限的范围却恰好等于行段。若全图只有一行，则 prev 和 next 同时补零；小例能检验边界分支，但不能替代跨线程分界的例子。

为什么只需存火，不需存所有邻居的状态：目标格只有两种询问。它自己是什么，用本格原来的 b1、b0 直接回答。它能否被点燃，需要的只是邻居是不是火，不关心它是树、灰还是空中的哪一种非火。把邻居压成一个真假并没有丢掉转移所需的信息。

这段结束后的状态：进入内层字循环前，三个数组分别对应 r−1、r、r+1 的火；全图边沿用全零代替缺少的行。

写错会怎样：把 r< n 误写成 r<end_r，会在人为分工接缝上造一条防火线。两条相邻线程都可能算出一行不易察觉的错格。

和前后版本的关系：v4 – v6 用共享火平面的上下两行。v7 在每个线程里自己解码，不需要等待别人先写火图。

> 一句话记住：只在地图边沿补零，线程边沿照读邻行。

第 189–221 行 • 把四邻火投到树上，等全图写完

这段在干什么：看一字的 64 格。左、右运算不是让旧火改变位置，而是在目标格上标记‘这个方向有火’。上、下旧火行已经和目标行列号对齐，不需要移位。

代码块：

```
189                     for (int j = 0; j < W; ++j) {
190                         uint64_t f = f_curr[j];
191                         uint64_t L = (f << 1) | (j > 0 ? f_curr[j - 1] >> 63 : 0ULL);
192                         uint64_t R = (f >> 1) | (j + 1 < W ? f_curr[j + 1] << 63 : 0ULL);
193                         uint64_t nbr = L | R | f_prev[j] | f_next[j];
194 
195                         size_t idx = curr_offset + j;
196                         uint64_t tree = b1[idx] & ~b0[idx];
197                         uint64_t burn = tree & nbr;
198 
199                         nb1[idx] = tree;
200                         nb0[idx] = b0[idx] | burn;
```

```
201                     }
202 
203                     std::swap(f_prev, f_curr);
204                     std::swap(f_curr, f_next);
205                 }
206             }
207 
208             // 【显式栅栏】等待所有线程计算完毕
209             #pragma omp barrier
210 
211             // --- 阶段三：交换全局指针 ---
212             #pragma omp single
```

```
213             {
214                 std::swap(b1, nb1);
215                 std::swap(b0, nb0);
216                 ms_move += ms_since(t_step); // 此时 t_step 精确表示 move 阶段的耗时
217             }
218             // 隐式 barrier
219         }
220     }
221 
```

逐行讲解

第 189 行 · `for (int j = 0; j < W; ++j) {`

j 依次取当前行第 0 到 W−1 个字。若误用 j<n，这里会把格子个数当字个数，远远越过只装一行的窗口。

第 190 行 · `uint64_t f = f_curr[j];`

把当前字的火复制到局部整数 f，后续两个方向都用这份旧火。它是 64 个真假值，不是‘本行共有多少火’这个计数。

第 191 行 · `uint64_t L = (f << 1) | (j > 0 ? f_curr[j - 1] >> 63 : 0ULL);`

f<<1 使位 k 的火标到候选目标 k+1；上一字最高位右移 63 后落在本字位 0。两者按位或合并。j=0 时左边是地图外，不能去取 f_curr[-1]。

拿数字验一下：把位序画成左字 [63 … 0]、右字 [63 … 0]，注意书写时最高位在左、实际列号越大位越高。若左字只在位 63 有火，右字 f=0，(f<<1) 本身为零；补入那一位才让右字位 0 知道左方着火。

第 192 行 · `uint64_t R = (f >> 1) | (j + 1 < W ? f_curr[j + 1] << 63 : 0ULL);`

镜像方向：f>>1 把旧火标到较小列；下一字的最低位左移 63，以它补本字最高位。例：列 64 有火能点第 63 列，而不会从行末绕到下一行。

拿数字验一下：反向检验：若 j 是一行的末字，j+1<W 为假，就算内存里紧跟着下一行的首字，也必须补 0。否则地图末列会被下一行首列点燃，这两个格子并不是水平邻居。

第 193 行 · `uint64_t nbr = L | R | f_prev[j] | f_next[j];`

四张对齐到目标列的候选图按位或，有任一方向来火就标 1。没有把 f 本身直接加进来，也没有对角线；本格是否是树留给第 196 行。

拿数字验一下：以八位演算，当前火 00000100，左右投影分别为 00001000 和 00000010；上方火为 00100000、下方无火，则候选为 00101010。它只说这三个目标旁有火；拿旧树再交一次才决定真正新火。

第 194 行

空行分开‘周围有没有火’与‘这里能不能点燃’。邻火掩码里标上灰或空地是允许的，下一步会筛掉。

第 195 行 · `size_t idx = curr_offset + j;`

把当前行起点与字编号 j 相加，同时定位两张旧面和两张新面。所有面采用相同布局，否则表达式写得再对也会把两处格子拼成一个状态。

第 196 行 · `uint64_t tree = b1[idx] & ~b0[idx];`

`tree = b1 & ~b0` 只选 10；~ 是逐位翻转而不是逻辑非 !。后者只产生一个布尔值，把整个字除最低位外的信息都丢掉。

第 197 行 · `uint64_t burn = tree & nbr;`

tree & nbr 把能着火的旧树与至少有一邻火的目标相交，burn 只代表新点燃的格。旧火不能再算作新点燃，因为这一步它要变灰。

第 198 行

空行提醒读者目前全是局部推导，旧图没有被改。下面才分别写新一代的两个问题的答案。

第 199 行 · `nb1[idx] = tree;`

新高面等于所有旧树：未燃的仍是树，刚燃的成为火，这两种新状态高位都是 1。旧火这一拍变灰，高位恰好应清零，故不能复制旧 b1。

拿数字验一下：拿一列验真值：空地 00 → 00；旧树 10、无邻火 → 10；旧树 10、有邻火 → 11；旧火 11 → 01；旧灰 01 → 01。五行中输出高位都是‘输入是不是树’。这就是能直接赋 tree 的推导，不用背简化公式。

第 200 行 · `nb0[idx] = b0[idx] | burn;`

新低面等于旧低面或新点燃集合。旧灰仍灰、旧火变灰都保留低位 1，新火也需要低位 1。若只写 burn，旧灰会被意外抹成空地。

第 201 行 · `}`

合上一个个字的循环。现在只保证当前行的新状态已经写完，不代表别的行或别的线程都已结束。

第 202 行

空行把本行写回与滚动窗口的轮换分开。此刻还不能交换全局两代图。

第 203 行 · `std::swap(f_prev, f_curr);`

交换 f_prev、f_curr 两个容器的内容归属。以 A、B、C 分别记旧的上、中、下，执行后变成 B、A、C；不必按 W 个字复制一行。

第 204 行 · `std::swap(f_curr, f_next);`

再交换中、下，得到 B、C、A：旧中成新上，旧下成新中，旧上的空间下轮重填。把两次交换顺序颠倒会让窗口反向轮换。

第 205 行 · `}`

合上本线程逐行循环。所有自己负责的真行已经写到 next 面；暂时仍不能在别的线程面前发布新图。

第 206 行 · `}`

合上非空行段的判断。连没有分到行的线程也会抵达下方等待，这才能使参与者人数与建队时一致。

第 207 行

空行结束并行计算正文。它没有同步作用；如果没有后面的指令，快线程可以继续跑。

第 208 行

纯注释提示显式等待的位置。它和第 166 行一样不生成指令，但这次下一行确实有一条单独的栅栏指令。

第 209 行 · `#pragma omp barrier`

#pragma omp barrier 要求团队全部到达，并让等待前的结果在同步后可见。如果只有 single 而没有这道入口前的等待，快线程可能在慢线程写完前交换容器。

拿数字验一下：做个时序实验：甲负责行 1–4，刚写完就把 b1 与 nb1 换位；乙还在算行 7，要从旧 b1 读行 8。若乙这时读到的是新面，甚至可能把旧低面和新高面拼成并不存在的状态。两人不写同一行也会出错，原因在共用容器的角色发生了变化。

第 210 行

空行标志从各写各行过渡到一次性的全局交换。有了上一行的等待，这里才能把整张下一代当作完整结果。

第 211 行

注释里的‘交换指针’是直观简称；实际对 std::vector 使用 swap，不去修改某个格子的指针，也不逐字复制。

第 212 行 · `#pragma omp single`

single 只选一人交换全图。若每个人都交换，偶数次可能又回到旧图，更糟的是并发修改同一容器没有正确同步。

第 213 行 · `{`

开启交换和累计计时共用的单人块。只保护其中一个位面的交换会导致两面来自不同时间。

第 214 行 · `std::swap(b1, nb1);`

把高面的新旧容器互换，原来的旧存储供下一轮覆盖。到这行两面暂时不同代，因此需要把下一行也留在同一个发布阶段。

第 215 行 · `std::swap(b0, nb0);`

同样交换低面，完成完整一代的发布。不必清零旧面，因为下一步转移对每个真行的每一字都做完整赋值。

第 216 行 · `ms_move += ms_since(t_step);`

在等齐和两次交换之后累计 move 耗时。它是包含等待和组织工作的经过时间；称‘精确’不表示测得纯内核运算时间或每个线程各自的工作量。

拿数字验一下：一个线程算得慢时，其他线程抵达 barrier 后停住，move 的读数仍在走。即使两份实现对每一字调用相同数量的位运算，只要任务分得不均、等待不同，记录的 move 就可能差很多。

第 217 行 · `}`

合上第二个 single；默认末尾再次同步。若去掉等待，某些线程可能直接开始新一轮事件，读到尚未成对交换的状态。

第 218 行

注释指向上一行附带的隐式栅栏，并没有增加第四次显式等待。比较同步数时按指令语义数，不按注释里出现几次 barrier 数。

第 219 行 · `}`

结束当前 ts 的主体。未到 t 则所有线程一起回到第 124 行；到 t 后离开时间循环，当前面就是最终森林。

第 220 行 · `}`

并行区在这里结束并汇合，线程的私有窗口离开作用域。随后串行写文件，不应由队内每个人各写一遍。

第 221 行

空行分开并行区和输出段。任务说明所称 221 行到这里甚至还没开始写文件，所以继续读到真实末行。

拿一个完整的八位字走完转移。 设本行只有一字，左右没有借位；旧高面 01010110、旧低面 01001100，上行火 00100000、下行火全零。旧火是 01000100，旧树是 00010010。算到哪一步都先写到草稿中，不在半途中动旧面。

计算项	八位结果	判读
左方向候选（旧火左移一位）	10001000	位 7、3 的左邻有火
右方向候选（旧火右移一位）	00100010	位 5、1 的右邻有火
四向并集	10101010	只说明邻火存在，不说明这里是树
新点燃＝旧树与四向并集	00000010	只有位 1 同时满足两条件
下一代 b1	00010010	两棵旧树仍具有高位
下一代 b0	01001110	旧低位保留，新火位 1 加入

读回位 1 是 11，确实成了火；位 2 从 11 到 01，是灰；位 4 仍 10，是树。一个向量例里同时看到三种变化，比孤立背两条赋值语句更能检验自己是否真读懂。再把上行火改成 00010000，只应额外点燃位 4，而不能影响位 0；这是四邻与位对齐的快速复核。

两个尺度的交换：一行写完就轮换三个私有窗口，全步写完才轮换两张全局状态面。若你在第 205 行之后急着画一张“全图已新”的图，就会把这两个尺度弄混。

这段结束后的状态：这一时间步的两张新面都已成为当前图；线程通过交换后的同步点，才能进入下一次事件处理。

写错会怎样：旧火要变灰，新火要保留一拍。若 nb1 只写没被点燃的树，刚点燃格便会落入 01（灰）而不是 11（火）。

和前后版本的关系：v6 的四邻公式相同但火的取址不同。v8 把同步显式化，br 则连两位编码都不同。

> 一句话记住：先写全一代，再发布这一代。

第 222–261 行 · 把位平面翻译回人能读的森林

这段在干什么：模拟结束，重新按真行真列走一遍，把两个真假合成表下标，写出数字字符和分隔符。一行恰好 n 个字段，补齐字的高位永远不输出。

代码块：

```
    // ========================================
    // 4. 快速缓冲输出
    // ========================================
    t0 = Clock::now();

    FILE* fo = std::fopen(argv[2], "wb");
    if (!fo) { std::fprintf(stderr, "open out failed: %s\n", argv[2]); return 1; }

    std::vector<char> obuf(1 << 22);
    char* q = obuf.data();
    char* qend = q + obuf.size();

```

```
    auto flush = [&]() {
        std::fwrite(obuf.data(), 1, size_t(q - obuf.data()), fo);
        q = obuf.data();
    };

    const char tbl[4] = {'0', '3', '1', '2'};

    for (int i = 1; i <= n; ++i) {
        size_t row_offset = size_t(i) * W;
        for (int j = 0; j < n; ++j) {
            if (q + 2 > qend) flush();
            size_t idx = row_offset + (size_t(j) >> 6);
```

```
            uint64_t bit = 1ULL << (j & 63);
            int v1 = (b1[idx] & bit) ? 1 : 0;
            int v0 = (b0[idx] & bit) ? 1 : 0;
            *q++ = tbl[(v1 << 1) | v0];
            *q++ = (j == n - 1) ? '\n' : ' ';
        }
    }
    flush();
    std::fclose(fo);
    double ms_out = ms_since(t0);

    fprintf(stderr, "read=%.1f event=%.1f move=%.1f out=%.1f total=%.1f\n",
```

```
            ms_read, ms_event, ms_move, ms_out, ms_since(t_all));

    return 0;
}
```

逐行讲解

第 222–224 行
三行注释划出缓冲输出段。打印森林和打印计时使用不同目的地，结果文件不得混入时间标签。

第 225 行 · `t0 = Clock::now();`
把可复用的起点 t0 设置到输出开始。总起点 t_all 仍保留，第 255 行读的是输出段，第 258 行读的是全过程。

第 226 行
空行把计时与打开文件隔开；因此打开和关闭文件的时间也属于输出阶段。

第 227 行 · `FILE* fo = std::fopen(argv[2], "wb");`
fopen(argv[2],"wb") 以二进制写模式打开输出，已有文件通常会被截短，失败返回空指针。二进制模式仍能写文本，只是不做平台换行转换。

第 228 行 · `if (!fo) { std::fprintf(stderr, "open out failed: %s\n", argv[2]); return 1; }`
检查输出文件指针；若打开失败，把真实路径写到 stderr，再返回失败，不会假装森林已生成。这只能发现打开时的问题，不能发现稍后的磁盘写满。

第 229 行
空行把文件准备和内存缓冲区准备分开。输出通道已经打开，但还没有格子写入。

第 230 行 · `std::vector<char> obuf(1 << 22);`
1<<22 在目标平台上得到 4,194,304；`vector<char>` 以字节为单位分配 4 MiB。每格恰好写两字符，一块能装 2,097,152 格。

拿数字验一下：若缓冲用八字节来模拟，四格‘1 2 3 0\n’刚好用满；下一格到来前须刷新。刷新按当前长度写出并复位，不需要每写一格都去问操作系统。用这个小例同时检查‘恰满’与‘还有尾部’两种情况。

第 231 行 · `char* q = obuf.data();`
q 是下一次写入的可写地址，开始指向缓冲区首部。这里不用 const，因为要经 *q 修改内容；变量 q 的移动不改变容器大小。

第 232 行 · `char* qend = q + obuf.size();`
qend 是容器末尾之后的合法哨点，不能解引用。剩余容量应从 qend 与 q 的距离理解，不能把它当作末尾那个可写槽的地址。

第 233 行
空行分隔游标设置与刷新过程。刷新的是‘目前已经填的’部分，不等于每次把整块内存写出去。

第 234 行 · `auto flush = [&]() {`
flush 是按引用捕获的小函数，不带参数。这样既能看到最新的 q，也能在调用后把同一个 q 复位；不是标准库里同名的流操纵符。

第 235 行 · `std::fwrite(obuf.data(), 1, size_t(q - obuf.data()), fo);`
fwrite 写入从首地址开始的实际字节数 q-data()，每项 1 字节。指针相减要求属于同一数组；它返回写成的项数，这里未检查，所以磁盘错误未必能在退出码中反映。

第 236 行 · `q = obuf.data();`

刷新后把游标放回开头，下一批覆盖旧内容即可。不必先 memset 清零：写出长度由新的 q 决定，旧尾部不会混入文件。

第 237 行 · `};`

合上 lambda 定义并用分号结束赋值声明。到这里仅得到一个函数对象，还没有触发任何实际写入。

第 238 行

空行隔开‘如何把字符送到文件’和‘该产生哪个字符’两件事。

第 239 行 · `const char tbl[4] = {'0', '3', '1', '2'};`

表下标由两位 b1b0 构成：0→字符 0，1→字符 3，2→字符 1，3→字符 2。别把字符常量写成整数 0、3、1、2，否则文件里会出现控制字节。

拿数字验一下：反向核对：从输入的 2 经第 66、67 行得到 b1b0=11，拼成下标 3，tbl[3] 是字符 ’2’。输入 3 得 01，下标 1 取 ’3’。读入和输出互为反向不代表转移也正确，但能独立验证编码层。

第 240 行

空行切换到逐格解码。此时森林在当前面，nb1/nb0 是上一轮留下的存储，不应拿来输出。

第 241 行 · `for (int i = 1; i <= n; ++i) {`

按内部行号 1..n 访问，跳过上下哨兵行。若额外输出一圈，森林本身可能算对了，格式仍不符题面。

第 242 行 · `size_t row_offset = size_t(i) * W;`

行首偏移按字数 W 计算。第 i 行第一个真实格子在这一字的位 0；不要照字节数组版再加一个左边框。

第 243 行 · `for (int j = 0; j < n; ++j) {`

逐列只走 n 个真格，哪怕尾字尚有填充位置也不输出。遍历 W×64 列会在 n=65 时多写 63 项。

第 244 行 · `if (q + 2 > qend) flush();`

意图是在写数字和分隔符前确保还有两格空间，不足就 flush。须辨清：当 q 已等于 qend 时，q+2 本身超出数组允许的指针运算范围；更可靠的判断按剩余长度比较，详见附录 D。

第 245 行 · `size_t idx = row_offset + (size_t(j) >> 6);`

用列号除以 64 找字，行首偏移找到对应行。它和读入第 76 行是互逆定位，用错一个下标可能仍输出 n² 个看似合法的数字。

第 246 行 · `uint64_t bit = 1ULL << (j & 63);`

取一个单置位掩码选择该字内的列。位 63 也要用宽无符号常量，不能先用窄类型移到高位再转换。

第 247 行 · `int v1 = (b1[idx] & bit) ? 1 : 0;`

三元表达式把 b1 与掩码的结果归一化成 1 或 0。若保留原掩码的数值，比如 1<<40，就不能把它当数组下标的高位。

第 248 行 · `int v0 = (b0[idx] & bit) ? 1 : 0;`

对低面做同样的归一化。两面必须用同一字同一位；若误用上一列，四状态都可能仍合法，单看输出范围抓不出错。

第 249 行 · `*q++ = tbl[(v1 << 1) | v0];`

先用 (v1<<1)|v0 拼出 0..3，再查表。*q++ 表示写入当前 q 所指的位置，随后指针前进；不是让缓冲里的字符本身加 1。

第 250 行 · `*q++ = (j == n - 1) ? '\n' : ' ';`

非行末写空格，行末写换行字符 '\n'；q 一样向前。每格固定产生两字节，这一约定让缓冲容量和文件长度都可推算。

第 251 行 · `}`

结束一行的列循环。末格已写下换行，所以行循环本身不必再补一个空白行。

第 252 行 · `}`

结束所有真行。q 仍可能指向缓冲区中段；循环结束不等于这些尾部字符已由文件接口接收。

第 253 行 · `flush();`

显式调用 flush() 写尾块。用 6×6 小例只产生 72 字节，不会触发容量检查；若漏此行，整幅小图都可能不出现在文件里。

第 254 行 · `std::fclose(fo);`

fclose 将 C 文件流关闭，结束文件层的缓冲工作。第 253 行处理的是程序自己的向量，两级缓冲不能相互替代；本行返回值也没有检查。

第 255 行 · `double ms_out = ms_since(t0);`

ms_since(t0) 得到输出阶段毫秒数。含逐格解码和写文件，不能只用它估磁盘速度，也不能把它算进 move。

拿数字验一下：原始大图的可见文本数据每格两字节，按纯 LF、每格一数字一分隔，输出应为 536,870,912 字节。这是格式推算。若程序只写了缓冲的末段或多打了一圈哨兵，长度就能提示从输出段开始排查。

第 256 行

空行把结果写入与诊断输出分开。即使屏幕上显示计时，森林的数字仍然只应该在目标文件中。

第 257 行 · `fprintf(stderr, "read=%.1f event=%.1f move=%.1f out=%.1f total=%.1f\n",`

fprintf 以标准错误流为目的地；五个 %.1f 表示浮点数保留一位小数。此行用逗号续到下一行，同一个函数调用还没结束。

第 258 行 · `ms_read, ms_event, ms_move, ms_out, ms_since(t_all));`

按格式占位的顺序给出读、事件、转移、输出和总时长。末尾那个当场用总起点求值。初始化、框架等待等口径存在差别，四项不必严格相加等于总数。

第 259 行

空行隔开诊断与正常返回。它不会等待别的 MPI 进程；v7 根本没有 MPI 协调多进程输出。

第 260 行 · `return 0;`

main 正常返回 0 表示程序走完成功路径。它没有调用判题器，也没验证写出返回量，所以 0 不能证明结果正确或性能达标。

第 261 行 · `}`

合上 main 的最外层花括号，局部容器等对象离开作用域并释放各自资源。到这里才是此份 v7 的物理末行。

用八字节输出缓冲手推。 忽略真实容量 4 MiB，假设只有八个字符槽。要写两行、每行三个格子，第一行 0 3 1，第二行 2 0 3。每格先写数字后写分隔符，第一行后的游标在第 6 格；第二行首项和空格占完余下两格。第二行第 2 格要写入前，旧内容必须刷出。以下用“空”和“换”显示通常看不见的字节。

时间点	本批缓冲已用	实际文件应得到的片段
第一行完成	6	仍可能完全为空
第二行第 1 格完成	8	仍可能完全为空
接下来的写入之前	0	0 空 3 空 1 换 2 空
第二行末尾刚写完	4	尾段 0 空 3 换 还在内存
显式 flush 并关闭	0	两段依次连起来组成完整两行

注意这张表是在验证设计意图：真实源码在“游标恰满”的判断处有指针表达式的可移植性问题，见第 244 行；不能拿纸上的正确缓冲协议当成该表达式合法的证明。再问一个常见问题：程序在终端打印了 total 是否说明末行一定写盘成功？不说明。输出通道是不同的对象，且源码没检查 fwrite/fclose 的返回状态。

合上源码，试着重建第七版。 先写在纸上，不需要安装编译器。给自己六个空框，每个框只写输入、输出和一条不会被下一阶段破坏的约定。画好框之间的箭头，再和下面的答案比。这样能检查你会不会独立重写，而不只是能否认得一行已经写好的 C++。

框	拿到什么	交出什么	进入下一框前要成立的约定
取文本	路径	缓冲与有效范围	有效字节数等于预期；哨兵不属于文件内容
装入状态	n 行原始数字	两张当前面	任一真格可由同一字、同一位解回原值
消费事件	旧图、步号、下一单的下标	魔法后快照	所有到期单子恰处理一次，未来单子未被吃掉
制作窗口	快照及线程的行区间	三行邻火	读到相邻线程的真实边界行，只有图外是零
转移并发布	三行火和本行两面	下一代的完整地图	所有人写完才对换；对换完才进入新步
导出	末代两面	n 行字符	尾字的填充格不输出，未装满的尾块也写出

五张最小读码卡。 每张卡故意只刁难一个接口。先写下预测，再去指定行验证。不需要相信一份“看上去像森林”的大图就一定是对的。

1. 一格旧火，零事件，一步。 输入头 `1 0 1`，图只有 2。上下都补零，左右也没有邻字；树掩码为 0，旧低位保留，于是输出 3。这张卡检验新火与旧火是否混为一谈。若仍输出 2，去看 L199–200，而不是先调线程数。
2. 两棵相邻树，惊雷击左，走一步。 把这两格放在一个 2×2 图的第一行，另一行全空。魔法后第一行是 2 1，步后应为 3 2。若得到 2 1，你看到的是魔法快照；若仍 1 1，去查点坐标的一基行与零基列；若第二棵树已是灰，可能把同一拍连做了两次转移。
3. 第 63 列是火，第 64 列是树。 在 65×65 的图中只设这两格，其余全空，二者处于同一行。旧火在字 0 的最高位，树在字 1 的最低位；一步应为灰、火。将两个状态对调再走一遍，期望反向传回。两张互为镜像的卡分别检查 L191 和 L192，只有一个方向失败就不必从文件头重查。
4. 线程接缝上的火。 令 n=4、两线程。内部真行 1、2 归甲，3、4 归乙；把火放在行 2 列 0，把树放在行 3 列 0。两行在不同团队成员的窗口中，却仍在同一张旧图上。若只在双线程出错、单线程正确，先找 L170–174 与 L181–185 是否把行段边缘当物理边缘。这里说的是排查方向，不是一项已做的计时实验。
5. 一个字全部回春，但没有任何火。 令 n=64，某一行 64 格全灰，其他地方全空；回春矩形恰好等于那一行的 0..63 列。一轮结束该行应全为树，没有一格变火。它同时区分“整字掩码错误”和“灰被解成旧火”两种故障：若整行不动，先看 L151；若复活后仍自燃，查旧火的判定是否真是两个位相与。

如何定位而不是盲改。 对一个错误格，先写出期望输出的字符，再反查 tbl 得期望两位；核对两张新面里这个位，若对，问题就在导出；若不对，查看魔法后旧树位和四向邻火位。旧树位错回头查读入或魔法，邻火位错再分别查字接缝、行接缝和地图边缘。一次缩小一个区间，通常比盯着 261 行找“看起来很奇怪的符号”更有效。

这段结束后的状态：成功路径已把缓冲区的尾段写出并关闭文件，标准错误通道列出毫秒计时，进程返回成功状态。

写错会怎样：容易漏掉未装满的尾块，也容易把编码下标误当原始状态：下标 1 应输出灰的字符 3。写错误时，当前版本并没有逐次验证写出量。

和前后版本的关系：v2 就开始攒块输出；v7 多了一层状态反查。br 采用数值本身的二进制编码，所以它的反查方式不一样。

> 一句话记住：索引还原状态，状态还原字符，尾块也要落盘。

CHAPTER 03

第 3 章　把 v1–v6 摊在 v7 旁边

这一章你会得到什么：给定一个旧版本，能找到 v7 中做同一件事的那几行。每组左右证据各用自己文件的行号，标题后的第一框是旧版，第二框是第 2 章的原行。 代码框保留关键原行，行号不连续时，中间行见标注的源文件区间。

版本	相对 v7 少了什么	多了什么	后来如何取舍
v1	位面、固定行段、缓冲写出	二维整数与每步复制	保留读旧写新，移除复制
v2	连续位面和行窗口	预分配的二维整数网格	保留交换与缓冲，缩窄元素
v3	字级并行	一格一字节与左右哨兵列	改为一字 64 格，跨字补位
v4	线程团队、滚动行	完整缓存的火图	改成三行临时火
v5	贯穿多步的团队	每步两次组合并行区及火图	派工范围外移
v6	私有的行窗口	全局火图和填火阶段的同步	保留团队、交换与计时口径

> 一句话记住：对齐同一个问题的两种答案，比对齐两个文件的同一个行号更有用。

3.1　v1　用第 64 列反查两种地址

存一棵树。取输入坐标 (0,63)。v1 内部地址是 sta[1][64]，早期版本同时给行列留了边；v7 取行 1 的第 0 个字、位 63。v7 的第 69、71 行一基行配零基列，对照第 76 – 77 行就能自己推出地址。保留的设计是不让森林外部提供火，不是必须给每一方向都留一格。

选行 • 1-fire_pr.cpp L39 – 45

```
39    int n, m, t;
40    std::cin >> n >> m >> t;
41    std::vector<std::vector<int>> sta(n + 2, std::vector<int>(n + 2));
42    for(int i = 1; i <= n; i++) {
43        for(int j = 1; j <=n; j++) {
44            std::cin >> sta[i][j];
45        }
```

选行 • 7-fire_final.cpp L69 – 79

```
73            if (p >= end) { std::fprintf(stderr, "input truncated\n"); return 1; }
77                uint64_t bit = 1ULL << (j & 63);
```

算一格，写去哪里。v1 的 if 用整数是否等于 2 识别火，v7 第 196 – 200 行把判断拆成两条位面表达式。给三棵相连树的最左格点火，若原地从左往右更新，火能虚假连跳两次；两版都用另一

份存储消除此顺序依赖。v7 的两个 swap 在这里交换的是窗口，真正发布代际要看第 214 – 215 行。

选行 • 1-fire_pr.cpp L104 – 120

```
106         std::vector<std::vector<int>> temp = sta;
110                     (sta[r - 1][c] == 2
112                     || sta[r][c - 1] == 2
116                 else if(sta[r][c] == 2) temp[r][c] = 3;
120         sta = temp;
```

选行 • 7-fire_final.cpp L189 – 205

```
190                         uint64_t f = f_curr[j];
192                         uint64_t R = (f >> 1) | (j + 1 < W ? f_curr[j + 1] << 63 : 0ULL);
196                         uint64_t tree = b1[idx] & ~b0[idx];
200                         nb0[idx] = b0[idx] | burn;
204                     std::swap(f_curr, f_next);
```

> 一句话记住：地址能变，同时更新不变。

3.2 v2 仅交换容器还不够

不变的格子也要落笔。v2 的第三个分支把旧值原样写到 temp，保证每个位置在发布前已被赋值；v7 第 199、200 行不分情况，两个完整字直接覆盖上一代留下的数据。二者都不靠每轮清零来保持正确。拿一格从来不着火的灰，就能检查新图回收之后有没有丢掉它。

选行 • 2-fire_pr.cpp L119 – 137

```
123                 if(sta[r][c] == 1 &&
125                     || sta[r][c - 1] == 2 || sta[r][c + 1] == 2)) {
128                 else if(sta[r][c] == 2) {
129                     temp[r][c] = 3;
136         std::swap(sta, temp);
```

选行 • 7-fire_final.cpp L195 – 200

```
195                         size_t idx = curr_offset + j;
196                         uint64_t tree = b1[idx] & ~b0[idx];
197                         uint64_t burn = tree & nbr;
198
199                         nb1[idx] = tree;
200                         nb0[idx] = b0[idx] | burn;
```

写一个数字，两种翻译。v2 已有字符缓冲、双字符输出和尾部刷新。它以 '0'+整数 直接造字符；若套在 v7，把编码下标 2 直接加到字符 0 会打印 2，实际下标 2 表示树，要打印 1。第 239、249 行那张逆表因此是 v7 状态编码的必要搭档，缓冲大小则是可以调整的工程选择。

选行 • 2-fire_pr.cpp L145 – 165

```
146     if (!fo) { std::fprintf(stderr, "open out failed: %s\n", argv[2]); return 1; }
149     char* q = obuf.data();
153         std::fwrite(obuf.data(), 1, size_t(q - obuf.data()), fo);
159             if (q + 2 > qend) flush();
161             *q++ = (j == n) ? '\n' : ' ';
```

选行 • 7-fire_final.cpp L234 – 250

```
235         std::fwrite(obuf.data(), 1, size_t(q - obuf.data()), fo);
236         q = obuf.data();
244             if (q + 2 > qend) flush();
249             *q++ = tbl[(v1 << 1) | v0];
250             *q++ = (j == n - 1) ? '\n' : ' ';
```

> 一句话记住：先写全再交换，先解码再造字符。

3.3 v3 从相邻字节到相邻位

同样是扁平数组，偏移单位不一样。v3 的 stride 数格，v7 的 W 数字。设 n=65，v3 内部坐标 (1,65) 在单字节数组的偏移 132；v7 对同一棵树用内部行 1、外部列 64，字下标 3、位 0。要从 v3 迁移，不能只改元素声明就照搬原公式。

选行 • 3-fire_pr.cpp L59 – 72

```
60     size_t stride = static_cast<size_t>(n) + 2;
62     std::vector<uint8_t> cur(stride * stride, 0);
63     std::vector<uint8_t> nxt(stride * stride, 0);
66         size_t row_offset = static_cast<size_t>(i) * stride;
69             if (p >= end) { std::fprintf(stderr, "input truncated\n"); return 1; }
```

选行 • 7-fire_final.cpp L59 – 79

```
63     std::vector<uint64_t> b1(total_words, 0), b0(total_words, 0);
64     std::vector<uint64_t> nb1(total_words, 0), nb0(total_words, 0);
66     const uint64_t b1bit[4] = {0, 1, 1, 0};
73             if (p >= end) { std::fprintf(stderr, "input truncated\n"); return 1; }
77                 uint64_t bit = 1ULL << (j & 63);
```

上下可按行取，左右要跨字拼。v3 对上、下取 idx±stride，对左、右取 idx±1，因四面补零所以有效。v7 第 191 – 193 行先移位，还需在 j>0、j+1<W 时借一位。字地址前进 1 跨了 64 列，不是到达相邻一棵树。只有在 j 的首尾加上条件才不会把下一行串进来。

选行 • 3-fire_pr.cpp L131 – 149

```
132             size_t row_offset = r * stride;
138                     (cur[idx - stride] == 2 ||
139                      cur[idx + stride] == 2 ||
```

选行 • 7-fire_final.cpp L189 – 200

```
190                         uint64_t f = f_curr[j];
192                         uint64_t R = (f >> 1) | (j + 1 < W ? f_curr[j + 1] << 63 : 0ULL);
196                         uint64_t tree = b1[idx] & ~b0[idx];
197                         uint64_t burn = tree & nbr;
200                         nb0[idx] = b0[idx] | burn;
```

> 一句话记住：同一个 +1，有时走一格，有时走六十四格。

3.4 v4 同样的公式，为什么不存 fire

第五张图是缓存，不是第五种状态。两边都用 b1 标树或火、b0 标火或灰，也都要备一套下一代。v4 第 64 行多的 fire 可以由 b1 & b0 恢复，它不是不可丢的数据。v7 不改变编码表，把这份中间结果的保留范围限制到第 120–122 行的三个行向量。

选行 • 4-fire_bit.cpp L62–68

```
62      std::vector<uint64_t> b1(total_words, 0), b0(total_words, 0);
63      std::vector<uint64_t> nb1(total_words, 0), nb0(total_words, 0);
64      std::vector<uint64_t> fire(total_words, 0);
65
66      // 读入状态映射表
67      const uint64_t b1bit[4] = {0, 1, 1, 0};
68      const uint64_t b0bit[4] = {0, 0, 1, 1};
```

选行 • 7-fire_final.cpp L59–67

```
63      std::vector<uint64_t> b1(total_words, 0), b0(total_words, 0);
64      std::vector<uint64_t> nb1(total_words, 0), nb0(total_words, 0);
66      const uint64_t b1bit[4] = {0, 1, 1, 0};
67      const uint64_t b0bit[4] = {0, 0, 1, 1};
```

用内存换计算，也能反过来换。v4 生成整个 fire 后，直接按 idx±W 取上下邻；v7 第 178、183 行在每个线程行段开头和滚动时解码火。线程接缝可能各自重复计算一行，这是真成本；代价换来不必让所有线程先写一遍共享大数组。若图小、线程多，窗口并非必然占优。

选行 • 4-fire_bit.cpp L166–190

```
168             fire[k] = b1[k] & b0[k];
179                 uint64_t L = (f << 1) | (j > 0 ? fire[idx - 1] >> 63 : 0ULL);
182                 uint64_t nbr = L | R | fire[idx - W] | fire[idx + W];
185                 uint64_t burn = tree & nbr;
188                 nb0[idx] = b0[idx] | burn;
```

选行 • 7-fire_final.cpp L177–200

```
190                         uint64_t f = f_curr[j];
192                         uint64_t R = (f >> 1) | (j + 1 < W ? f_curr[j + 1] << 63 : 0ULL);
196                         uint64_t tree = b1[idx] & ~b0[idx];
197                         uint64_t burn = tree & nbr;
200                         nb0[idx] = b0[idx] | burn;
```

> 一句话记住：可从旧面重算的火，不必保留整图。

3.5 v5 把派工位置画出来

两次并行区对一个常驻区。v5 两个 parallel for 各自覆盖一段循环。v7 第 111 行 parallel 包住时间步，连自己的起止行都要在第 116 – 118 行计算。若照抄旧程序，既自己分好区又让运行库只分给你一部分工作，可能分重或漏行；两种分工方式应完整择一。

选行 • 5-fire_bit.cpp L156 – 170

```
159         #pragma omp parallel for schedule(static)
161             fire[k] = b1[k] & b0[k];
166         #pragma omp parallel for schedule(static)
```

选行 • 7-fire_final.cpp L111 – 124

```
111     #pragma omp parallel
120         std::vector<uint64_t> f_prev(W, 0);
121         std::vector<uint64_t> f_curr(W, 0);
122         std::vector<uint64_t> f_next(W, 0);
```

在哪个位置需要全员到齐。v5 两个组合构造正常结束才能交换全图，结构上自带先后；v7 手工循环结束只说明本线程写完，必须在第 209 行显式等待。即使处理的行永远不重叠，交换的却是团队公用的两个容器，不能由最快的人随时更换。

选行 • 5-fire_bit.cpp L171 – 186

```
171                 uint64_t f = fire[idx];
174                 uint64_t R = (f >> 1) | (j + 1 < W ? fire[idx + 1] << 63 : 0ULL);
177                 uint64_t tree = b1[idx] & ~b0[idx];
181                 nb0[idx] = b0[idx] | burn;
185         std::swap(b0, nb0);
```

选行 • 7-fire_final.cpp L203 – 217

```
203                     std::swap(f_prev, f_curr);
204                     std::swap(f_curr, f_next);
209             #pragma omp barrier
214                 std::swap(b1, nb1);
215                 std::swap(b0, nb0);
```

> 一句话记住：分工方式变了，等待的位置也得重新找。

3.6 v6 少掉的是一轮全图生产

不要把已有的改进再发明一次。v6 已把事件终点记下并重置 t_step，v7 第 161、164 行保留。v6 的下一段让所有人填满 fire，并依赖 for 的末端等待；v7 第 169 – 178 行各自从不再被修改的旧面提取火，不必等待邻线程先生产一块 fire。取消同步的依据是依赖已经移除。

选行 • 6-fire_bit.cpp L162 – 179

```
171             #pragma omp for schedule(static)
173                 fire[k] = b1[k] & b0[k];
178             #pragma omp for schedule(static)
```

选行 • 7-fire_final.cpp L161 – 178

```
161                 ms_event += ms_since(t_step);
170                 if (start_r > 1) {
178                 for(int j = 0; j < W; ++j) f_curr[j] = b1[offset + j] & b0[offset + j];
```

写回公式没变，窗口与发布不能混讲。v6 四邻的逻辑式与 v7 第 191 – 200 行等价，区别在读取源。v6 第 201、202 行交换全局面；v7 额外有第 203、204 行的小窗口交换，但每步仍只在第 214、215 行发布一次完整地图。对一个中间字检查 64 个格都对，不代表窗口跨行轮换与发布时序也对。

选行 • 6-fire_bit.cpp L182 – 205

```
183                     uint64_t f = fire[idx];
186                     uint64_t R = (f >> 1) | (j + 1 < W ? fire[idx + 1] << 63 : 0ULL);
190                     uint64_t burn = tree & nbr;
199             #pragma omp single
202                 std::swap(b0, nb0);
```

选行 • 7-fire_final.cpp L189 – 218

```
190                         uint64_t f = f_curr[j];
193                         uint64_t nbr = L | R | f_prev[j] | f_next[j];
199                         nb1[idx] = tree;
209             #pragma omp barrier
215                 std::swap(b0, nb0);
```

> 一句话记住：四邻逻辑延续，火图的生存范围变了。

CHAPTER 04

第 4 章　v8：评测没有打开 OpenMP，怎么办

这一章你会得到什么：看清作者换的是线程组织方式，而不是火势公式；能在纸上走完三个同步点，并指出哪些时间不能跨版本直接比。 代码框保留关键原行，行号不连续时，中间行见标注的源文件区间。

4.1　评测命令缺了什么

题面第 247 – 258 行给的是 `mpiicpx forest.cpp -O3 -xHost -o forest`，运行时另用 `mpirun ... -np 64`。编译驱动（compiler driver，组织编译和链接的工具）名字里有 MPI，保证不了同一进程内的循环被多线程分担。-O3 要求较高等级的优化，-xHost 允许为编译主机的指令集优化，-o 只指定输出文件名。三者都不等于 OpenMP 的启用选项。

v8 文件开头把没有 -fopenmp 列为迁移动机。按 Intel 编译器文档，默认不为这些指令生成多线程代码；Intel 也有 -fiopenmp 等启用方式，题面没有给出。若包装器没有额外代加，v7 的 pragma 不能组织一队线程；此时还要分两种情况：能链接到运行库，串行环境里团队为一人；无法解析 omp_get_num_threads 等符号，可能在链接阶段就失败。是否打印“忽略指令”的警告取决于工具配置，包里没有评测编译日志。v8 要消掉的正是这层依赖，不是 OpenMP 算不出正确答案。选行 • 8-fire_final.cpp L3 – 10

```
 3 // 设计要点（与 7-fire_final.cpp 的 OpenMP 版相比）：
 7 //   2. 事件段（妙手回春的大矩形）从串行改成和转移共用线程团队。
10 //      其余进程立即退出，避免 64 个进程同时写同一个输出文件。
```

> 一句话记住：进程数、编译优化等级和线程团队是三个独立开关。

4.2　三个人怎样在一道门后同时看到新数据

类（class，封装数据和方法的类型）里 public 是能从外部调用的部分，private 是内部状态。explicit 是显式构造约束，禁止拿整数自动冒充这个类的对象。看到 `std::atomic<unsigned>` 时先读作‘可以原子操作的无符号世代号’，不要把所有访问普通变量的动作也误认为原子。

原子变量（atomic variable，使多线程对该对象的特定读写不可分割的变量）解决的是‘几个人同时到站，别漏数’。仅有计数还不够，每一步要重复过同一道门，所以用世代号（generation，标识这是第几次会合的编号）。SpinBarrier 的构造器记住固定人数 n_，cnt_ 和 gen_ 从 0 开始；wait() 没有返回数据，返回本身表示这轮到齐。

设三人 A、B、C 看见 gen=5。A 的 fetch_add 返回旧数 0，B 返回 1，两人继续等；C 返回 2，加一恰为 3。C 把 cnt_ 归零，再把 gen 发布为 6；A、B 观察到改变便能离开。若只等 cnt_==0，跑得快的人下一轮已经把它加到了 1，慢人可能错过那瞬间。给每一轮单独的代号就不必赌是否恰好看到零。

获取与释放（acquire/release，建立线程间先后与可见性的原子内存顺序）在这里有明确方向。到站的读改写采用 acq_rel，接过先前到站者的工作并发布自己的工作；末位到站者把新世代以 release 写出；等待者用 acquire 读到新值之后，才能按前一阶段已经完成来用共享数据。计数清零用 relaxed，不单独传递其他数据，由后面的世代发布承接。_mm_pause 给某些处理器忙等提示，不是睡眠、互斥或同步的替代品。注释说比条件变量快一个量级，没有原始重复测量，不拿来填成绩表。 选行 • 8-fire_final.cpp L38 – 43

```
38 #if defined(__x86_64__) || defined(_M_X64)
39 #include <immintrin.h>
40 #define FIRE_PAUSE() _mm_pause()
41 #else
42 #define FIRE_PAUSE() ((void)0)
43 #endif
```

选行 • 8-fire_final.cpp L54 – 72

```
59         const unsigned gen = gen_.load(std::memory_order_acquire);
61             cnt_.store(0, std::memory_order_relaxed);
62             gen_.store(gen + 1, std::memory_order_release);
64             while (gen_.load(std::memory_order_acquire) == gen) FIRE_PAUSE();
71     std::atomic<unsigned> gen_{0};
```

> 一句话记住：计数解决谁到齐了，代号解决到齐的是哪一轮。

4.3 谁来工作，行少于人又怎么办

标准线程（std::thread，执行可调用对象的一条 C++ 线程）显式创建后会运行 worker(tid)。T 默认 16；getenv 查询环境变量 FIRE_THREADS，不存在返回空指针，存在则返回字符串地址；atoi 把字符串前部按整数解析，代码随后作下限和硬件提示的上限处理。hardware_concurrency 返回的是并行能力提示，允许为 0，不保证等于这个进程获调度的核数。特别是题面给了 I_MPI_PIN_DOMAIN=core，具体绑核结果必须看机器，不能承诺 16 个线程一定各占一核。

n=5、T=4 时 chunk=2，三人做行 1 – 2、3 – 4、5，第四人拿不到行。空闲线程把区间标成 0..−1，看似没有贡献却不能直接退出；人数已写进栅栏，如果只剩三个人再也过不了门。主线程编号为 0，pool 里只放 T−1 个新线程，不是再建 T 个然后让主线程闲等。join（thread join，等待指定线程结束）在模拟之外执行一次。v7 的并行区其实已经包住全部步数，不能说‘从 v7 到 v8 才发明常驻’。 选行 • 8-fire_final.cpp L182 – 203

```
185         const int hw = int(std::thread::hardware_concurrency());
200             std::vector<ui> f_prev(w_sz), f_curr(w_sz), f_next(w_sz);
```

选行 • 8-fire_final.cpp L289 – 294

```
289
290         std::vector<std::thread> pool;
291         pool.reserve(size_t(T) - 1);
292         for (int k = 1; k < T; ++k) pool.emplace_back(worker, k);
293         worker(0);
294         for (auto& th : pool) th.join();
```

> 一句话记住：有无工作由行段决定，是否必须到站由团队人数决定。

4.4 把回春分出去，为什么惊雷仍归一个人

v7 的 single 内不论矩形多大都由一个人做。v8 让每人看相同的事件表和同样的步号，却给每人自己的 cnt_event。若共享一个游标并任一人先加，别人可能还没处理自己那几行，就把事件跳过了。惊雷只有一个目标，指定 tid==0 修改；回春则用 max(事件起点,区段起点) 与 min(事件终点,区段终点) 取行的交集，按列的首尾掩码仍沿用 v7 的做法。

试用三人分十行的例子：矩形内部行 3..7，线程 0 改 3..4、线程 1 改 5..7、线程 2 不改。两个线程不会对同一个 64 位字进行各自的读改写，因为它们写的是不同行。不应随意改成按格子均分；相邻两格可能在同一个字内，对不同位的普通 |= 也可能互相覆盖。题目保证同一步最多一个事件，所以在本实现中不必处理同一步多种魔法之间的先后依赖。 选行 • 8-fire_final.cpp L203 – 224

```
209                         if (tid == 0) {
```

选行 • 8-fire_final.cpp L225 – 241

```
227                                     const ui mask = (lo == 0 && hi == 63)
231                                     const ui ash = mask & b0[idx] & ~b1[idx];
232                                     b1[idx] |= ash;   // 01 -> 10
240                 if (tid == 0) ms_event.store(ms_event.load() + ms_since(te));
241                 bar.wait();
```

> 一句话记住：分行是在分独占的字，而不仅是在给线程分坐标。

4.5 从三个同步点看时间账

第 241 行：每人完成自己的魔法后才可读邻行。第 280 行：所有线程已把自己的新面写完。第 286 行：线程 0 已把两张面都换好，你才可开始下一拍。第一道防止读到还没复活的树，第二道防止旧图在别人仍用时被替换，第三道防止一人见新高面、另一人见旧低面。把边界例记在脑子里，比背代码行号有用。

v8 第 240、279 行只由 tid=0 取时间差，在相应的公共等待之前累加。这两个分项既不是全线程耗时之和，也不保证代表最慢那个人；第 297 行的 sim 则是团队建起、运行、join 之后的阶段时长。v7 第 216 行是在大家到齐、交换后再取 move；不能拿两版同名 move 相除就断言提速。v8 在读完输入后 clear/shrink_to_fit，以缩小输入缓冲的占用；后者是非强制的收缩请求，不保证物理内存立即归还给系统。 选行 • 8-fire_final.cpp L274 – 287

```
275                         std::swap(f_prev, f_curr);
279                 if (tid == 0) ms_move.store(ms_move.load() + ms_since(tm));
282                 if (tid == 0) {
283                     std::swap(b1, nb1);
286                 bar.wait();
```

选行 • 8-fire_final.cpp L174 – 185

```
185         const int hw = int(std::thread::hardware_concurrency());
```

> 一句话记住：比较两个计时标签，先看时间戳在门前还是门后。

4.6 可选 MPI 是输出保护，不是四节点加速

条件编译（conditional compilation，编译前按条件保留或移除一段源码）由 __has_include 判断是否能找到 mpi.h。找到则定义 FIRE_HAS_MPI，从而在 main 中调用 MPI_Init 初始化、MPI_Comm_rank 查询自己在全局通信器 MPI_COMM_WORLD 中的编号，非零号调用 MPI_Finalize 收尾，不进入读图、算图与写图路径。收尾是集体操作，不能把注释‘立即退出’理解为必定立即结束整个系统进程。能找到头文件也不自动保证链接器找得到 MPI 库。

若用题面 64 个进程，启用了上述分支时 rank 0 持有整图，rank 1 – 63 都不分到任何格子；没有向其他节点传递行。若编译时完全没启用 MPI 分支，却用外部启动器起 64 份相同程序，分支不会凭空出现，各份进程仍可能抢写同一个文件。程序正常尾部配对 Finalize，但若 rank 0 在若干文件错误分支提前返回，其他进程退出的行为仍需留意。 选行 • 8-fire_final.cpp L31 – 36

```
31 #if defined(__has_include)
32 #if __has_include(<mpi.h>)
33 #include <mpi.h>
34 #define FIRE_HAS_MPI 1
35 #endif
36 #endif
```

选行 • 8-fire_final.cpp L76 – 98

```
76 int main(int argc, char** argv) {
82     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
85         return 0;
89     (void)rank;
97         return 1;
```

选行 • 8-fire_final.cpp L329 – 337

```
334     MPI_Finalize();
336     return 0;
```

> 一句话记住：只让 0 号写可以避免争写，却不会把计算自动分到其他节点。

4.7 并排看两版的边界

问题	v7	v8
谁建团队	OpenMP parallel	标准库建 T-1 人，主线程加入
团队寿命	跨全部时间步	同样跨全部时间步
谁做事件	任选一人执行 single	惊雷给 0 号；矩形按行分
谁等谁	两个 single 末端与一道显式栅栏	每步显式三次 wait
位面、窗口、旧火规则	两面两代，各人三行火	数学不变
多进程输出	无 rank 保护	有 MPI 时仅 0 号；无 MPI 时无保护
move 的终点	等齐并交换后	0 号在第二道等待前取样

> 一句话记住：一次迁移要对齐算法、同步和测量这三件事。

CHAPTER 05

第 5 章　另一位作者：只把可能变化的块排进队列

这一章你会得到什么：能沿实际入口追到每一类函数，并把活动块、局部线程与跨进程通信分开。下文行号属于 br-fire-pool.cpp；锁的十二行另标文件名。文件共 716 行。 代码框保留关键原行，行号不连续时，中间行见标注的源文件区间。

5.1　读这条路线，先换一张编码卡

同学版不是 v9。它用相邻的一对字 a、b 存一排 64 棵树。a 是题面数字的低位，b 是高位。按 (b,a) 顺序写，空 00、树 01、火 10、灰 11；按内存里取出的 (a,b) 顺序读，树是 (1,0)，火是 (0,1)。这和作者版的 b1、b0 符号含义不一样。遇到“第一个字”和“高位”这两个词务必拆开：内存顺序没规定第一个一定代表高位。

自己算一次：旧树 a=1,b=0。有邻火就令 ignite=1，新 a=(1|0)&~1=0，新 b=0|1=1，读回高低顺序为 10，即火。旧火 a=0,b=1，不在旧树集合里，ignite=0，于是新 a=1、新 b=1，正好是灰。若旧格是灰，两个位原本都为 1，仍两位皆 1。空格两位皆 0，纵使 near=1 也不会烧。看完这四种取值，再看左右移动才不会把题面数字和数学运算搅在一起。

br 选行 • L94 – 101

```
 94            const ui near = prev[j] | next[j] | _shld<1>(C, Lw) | _shld<63>(R, C);
 97            const ui ignite = (a & ~b) & near;
 98            dst[2 * j] = (a | b) & ~ignite;
 99            dst[2 * j + 1] = b | ignite;
101            row_burn |= ignite;
```

br 选行 • L335 – 344

```
335                    for (int w = 0; w < TileSize; ++w) {
340                            b |= ui(v >> 1) << k;
344                    }
```

> 一句话记住：名字只给提示，实际编码以装入和解码的公式为准。

5.2　跨字位移：_shld 不会替你完成火灾规则

模板（template，让编译器按参数生成具体版本的定义）此处把位移量也作为参数；`_shld<1>` 中的 1 在编译前就确定。static_assert 在编译期检查 amount<64，不通过便不会生成这个实例。if constexpr 按编译期条件保留分支：位移量为零直接返回 a，避免计算 b>>(64−0)。inline 允许相应的函数定义用法，不保证每次都展开成一条机器指令。

把两个八位玩具字代进公式：C=00000010，左字 Lw=10000000。左移一位，C 变为 00000100；左字最高位落到最右边的低位，结果 00000101。原代码是 64 位，左邻方向用 `_shld<1>`(C,Lw)，右邻用 `_shld<63>`(R,C)，后者等价于 C>>1 再从 R 的最低位补最高位。可选宏 ASM_SHLD 才挑选内

联汇编（inline assembly，源码内嵌目标机器指令）；默认分支只是等价 C++ 表达式。没有汇编清单时，不能声称评测一定执行了某一条指令。

br 选行 • L17 – 34

```
17 using ui = uint64_t;
20     static_assert(amount < 64);
23         return a;
26         return a;
32         return (a << amount) | (b >> (64 - amount));
```

br 选行 • L88 – 99

```
91             const ui C = curr[j];
93             const ui R = curr[j + 1];
94             const ui near = prev[j] | next[j] | _shld<1>(C, Lw) | _shld<63>(R, C);
97             const ui ignite = (a & ~b) & near;
99             dst[2 * j + 1] = b | ignite;
```

> 一句话记住：双字移位解决接缝，一整个转移还要看树、火和四邻。

5.3 给活动块排队：enqueue_next 与去重表

块队列（task queue，保存待处理任务编号的容器）用 deque 表示，可在尾部加入、在头部取出。marked 是去重表。如果本步新火出现在两个块的公共角落，两个工作者都可能希望下步处理同一块。不去重，既重复做工，也可能同时写同一块 next，令双缓冲失去‘独占写入者’这个保证。函数本身不负责加锁，调用者必须知道当前处于单人准备阶段还是多线程阶段。

队列编号是带外框网格的位置，实际存储编号却不含外框。以 N=2 为例，带边框每行 4 个位置，四个真块编号是 5、6、9、10，对应存储编号 0、1、2、3。id=5 左边的 4 属于框；函数可以给框标记真，却不把框压入队列。这样上层可以统一提出‘请排我上方的块’，不必在每条边界分支先判断物理邻居是否存在。前提是传来的 id 连外框也不越过。

br 选行 • L36 – 45

```
36 template <int __tiles_side_cnt>
37 void enqueue_next(int id, bool* marked, std::deque<int>& tasks) {
43             tasks.push_back(id);
```

br 选行 • L118 – 130

```
121             enqueue_next<__tiles_side_cnt>(id, marked, tasks);
123             enqueue_next<__tiles_side_cnt>(id - __tiles_side_cnt - 2, marked, tasks);
125             enqueue_next<__tiles_side_cnt>(id + __tiles_side_cnt + 2, marked, tasks);
127             enqueue_next<__tiles_side_cnt>(id - 1, marked, tasks);
129             enqueue_next<__tiles_side_cnt>(id + 1, marked, tasks);
```

> 一句话记住：先统一编号和去重，再谈一块分给谁。

5.4 propagate_tile_and_append：在哪里算，在哪里锁

一个 TileSize=S 的块不是 S×S 个格。它每行有 S 对字，每字代表 64 列；共有 64S 行，所以边长是 64S。S=16 时边长 1024，一块含 1,048,576 格，一代占 256 KiB。三个局部火行各多两个字作边沿槽；prev=row1+1 后，prev[-1] 仍落在 row1[0] 内，不是随便允许对数组用负下标。load_burn 是本函数里的小函数，把输入字对转成 b & ~a。邻居指针为空时写零，表示地图之外没有火。

每处理一行，curr[-1] 使用左块末字，curr[width] 使用右块首字；上、下边缘可能由邻块某行或进程边界缓冲提供。核心内层只读当前代、写独立 pong_buf。三行轮转的不只是算当前块：row_burn 汇总一行的新火，all_burn 汇总整块，top/bottom/left/right 记录四条边有没有新火。新火下步会变灰，在自己块内必须再处理一次；如果新火触边，隔壁块下一步也可能发生变化。因此块即使只剩一处新火，也不能跳过自己的下一步。

所有按位计算都在锁外完成。只在末段插入下轮队列时取得 B 锁；一次拿锁可以把本块和至多四邻的请求放好，再自动释放。若把整块百万格都算在锁里，队伍即使有 16 人也只能依次算。若为了更快删掉末端的锁，去重检查与入队这两个动作就不再是一个不可拆的决定。

br 选行 • L53 – 81

```
53      static_assert(__tile_size > 0);
74      ui* curr = row2 + 1;
77      load_burn(me, curr);
81          curr[-1] = left ? left[2 * i * width + 1] & ~left[2 * i * width] : ui{0};
```

br 选行 • L95 – 116

```
97              const ui ignite = (a & ~b) & near;
99              dst[2 * j + 1] = b | ignite;
103                 first_ignite = ignite;
104             last_ignite = ignite;
115         curr = next;
```

br 选行 • L118 – 130

```
121             enqueue_next<__tiles_side_cnt>(id, marked, tasks);
123             enqueue_next<__tiles_side_cnt>(id - __tiles_side_cnt - 2, marked, tasks);
125             enqueue_next<__tiles_side_cnt>(id + __tiles_side_cnt + 2, marked, tasks);
127             enqueue_next<__tiles_side_cnt>(id - 1, marked, tasks);
129             enqueue_next<__tiles_side_cnt>(id + 1, marked, tasks);
```

> 一句话记住：重活留在锁外，队列与去重表作为一件事保护。

5.5 谁拥有存储：TileLink、FireHalo 和 make_tile_links

TileLink 只保存 current、next 两个指针，自己不拥有那两段内存。真实空间由 data_A、data_B 两个向量拥有；建表后若让它们再次分配，旧指针会失效。make_tile_links 接收两段起

点，按 id×tile_words 填成指针对并返回表。主路径读取函数里自己做了类似的建表，没有调用这个辅助函数；同一设计出现两处不等于运行时做了两遍。

为什么不能每步换掉全部块？假设 A、B 两个块本步都在旧图，但只有 A 入队。A 的 next 会写全，B 的 next 却可能是两步以前的内容。若整片交换，B 就倒退了。这里每个块独立决定 current 指向哪一侧，主线程只给 processed 清单上的块换指针。下次读取 B 时仍见它保留下来的正确现状。

FireHalo 是邻界数据（halo，邻区贴着本区边沿的一层信息）。tile_edge(direction,i) 先判断某方向有没有数据，没有就返回 nullptr；上下边缘每个块仅需一行字对，左右方向则按原内核行跨度存放，跨行步长不一样。若错用一个统一 stride，水平方向也许正确，竖直方向却会在第二行起拿错字。

br 选行 • L133 – 151

```
142 template <int TileSize, int N>
144     std::vector<ui> edges[4]; // up, down, left, right as seen by this rank
147         if (edges[direction].empty()) return nullptr;
148         const size_t stride = direction < 2 ? size_t(2) * TileSize
150         return edges[direction].data() + size_t(i) * stride;
```

br 选行 • L156 – 165

```
156 template <int TileSize, int N>
157 std::vector<TileLink> make_tile_links(ui* initial_data, ui* spare_data) {
158     static_assert(TileSize > 0 && N > 0);
163         tiles.push_back({initial_data + id * tile_words, spare_data + id * tile_words});
164     return tiles;
```

> 一句话记住：块指针交换得有选择，边界数据也得按方向解释。

5.6 两种工作调度：node_step 与真正用到的 SpinLock

互斥（mutual exclusion，一次只准一个执行者进入临界区）在这里通过自旋锁实现。临界区（critical section，会碰共享队列的不容交错的操作范围）只覆盖领取或加入任务，不应把整块计算也关进去。线程结束时的 join 只处理线程生命周期，不保护 deque 内部。

node_step 是保留的每步建队版本，不是当前 main 的默认入口。它先复制 A 中的编号到 processed；接下来 A 会被弹空，这张清单是事后知道哪些块要换指针的凭据。一把 A_lock 管领取任务，一把 B_lock 管下轮提名。工作者拿到 id，退出短锁区后拆成块行、块列，从 table 找 current 和 next；若同进程内有相邻块，优先取真实邻块的边，否则保留 halo 或空指针。线程 0 也执行取任务的循环，不只是管理者。goto empty 在这里从主线程的空队列分支跳到等待和发布区，不是让后台线程直接消失。

看随包 spin_lock.hpp。flag 是原子布尔，false 表示无人占锁。lock() 用 exchange(true,acquire) 写入 true 并返回旧值；读到 false 的那个人拿到锁，读到 true 的继续循环。unlock() 用 release 写回 false，下一位成功获取锁的人能在正确顺序下看到上位留

下的队列状态。lock_guard 是作用域锁卫士：构造时上锁，离开花括号、遇到 return 时也会析构解锁。所谓‘不用 mutex’是没有拿 std::mutex 当那把队列锁；源码仍引用 mutex 头里的通用 lock_guard。

自旋（spin waiting，反复检查直到条件满足）可能适合非常短的任务领取，但并不保证公平，也不保证一定比休眠划算。若持锁线程没获调度，其他线程会不断消耗处理器时间。代码没有用条件变量，没有等待新任务出现的休眠消费者；A 空就结束这一轮取任务，B 里的任务要等下一步。

br 选行 • L167 – 187

```
167 template <int __thread_cnt, int __tile_size, int __tiles_side_cnt>
168 void node_step(std::deque<int>& deque_A, std::deque<int>& deque_B, std::vector<TileLink>&
    tiles, bool* marked,
173     const std::vector<int> processed(deque_A.begin(), deque_A.end());
178         workers.emplace_back([&, tid] {
184                         return;
```

br 选行 • L191 – 213

```
196                     return tiles[tile_id].current;
198                 const ui* me = tile_ptr(id);
205                     up = tile_ptr(id - N) + tile_words - row_words;
209                     lf = tile_ptr(id - 1) + row_words - 2;
211                     rh = tile_ptr(id + 1);
```

br 选行 • L250 – 265

```
255         t.join();
261         std::swap(tiles[id].current, tiles[id].next);
263     deque_A.swap(deque_B);
```

spin_lock.hpp • L1 – 12

```
 1 #pragma once
 2 #include <atomic>
 3
 4 // 最小自旋锁：同学的代码里用它保护任务队列
 5 struct SpinLock {
 6     std::atomic<bool> flag{false};
 7     void lock() {
 8         while (flag.exchange(true, std::memory_order_acquire)) {
 9         }
10     }
11     void unlock() { flag.store(false, std::memory_order_release); }
12 };
```

> 一句话记住：A 空表示这一步领完了，不表示森林永远不再着火。

5.7 按块读文本：seek_input、tell_input、read_forest_region

seek_input 以字节偏移定位，Windows 和其他平台分别调用不同的大偏移接口；tell_input 返回同一套单位的当前文件位置。它们不是解析整数的函数。以二进制方式打开，才能通过两次位置

相减得到物理行长。若一行是六个单数字、五个单空格和一个 LF，物理行宽 12；同样的内容若用 CRLF，物理行宽 13。用第一行量出的 stride 跳后续行，前提是整份文件行宽一致。这比题目对数值格式的要求更窄。

read_forest_region 读头部 n,m,t，根据 tile_side 申请两份块存储。真正关键在 local_side=n；这和第 289–293 行的‘四进程四象限’注释不同。随后仍用 rank/2、rank%2 乘 local_side 算行列起点。对 rank 0 两者都为零；对 rank 1 列起点已等于整图宽 n，并非右半张图的起点。读一批 tile_side 行后，parse(tid) 每隔 Threads 行选一行来转换，各人写不同的块内行。每个格直接取行起点加 2×列，这假设每个状态一个字符、相邻状态之间恰一个空格。它截取输入数字自身的两个二进制位交织写入 a,b，和 v7 的查表完全不同。

这一读入步骤分批建线程、join 后再读下一批，不能把模拟线程的常驻误写成整个程序从头到尾只建过一次线程。所有工作者完成后，文件重新定位到事件区，下一段才读行为事件。

br 选行 • L273–286

```
283     return ::_ftelli64(fp);
285     return static_cast<int64_t>(::ftello(fp));
```

br 选行 • L294–319

```
294 template <int TileSize, int Threads>
296                         std::vector<ui>& data_B, std::vector<TileLink>& tile_table) {
305     const int tiles_per_side = local_side / tile_side;
317     const int64_t stride = tell_input(fp) - base;
319     const int col_begin = (rank % 2) * local_side;
```

br 选行 • L327–344

```
334                     const char* src = row + size_t(2) * (col_begin + tc * tile_side);
```

br 选行 • L348–357

```
348         std::vector<std::thread> workers;
351             workers.emplace_back(parse, tid);
354             worker.join();
356     seek_input(fp, base + int64_t(n) * stride);
```

> 一句话记住：能算出某一行的物理地址，是因为输入满足特定排版，不是因为文本天生可随机访问。

5.8 两种魔法为何不以同一方式入队

handle_magic_ignite(rank,i,j,...) 用全局坐标判断归属，再依次确定局部块号、块内行、字号、字内位。先求 data[pos] & ~data[pos+1] & bit，只会选中树；点火要清低位 a、设高位 b，让 01 变 10（高位在前）。源码接着检查目标位置是否为火；原本就在烧的格也可能触发提名，这是允许重复请求但由 marked 去重的设计。火在块内，本块需要处理；若它恰在下沿，还得把下方邻块加入本步 A，而不是等本步传播完才发现边界。

revive_tile 只操作一块里的一个闭矩形，按字取交集。lo=0、hi=63 时 (~0<<lo)&(~0>>(63-hi)) 也能直接得到全 1；两个移位量各在 0..63，无须移 64。旧灰是 a&b；保留 a、只清 b，就从 11 变成树 01。把这一行换成 v7 的‘高面置 1、低面清 0’会把目标状态弄反。

handle_magic_revive 做的是两层裁剪：先用全局坐标与本进程地块求交，确认非空后才减去无符号起点；再把交集分配到每个相交小块。apply(tid) 按 job=tid,tid+thread_count,… 交错领块，每块只归一人。没点出新火的回春本身不用把所有复活块都排进转移队列：没有火邻时树不变，有火邻时原有火和边缘提名应覆盖那一块。这依赖活动块集合一直包含所有可能受火影响的位置。回春在当前图上已完成，所以即使块不参与转移，它的树也不会丢。

br 选行 • L361 – 386

```
364     const ui row_begin = ui(rank / 2) * LocalSide;
367         return;
374     data[pos] &= ~ignite;
379         const int stride = tiles_per_side + 2;
385             enqueue_next<tiles_per_side>(static_cast<int>(marked_id) - 1, marked, deque_A);
```

br 选行 • L394 – 404

```
395     for (int word = c0 / 64; word <= c1 / 64; ++word) {
398         const ui mask = (~ui{0} << lo) & (~ui{0} >> (63 - hi));
401             const ui ash = data[pos] & data[pos + 1] & mask;
402             data[pos + 1] &= ~ash; // 11 -> 01; all other states unchanged
```

br 选行 • L413 – 439

```
414     const ui top = std::max(i1, row_begin);
416     const ui bottom = std::min(i2, row_begin + LocalSide - 1);
419         return;
421     const int r1 = int(bottom - row_begin), c1 = int(right - col_begin);
437         workers.emplace_back(apply, tid);
```

> 一句话记住：惊雷引入新的火源；回春改变燃料，火源既有的影响范围才决定是否转移。

5.9 打包和解包边界：能通信与已通信之间

pack_boundary_fire(direction,mask,table,packed) 把选中的边缘块按掩码位从小到大的次序串起来。__builtin_popcountll 数一个整数的 1 位个数；返回值乘 TileSize 决定总共需要多少个 64 位字。S=16，mask 低三位为 101 时，选两段，一共 32 字也就是 256 字节有效载荷，不包括消息头。上下边缘原来就是一行连续字对，直接提炼火位即可。左右边缘是一列，得每行只抽一个位，再把 64 行的这些位压进一个字。字数相同，拼装的方法不同。

unpack_boundary_fire 反向按同一掩码顺序消费。接收方的左边对应发送方的右边；把收到的火放在模拟边缘的 b 位面，a 默认为零，旧内核的 b&~a 自然能识别火。它先用 assign 把整个方向的边沿填零，连被掩码省略的段也覆写；否则上一步在边上的火会像幽灵一样多活一拍。若一个边段有火，再把本地相应外缘的块加入 A。保证顺序、方向以及掩码配套是协议的一部分，只有一个裸字数组不能告诉接收者应该填哪条边。

检查 main：send_buffers、recv_buffers 定义后只有显式转成 void 的语句，没有 MPI_Send、MPI_Recv 或对应的非阻塞调用。传给 process_tasks 的 halo 是个对象，但四个向量并未通过这个主流程填入收到的远方火。两函数体现了接口和设计意图，不能当作已经打通的四节点交换。

br 选行 • L446 – 462

```
446 template <int TileSize, int LocalSide = 4096>
447 void pack_boundary_fire(int direction, ui mask, const TileLink* table, std::vector<ui>&
    packed) {
450     packed.resize(size_t(__builtin_popcountll(mask)) * TileSize);
453         if (!(mask & (ui{1} << i)))
```

br 选行 • L464 – 474

```
464             const int word = direction == 2 ? 0 : TileSize - 1;
467                 ui fire = 0;
470                     const ui burn = data[pos + 1] & ~data[pos];
471                     fire |= ((burn >> bit) & ui{1}) << k;
473                 packed[out++] = fire;
```

br 选行 • L482 – 506

```
482 void unpack_boundary_fire(int direction, ui mask, const std::vector<ui>& packed,
491     edge.assign(size_t(N) * stride, ui{0});
495         ui* dst = edge.data() + size_t(i) * stride;
499             any_fire |= fire;
506                     dst[size_t(w * 64 + k) * row_words + 1] = ((fire >> k) & ui{1}) << bit;
```

br 选行 • L509 – 514

```
509         if (any_fire) {
510             const int r = direction == 0 ? 1 : direction == 1 ? N : i + 1;
511             const int c = direction == 2 ? 1 : direction == 3 ? N : i + 1;
512             enqueue_next<N>(r * (N + 2) + c, marked, deque_A);
513         }
514     }
```

br 选行 • L658 – 661

```
658     std::vector<ui> send_buffers[2], recv_buffers[2];
659     FireHalo<tile_size, tiles_per_side> halo;
660     (void)send_buffers;
661     (void)recv_buffers;
```

> 一句话记住：网络上的火必须有发送、接收和解包三段，缺一段就不会越过进程。

5.10 常驻池：SpinBarrier、process_tasks 和 worker

SpinBarrier(int n) 保存到站人数；wait() 复用‘计数＋世代’的门。同 v8 的机制相似，但这里的队伍不是按 tid 固定分几行，而是共同抢 A 中不规则出现的任务。若只有一个活动块，16

人里最多一人计算这个块，不能凭常驻线程数承诺并行度。若 256 块全部活跃，任务较易分摊，可队列锁也会更忙。任务稀疏时算得少不代表墙钟按面积同比例缩短。

process_tasks 在一个短锁范围里做 empty、front、pop_front；必须把‘看见任务’与‘拿走它’连成一件事。它再把带边框编号还原成紧凑 id，tile_ptr 局部函数返回对应块的 current。如果本地存在上下左右邻块，替换初始的 halo 指针；若没有，那一方向由 halo 或空边界承担。随后调用 propagate_tile_and_append，用 A 领取、B 提名，互不颠倒。没有等待条件变量，也没有线程一直守着空 A 等 B 生长。

main 里的 worker 包时步：0 号执行魔法、重置下轮去重表、把待处理块的列表抄入 processed；第一道门后所有人抽取 A；第二道门后 0 号按 processed 逐块交换，再互换队列；第三道门才放行下一轮。把 std::fill(marked,...,false) 放在当前任务名单形成以后，正是为了让本步同一块也能被下一步重新提名。工人线程只为此模拟循环创建一次；处理回春矩形时的额外小队是另一码事。

br 选行 • L533 – 549

```
537         const unsigned g = gen_.load(std::memory_order_acquire);
539             cnt_.store(0, std::memory_order_relaxed);
540             gen_.store(g + 1, std::memory_order_release);
542             while (gen_.load(std::memory_order_acquire) == g) POOL_PAUSE();
549     std::atomic<unsigned> gen_{0};
```

br 选行 • L595 – 617

```
599             if (A.empty()) return;
607         const ui* me = tile_ptr(id);
613         if (r > 0) up = tile_ptr(id - N) + tile_words - row_words;
614         if (r + 1 < N) dn = tile_ptr(id + N);
616         if (c + 1 < N) rh = tile_ptr(id + 1);
```

br 选行 • L681 – 704

```
684             bar.wait();                         // 事件 + 脏表清零完成
687             if (tid == 0) {                     // 发布新状态：交换指针 + 换队列
693                 deque_A.swap(deque_B);
700     std::vector<std::thread> pool;
704     for (auto& th : pool) th.join();
```

> 一句话记住：队列决定谁做哪一块，栅栏决定哪一代可以对外可见。

5.11 输出与入口：rank 0 没有被特判

write_forest_output(path,n,tiles) 按全图行顺序恢复文本，而非按活动队列输出。给定全局行 r，tr=r/side 找块行、rr=r%side 找块内行，然后横扫每个块列。对一对字还原数值为 a 的这位加 b 的这位乘 2。每写一格先写空格，到行末用 q[-1] 改成换行，尾块再 flush。即使一大片森林 100 步都没入队，完整结果也必须把它打印出来。

main 写死 tile_size=16、thread_cnt=16、tiles_per_side=16384/(64×16)=16，开局把 16×16 个真块全部塞入 A。用全块开始是因为程序并没有在读入时建好初始着火清单。它读每条事件存到 unordered_map（hash map，按键查值的散列表），键是步号。主路径使用 MPI 的三个调用：MPI_Init 初始化环境，MPI_Comm_rank 将全局进程编号写入 rank，MPI_Finalize 结束本进程的 MPI 使用。没有 MPI_Comm_size 来判断启动数，也没有 rank!=0 就退出的分支。第 708 行每个跑到这里的进程都尝试写同一路径。不能按任务说明原样宣称‘只有 Rank 0 干主要活’；那是 v8 有 MPI 时的行为。

具体比较：官方参考在 n=16384、64 个进程均分时，每个进程应拿 256 行；rank 0 是 0..255，rank 1 是 256..511，rank 63 是 16128..16383；一般 rank k 对应 256k..256k+255。它由 Scatterv 分发，用 Isend/Irecv 送收邻行，Wait 等完成，再 Gatherv 汇合。br 的注释还写有四象限设计，但读入改为 rank 的 local_side=n，四象限的偏移计算却未一起改。取 rank 2，则 row_begin=n，定位已到地图行之后。没有收边界、聚合结果或唯一写者的证据。单进程固定尺寸是可能的使用情景，并非这本书验证过的可运行承诺。

br 选行 • L553 – 570

```
553 static void write_forest_output(const char* path, int n, const std::vector<TileLink>& tiles)
    {
561     std::vector<char> buf(1u << 22);
562     char* q = buf.data();
565         std::fwrite(buf.data(), 1, size_t(q - buf.data()), fo);
566         q = buf.data();
```

br 选行 • L574 – 585

```
575                     if (q + 2 > qend) flush();
577                     *q++ = char('0' + v);
578                     *q++ = ' ';
582         q[-1] = '\n';
```

br 选行 • L621 – 645

```
621 int main(int argc, char** argv) {
631     int rank;
633     std::FILE* fp = std::fopen(argc > 1 ? argv[1] : "forest.in", "rb");
636     std::vector<TileLink> tile_table;
640         for (int c = 1; c <= tiles_per_side; ++c) enqueue_next<tiles_per_side>(r *
    marked_stride + c, marked, deque_A);
```

br 选行 • L647 – 657

```
647     std::unordered_map<int, std::vector<int>> events;
```

br 选行 • L700 – 716

```
700     std::vector<std::thread> pool;
704     for (auto& th : pool) th.join();
708     write_forest_output<tile_size>(argc > 2 ? argv[2] : "forest.out", n, tile_table);
710     auto ms = [](auto a, auto b) { return std::chrono::duration<double, std::milli>(b -
    a).count(); };
714     return 0;
```

> 一句话记住：要证明一个进程不写文件，得在入口到输出的路径上找到实际分支。

5.12 手推一对相邻块：何时可以不算

画两个横向相邻块 U、V。第 k 步后只有 U 的最右列出现新火，V 最左列是一棵树。U 的 right_burn 被设为非零，所以 U 与 V 都在 k+1 的 A 中。处理 V 时，curr[-1] 读 U 边字，`_shld<1>` 把那一位引到 V 第一位，于是 V 的树成为新火。U 的旧火这拍要变灰，因此 U 也绝不能因‘火已经向右传’而跳过。等大家写完 next，才交换 U、V 的链接，其他未动的块继续保留 current。这串因果能同时检验字边界、块边界、活动队列和双缓冲。

把两个块改成分属不同进程，故事在何处断开？V 不能直接解引用另一个进程的地址。它要收到 U 的火边，解包进自己的 halo，补上 V 的入队请求，再执行同一内核。br 源中局部算子与打包器都能指出来，但实际入口没有组装这条跨进程路径。两位作者都用位平面与线程；v1→v8 并非一直使用同一布局，v8 相对 v7 才是基本保留布局而换团队；br 相对这两版既换编码又换了调度单位。这个区分能防止把‘少算一些块’误归功给另一种线程库。

> 一句话记住：路径完整时才能把设计当成行为；保留在文件里的工具不等于主程序已经使用它。

APPENDIX A

附录 A　术语第一次出现在哪里

这一章你会得到什么：认出拼法、查到含义，并能回到正文看它到底干了什么。顺序以正文从第 0 章到第 5 章，再到性能账本和问题记录的实际出现为准；A 组补充只在词典中详释的源码词。源码位置用 v1 – v8、br、锁、参考标出，必要时可直接翻原件。

正文第 0 章

术语	英文全称或原名	一句话定义	在本题里的作用	首次位置
*	pointer declarator, dereference or multiplication	区分指针声明、读写所指对象与乘法	p 是地址，*p 是地址处的字符	第 0 章；v1 L12
时间步	time step	一次魔法加一次同步转移	决定哪些火能在这一拍传播	0.1；v1 L78
状态	state	一个格子某时刻的取值	在空、树、火、灰中取一	0.1；v1 L105
闭矩形	inclusive rectangle	两端坐标都计入的矩形	单点回春也要生效	0.1
双缓冲	double buffering	用不同存储承载旧代与新代	防止一拍内连续传火	0.1
double	double precision floating-point type	双精度浮点类型	保存可带小数的耗时	0.1；v1 L9
坐标	coordinate	以行、列定位一个格子	外部从零，v7 内部真行从一	0.2；br L60
文本文件	text file	以字符存数据	地图数字要编码、解码	0.3
标准错误流	standard error stream	专门写诊断的输出通道	计时不污染森林文件	0.3
字段	token	切分出的一段文字	比对脚本逐项比较	0.3
节点	compute node	一台独立计算机	评测有四台	0.4
核	processor core	执行指令的硬件单元	64 核构成理想预算	0.4
进程	process	拥有独立地址空间的程序实例	不能直接引用另一进程的地图	0.4；v8 L10
线程	thread	进程中共享地址空间的执行路径	可并行读同一代位面	0.4；v6 L118
thread	standard thread object	代表一条标准 C++ 线程	v8 和 br 显式建队	0.4；v8 L4
MPI	Message Passing Interface	跨进程传递消息的标准接口	参考实现交换上下边界	0.4；v8 L9
OpenMP	Open Multi-Processing	共享内存并行指令与运行库	v5 – v7 组织工作	0.4；v6 L112
墙钟	wall-clock time	实际经过的时间	评测限时的口径	0.4
位	bit	一个二值信息位置	两位可编码四个状态	0.5
字	word	此处为一个 64 位无符号整数	字循环一次处理至多 64 列	0.5
位平面	bit-plane	把同一种状态属性的位排在一起	一字保存 64 个格子的同一属性	0.5；v4 L55

术语	英文全称或原名	一句话定义	在本题里的作用	首次位置
行窗口	rolling window	复用相邻几行的暂存空间	v7 不再保存全图火位	0.5
分块	tile	一片可以独立排任务的正方形网格	br 按活跃区域调度	0.5

> 一句话记住：查到定义后，带着末列的位置回到代码里验一次。

正文第 1 章

术语	英文全称或原名	一句话定义	在本题里的作用	首次位置
int	integer type	带符号普通整数类型	状态、坐标和循环编号	1.1；v1 L12
if	if statement	条件为真才进入分支	惊雷只作用于树	1.1；v1 L20
for	for statement	依次执行初始化、条件判断、循环体与更新	遍历行、字、位、事件	1.1；v1 L42
std::	standard namespace qualification	从标准库命名空间查找名字	区分库函数和程序私有函数	1.1；v1 L6
::	scope resolution	在指定作用域查找名字	从 Clock 或 std 查找成员	1.1；v1 L6
cin	standard character input stream	C++ 标准字符输入流	v1 用抽取运算读取整数	1.1；v1 L40
>>	right shift or stream extraction	依操作数决定移位或流输入	除以 64 找字，或读取整数	1.1；v1 L40
cout	standard character output stream	C++ 标准字符输出流	v1 按格格式化输出	1.1；v1 L128
<<	left shift or stream insertion	依操作数决定移位或流输出	区别位面传播和打印用法	1.1；v1 L21
二维数组	two-dimensional array	以行列两级定位元素的结构	v1 用嵌套容器模拟	1.1
vector	std::vector, dynamic array	管理一段连续可变长存储的容器	装字节、状态字、事件或线程	1.1；v1 L2
GiB	gibibyte	2^{30} 字节	估算两张 int 图	1.1
freopen	file stream reopen	按路径和模式重接既有流，失败返回空	v1 把 stdin 或 stdout 接到文件	1.1；v1 L26
++	increment	数值加一或指针移到下一元素	消费事件或移动字符游标	1.1；v1 L42
==	equality comparison	比较两个值是否相等	识别一种事件或一个边界	1.1；v1 L56
else	else clause	前一条件不成立的分支	决定是否保留旧状态	1.1；v1 L116
char	character type	容纳一个字符代码的类型	字符 2 要减字符 0 才得数值 2	1.2；v1 L12
const	const qualifier	限制通过这个名字修改值	游标可移动，不能通过它改字符	1.2；v2 L36
fopen	file open	按路径与模式开流，失败返回空指针	rb 读原始字节、wb 写结果	1.2；v2 L14
fread	file read	读入指定数目和大小的项，返回完整项数	与预期字节数比较	1.2；v2 L32

术语	英文全称或原名	一句话定义	在本题里的作用	首次位置
fclose	file close	关闭文件流并报告成功或失败	结束读或完成写出	1.2；v2 L33
while	while statement	只要条件成立就重复执行	跳过空白或消费到期事件	1.2；v2 L40
缓冲区	buffer	临时存放一批数据的区域	合并文件读写	1.2
swap	swap containers	交换两个对象或容器的内容归属	轮换窗口与两代地图	1.2；v2 L136
!	logical negation	把真假取反	检查文件句柄为空	1.2；v1 L26
fprintf	formatted file print	按格式向指定流写文本	格式化错误路径及毫秒数	1.2；v1 L133
stderr	standard error	C 的标准错误流	向人报告用法和耗时	1.2；v1 L133
%	remainder	求整数除法的余数	用块坐标找块内位置	1.2；v1 L133
argv	argument vector	保存各个参数字符串指针的数组视图	argv[1] 输入、argv[2] 输出	1.2；v1 L12
return	return statement	结束当前函数并可带回值	读数返回整数，main 返回状态	1.2；v1 L9
data()	container data pointer	返回连续元素区首地址	传给文件函数或建立游标	1.2；v2 L32
&	address-of, reference or bitwise AND	看上下文区分取地址、引用声明与按位交集	MPI 填 rank、队列按引用传递、火位求交	1.2；v1 L79
&&	logical AND	两边都为真，左边假时不求右边	先检查边界再访问数据	1.2；v1 L79
uint8_t	unsigned 8-bit integer	恰为 8 位的无符号整数	v3 的一个元素存一格	1.3；v3 L62
unsigned	unsigned integer	无符号整数	保存世代编号和移位所用类型	1.3；v8 L59
size_t	size type	能表示对象大小的无符号整数类型	计算字偏移和字节计数	1.3；v2 L153
static_cast	static cast	显式执行编译期可检查的类型转换	先转换尺寸，再算偏移	1.3；v3 L60
explicit	explicit constructor specifier	禁止利用该构造函数做隐式转换	参与人数要明确传给栅栏	1.3；v8 L57
byte	byte	由八位组成的字节，在本题平台成立	估算地图容量	1.3
stride	row stride	相邻两行对应位置的地址间距	分别以格、字或字节计	1.3；v3 L59
MiB	mebibyte	2^{20} 字节	衡量整幅位面与输出缓存	1.3
按位与	bitwise AND	逐位取交集	筛出树、火或灰	1.4
按位或	bitwise OR	逐位取并集	合并四个方向的火来源	1.4
~	bitwise complement	每一位独立取反	从高面中去掉低面已置位位置	1.4；v4 L148
uint64_t	unsigned 64-bit integer	恰为 64 位的无符号整数	一位表示一格的某个属性	1.4；v4 L62
掩码	bit mask	选择待处理二进制位置的一串位	回春只改变矩形里的灰	1.4；v4 L143

术语	英文全称或原名	一句话定义	在本题里的作用	首次位置
`0ULL`	unsigned long long zero	具有指定宽度类别的无符号零	取反得到整字全一	1.4；v4 L148
`#pragma`	pragma directive	向编译器提供实现或并行指示	OpenMP 以此引入	1.5；v5 L159
`#pragma omp parallel`	OpenMP parallel construct	同队线程各执行结构块	v6、v7 跨时步保留团队	1.5；v5 L159
`#pragma omp parallel for`	OpenMP combined parallel worksharing loop	同时建立团队和分派循环	v5 对两段扫描分别派工	1.5；v5 L159
`schedule(static)`	static loop schedule	提前给团队分配循环迭代	固定大段，避免逐项抢任务	1.5；v5 L159
`static`	static storage or internal linkage	视位置表示静态存储期或内部链接	br 标记表与文件内辅助函数	1.5；v5 L159
`#pragma omp single`	OpenMP single construct	由团队任一线程执行，通常末尾等待	v7 一人处理事件及交换	1.6；v6 L122
`#pragma omp for`	OpenMP worksharing-loop construct	在已有团队内分摊循环	v6 分派造火图与计算两段	1.6；v6 L171
`KiB`	kibibyte	2^{10} 字节	衡量一个线程的三行窗口	1.7

> 一句话记住：查到定义后，带着末列的位置回到代码里验一次。

正文第 2 章

术语	英文全称或原名	一句话定义	在本题里的作用	首次位置
`#include`	header inclusion	在预处理时引入头文件声明	让编译器识别库设施	第2章 第 1–25 行；v1 L1
`[]`	subscript or lambda introducer	依上下文表示下标或匿名函数开头	取数组元素或建立小函数	第2章 第 1–25 行；v1 L8
`lambda`	lambda expression	在使用位置构造可调用对象	解析整数、刷新缓冲、线程工作	第2章 第 1–25 行
`namespace`	namespace definition	给名字划分范围	匿名命名空间内放时钟别名	第2章 第 1–25 行；v1 L5
`using`	type alias declaration	为一种类型定义别名	Clock 和 ui 都不是新造的类型	第2章 第 1–25 行；v1 L6
`steady_clock`	monotonic clock	用来量经过时长的单调时钟	系统日历调整不应让阶段时长倒走	第2章 第 1–25 行；v1 L6
`auto`	placeholder type specifier	令编译器从初始化推断类型	保存时刻、匿名函数和迭代元素	第2章 第 1–25 行；v1 L8
`time_point`	clock time point	某种时钟上的一个时刻	传给 ms_since 的起点	第2章 第 1–25 行；v1 L8
`duration`	time duration	带计数类型和单位的时间间隔	换算成浮点毫秒	第2章 第 1–25 行；v1 L9
`std::milli`	millisecond period	一千分之一秒为时长周期	让 count 的数值采用毫秒	第2章 第 1–25 行；v1 L9
`milli`	one thousandth ratio	表示千分之一的单位比例	duration 的一格计数为 1 毫秒	第2章 第 1–25 行；v1 L9

术语	英文全称或原名	一句话定义	在本题里的作用	首次位置
now()	clock now	返回该时钟当前时刻	起点终点相减	第2章 第 1 - 25 行；v1 L9
count()	duration count	取得以当前单位计量的数值	传给 printf 风格的输出	第2章 第 1 - 25 行；v1 L9
int main(int argc, char** argv)	program entry signature	参数个数加参数字符串指针	题面通过命令行传入两个文件名	第2章 第 1 - 25 行；v1 L12
main	program entry point	程序入口函数	接收路径并组织全过程	第2章 第 1 - 25 行；v1 L12
argc	argument count	命令行参数数目，含程序名	检查是否提供两个路径	第2章 第 1 - 25 行；v1 L12
char**	pointer to pointer to char	指向字符串指针的指针	argv 按下标取路径	第2章 第 1 - 25 行；v1 L12
cerr	standard character error stream	C++ 标准错误流	v7 写用法提示	第2章 第 1 - 25 行；v1 L21
min	minimum	返回较小的候选值	截住末段的行号	第2章 第 1 - 25 行；v7 L118
fill	fill range	把半开区间中的每个元素赋给同一值	清边沿火或去重表	第2章 第 1 - 25 行；v7 L174
FILE	file stream object	C 文件流的类型	以指针传递已打开的文件	第2章 第 26 - 55 行；v2 L26
fseek	file seek	以起点、当前位置或末端为基准移动文件位置	测长和回到开头	第2章 第 26 - 55 行；v2 L28
SEEK_END	seek relative to end	以文件末端定位的常量	测量输入字节数	第2章 第 26 - 55 行；v2 L28
long	long integer type	带符号整型，宽度依平台	ftell 的返回类型	第2章 第 26 - 55 行；v2 L29
ftell	file tell	返回当前文件位置，失败可返回负数	末端位置当作文件长度	第2章 第 26 - 55 行；v2 L29
SEEK_SET	seek relative to start	以文件开头定位的常量	定位首字节或指定行	第2章 第 26 - 55 行；v2 L30
!=	not equal	比较值不相等	短读时进入失败分支	第2章 第 26 - 55 行；v3 L36
[&]	reference capture	匿名函数按引用使用外部局部量	跨调用更新字符游标	第2章 第 26 - 55 行；v2 L39
按位或赋值	bitwise OR assignment	只把掩码选中的位设成 1	不覆盖同一字中其他树	第2章 第 56 - 84 行
struct	structure type declaration	将相关字段组合成一种类型	记录不同魔法的字段	第2章 第 85 - 110 行；v1 L47
reserve	reserve capacity	预留容量而不创建新元素	线程池和链接表少搬家	第2章 第 85 - 110 行；v8 L291
+=	addition assignment	把计算量加到已有变量	累计若干时间步耗时	第2章 第 85 - 110 行；v1 L102
omp_get_num_threads	OpenMP team size query	无参数，取当前团队人数	算行段长度	第2章 第 111 - 140 行；v7 L113

术语	英文全称或原名	一句话定义	在本题里的作用	首次位置
omp_get_thread_num	OpenMP thread number query	无参数，取团队内零基编号	找本线程的起始行	第2章 第 111 – 140 行；v7 L114
std::min	minimum of two values	比较两个参数取较小者	截去超出本段的终行	第2章 第 111 – 140 行；v7 L118
nowait	OpenMP nowait clause	在允许的构造末尾取消隐式等待	v7 的 single 未指定该子句	第2章 第 111 – 140 行
&=	bitwise AND assignment	用按位交集更新原值	配合补集清除特定位	第2章 第 141 – 168 行；v4 L157
?:	conditional expression	按条件选择两个表达式之一	字两端取不同掩码	第2章 第 141 – 168 行
begin()	begin iterator	指向区间第一个元素	与 end 配对用于填充或复制	第2章 第 169 – 188 行；v7 L174
end()	past-the-end iterator	指向区间末端之后	不应解引用	第2章 第 169 – 188 行；v7 L174
#pragma omp barrier	OpenMP barrier construct	要求团队全部到达同步点	交换前等待所有写入完成	第2章 第 189 – 221 行；v7 L209
size()	container size	返回已存在的元素个数	字符向量里等于字节数	第2章 第 222 – 261 行；v2 L150
fwrite	file write	写入多项并返回成功写出的完整项数	将一段字符送入输出流	第2章 第 222 – 261 行；v2 L153
memset	memory set	按字节填充一段存储	只用于解释为何复用输出缓冲时不必先清零	第2章 第 222 – 261 行

> 一句话记住：查到定义后，带着末列的位置回到代码里验一次。

正文第 4 章

术语	英文全称或原名	一句话定义	在本题里的作用	首次位置
mpiicpx	Intel MPI C++ compiler wrapper	面向 C++ 的 MPI 编译驱动	题目指定的编译入口	4.1；v8 L5
-O3	optimization level three	请求较高级别优化	不等同主动启用 OpenMP	4.1；v8 L5
-xHost	host processor target	为编译主机处理器指令集优化	限制编译目标代机条件	4.1；v8 L5
-o	output file option	指定生成文件名	题面命名为 forest	4.1；v8 L5
mpirun	MPI launcher	启动多个进程	与每进程线程数是不同维度	4.1
-np	number of processes	向启动器指定进程数量	评测指定 64	4.1
-fopenmp	OpenMP enable option	启用 OpenMP 指令的一种编译选项	题面命令未列	4.1；v8 L5
-fiopenmp	Intel OpenMP enable option	启用 Intel 实现的一种编译选项	说明并非只有一种启用拼法	4.1
class	class type	把状态与方法封装为类型	SpinBarrier 把计数细节藏起来	4.2；v8 L55
public	public access	允许外部使用成员	调用 wait 和构造器	4.2；v8 L56

术语	英文全称或原名	一句话定义	在本题里的作用	首次位置
private	private access	仅类自身及获权者可直接使用	避免外部随意重置栅栏	4.2；v8 L68
atomic	atomic object	为特定对象提供不可分割的并发操作	栅栏的计数、代号以及锁旗标	4.2；v8 L22
fetch_add	atomic fetch and add	原子地加值并返回旧值	认出末位到站线程	4.2；v8 L60
_mm_pause	processor pause hint	忙等时的处理器提示	不承担同步或释放锁的工作	4.2；v8 L40
#if	conditional preprocessing	按预处理条件保留一段源码	选择平台或可用头文件	4.2；v8 L31
defined	macro definition test	判断指定宏是否存在	选择支持的指令集分支	4.2；v8 L31
#define	macro definition	定义预处理宏	设置可选 MPI 与等待提示	4.2；v8 L34
#else	alternate preprocessing branch	条件编译未选前支时改选此支	非 x86 等情况下使用替代写法	4.2；v8 L41
void	void type	函数不返回数据值	wait、lock、unlock 的价值在操作本身	4.2；v8 L42
#endif	end conditional preprocessing	结束条件编译区段	确定 MPI 专属语句覆盖范围	4.2；v8 L35
load	atomic load	取一次原子对象当前值	读取世代或累计计时	4.2；v8 L59
memory_order_acquire	acquire order	获取对方已发布的工作	读新世代后使用上一阶段数据	4.2；v8 L59
store	atomic store	把新值发布到原子对象	推进世代或解锁	4.2；v8 L61
memory_order_relaxed	relaxed order	只提供本对象的原子操作，不单独发布其他数据	栅栏先重置计数	4.2；v8 L61
memory_order_release	release order	把此前的修改随发布操作交给读者	推进代号或交回队列锁	4.2；v8 L62
getenv	environment variable lookup	按名取环境字符串，不存在则返回空指针	v8 查询 FIRE_THREADS	4.3；v8 L184
FIRE_THREADS	application thread-count setting	v8 读取的环境变量	覆盖默认线程人数	4.3；v8 L184
atoi	ASCII to integer	从字符串解析整数，无法转换时得到 0	v8 随后限制最小团队规模	4.3；v8 L184
hardware_concurrency	hardware concurrency hint	给出并行能力提示，可能为零	不能当作可用绑定核数	4.3；v8 L185
I_MPI_PIN_DOMAIN	Intel MPI process pinning domain	配置绑定域的环境变量	题面设置为 core	4.3
join	thread join	等待一个可汇合线程执行结束	发布或结束前确认工作完成	4.3；v8 L8
emplace_back	construct at end	在容器末端直接构造元素	以函数与编号创建线程对象	4.3；v8 L292
auto&	deduced reference	由表达式推导对象类型并保留引用	按引用遍历线程对象	4.3；v8 L294
max	maximum	返回较大的候选值	求两个矩形或行段的起端	4.4；v8 L219
->	member access through pointer	通过指针访问对象成员	halo 指针调用 tile_edge	4.4；v4 L213
clear	clear container	移除元素而不承诺释放容量	输入结束或下一队列重置	4.5；v8 L176

术语	英文全称或原名	一句话定义	在本题里的作用	首次位置
shrink_to_fit	capacity reduction request	请求容量缩至接近现有大小，不强制执行	v8 尝试释放大输入缓冲	4.5；v8 L177
__has_include	header availability test	预处理时检查某头文件能否找到	并不验证 MPI 库已链接	4.6；v8 L31
FIRE_HAS_MPI	compile-time MPI feature macro	源码内部声明的条件编译标记	启用编号查询与非零进程收尾	4.6；v8 L34
MPI_Init	MPI initialization	初始化进程的 MPI 环境	在查询进程编号前调用	4.6；v8 L80
MPI_Comm_rank	communicator rank query	把本进程在通信组内的编号写入输出参数	v8 用于区分唯一计算者	4.6；v8 L82
MPI_COMM_WORLD	world communicator	通常包含本次启动全部参与进程的通信组	供 rank 查询	4.6；v8 L82
MPI_Finalize	MPI finalization	完成 MPI 环境收尾，是集体操作	非零 rank 可先进入收尾	4.6；v8 L84

> 一句话记住：查到定义后，带着末列的位置回到代码里验一次。

正文第 5 章

术语	英文全称或原名	一句话定义	在本题里的作用	首次位置
template	template declaration	按编译期参数生成具体实现	br 用块宽、队伍人数定制内核	5.2；br L18
static_assert	compile-time assertion	条件不满足则拒绝编译	拒绝位移量大于等于 64	5.2；br L20
constexpr	constant expression specifier	可用于常量表达式的声明方式	固定块尺寸、裁剪编译期分支	5.2；br L22
inline	inline specifier	函数说明符，不保证展开	用在小型双字移位函数	5.2；br L19
ASM_SHLD	assembly shift feature macro	选择双字移位汇编形式	未定义则表达式实现	5.2；br L21
deque	double-ended queue	支持从两端操作的顺序容器	A 取本步任务，B 收下一步任务	5.3；br L8
bool	Boolean type	真或假	锁旗标及块的去重记录	5.3；br L37
bool*	pointer to Boolean	指向布尔标记数组	标记哪些块已经入队	5.3；br L37
push_back	append element	在尾端添加元素	将块加入工作队列	5.3；br L43
nullptr	null pointer literal	不指向对象的指针值	物理边界传空表示没有邻火	5.5；br L63
empty()	empty check	检查容器是否为空	不对空队列调用 front	5.5；br L147
SpinLock	spin lock	等待时反复测试的互斥锁	保护活动块队列	5.6；br L52
goto	jump to label	跳转到当前函数中的指定标号	旧 node_step 跳到等待工作者处	5.6；br L222
false	Boolean false	布尔假值	未占锁或仍未排队	5.6；br L681
lock()	acquire exclusive lock	尝试独占临界区，未获时等待	保护共享队列	5.6；锁 L7

术语	英文全称或原名	一句话定义	在本题里的作用	首次位置
exchange	atomic exchange	原子换新值、返回旧值	抢锁者由旧旗标判定是否成功	5.6；锁 L8
true	Boolean true	布尔真值	块已经被标记或锁已占用	5.6；br L39
unlock()	release exclusive lock	释放互斥	让下一工作者能领取任务	5.6；锁 L11
lock_guard	scoped lock guard	构造时加锁、离开作用域时解锁	短范围封住取任务或排任务	5.6；br L119
mutex	mutual exclusion	使临界区互斥进入的设施	用作与自旋的比较，不是 br 实际队列锁类型	5.6；br L10
const ui*	pointer to read-only words	通过此指针只读字内容	内核只读旧块	5.6；br L50
#pragma once	single inclusion pragma	同一编译单元只处理该头一次	spin_lock.hpp 避免重复定义	5.6；锁 L1
_ftelli64	Windows 64-bit file tell	查询大文件位置	算第一行的物理长度	5.7；br L283
int64_t	signed 64-bit integer	恰为 64 位的带符号整数	存物理文件偏移	5.7；br L273
ftello	POSIX file tell	用 off_t 报告当前位置	与 fseeko 的单位配套	5.7；br L285
static_cast<int>	explicit conversion to int	把值转换到 int	将尺寸或索引传给整型接口	5.8；br L316
std::max	maximum of two values	比较两个参数取较大者	求矩形交集的起端	5.8；v8 L219
__builtin_popcountll	population count	数无符号长长整数中置位数	为边界选中块分配打包空间	5.9；br L450
assign	assign a range or count	用给定范围或数量的元素替换内容	重建 halo 或复制 processed	5.9；br L491
MPI_Send	blocking send	阻塞发送接口	仅作为 br 主路径没有的通信举例	5.9
MPI_Recv	blocking receive	阻塞接收接口	同样没有出现在 br 主路径	5.9
resize	resize container	改变已有元素数量	为位数据与打包内容提供空间	5.9；br L307
front	front element	取得队列首元素	先读编号再弹出	5.10；br L185
pop_front	remove first element	移除首元素，不返回取出的值	领取一个活动块	5.10；br L186
unordered_map	unordered associative container	按键查关联值的散列表	br 用步号查事件	5.11；br L12
MPI_Comm_size	communicator size query	取通信组参与进程数	参考版按这个数分行，br 入口无此检查	5.11；参考 L36

> 一句话记住：查到定义后，带着末列的位置回到代码里验一次。

问题记录

术语	英文全称或原名	一句话定义	在本题里的作用	首次位置
fgets	read a line into a buffer	最多读容量减一的字符并终止字符串	br 按行装图或事件	D.2；br L299
sscanf	scan formatted string	从字符缓冲按格式提取字段并返回成功数	br 解析头行与事件	D.2；br L300

> 一句话记住：查到定义后，带着末列的位置回到代码里验一次。

补充源码用语

术语	英文全称或原名	一句话定义	在本题里的作用	首次位置
#ifdef	if macro defined	若宏已定义才编译相应段	可选 MPI 与汇编分支	附录 A；v8 L79
#ifndef	if macro not defined	宏未定义才保留此段	br 可排除主程序	附录 A；br L517
-machinefile	machine file option	指定参与主机清单	题面每节点 16 个进程位置	附录 A
. / ->	member access	前者访问对象成员，后者通过指针访问	从事件取时间、从 halo 取块边缘	附录 A
FIRE_NO_MAIN	main exclusion feature macro	预处理时可排除入口等代码	方便辨认库段和可执行段	附录 A；br L517
I_MPI_PIN	Intel MPI process pinning	配置进程绑定的环境变量	题面设置为 1	附录 A
MPI_Abort	MPI abort routine	要求异常终止通信组中的任务	参考版打开文件失败时使用	附录 A；参考 L49
MPI_Bcast	broadcast	将一份数据从根进程广播到全组	参考版发布输入参数和事件	附录 A；参考 L85
MPI_Gatherv	variable-count gather	按各进程数量与偏移收集	参考版收齐结果	附录 A；参考 L224
MPI_INT	MPI integer datatype	消息元素按 C 的 int 描述	参考版传行列和整数格	附录 A；参考 L85
MPI_Irecv	nonblocking receive	登记非阻塞接收并给出请求	参考版接邻行	附录 A；参考 L170
MPI_Isend	nonblocking send	启动非阻塞发送并给出请求	参考版送自己边缘行	附录 A；参考 L169
MPI_Request	MPI request handle	标识一个异步操作的请求	参考版保存发送和接收请求	附录 A；参考 L159
MPI_STATUS_IGNORE	MPI status ignore indicator	允许调用不返回状态结构	等待邻行到达	附录 A；参考 L184
MPI_Scatterv	variable-count scatter	按每进程给定数量与偏移分发数据	参考版分行	附录 A；参考 L116
MPI_Wait	request completion wait	等请求完成	收到边界才更新	附录 A；参考 L184
MPI_Wtime	MPI wall-clock timer	返回用于量经过时间的秒数	参考版记录阶段耗时	附录 A；参考 L33
ULL	unsigned long long suffix	标明无符号长长整数字面量	构造能选中位 63 的 1	附录 A
[]{}	array value initialization	为固定数组提供空初始化列表	坐标字段初值为零	附录 A

术语	英文全称或原名	一句话定义	在本题里的作用	首次位置
`_fseeki64`	Windows 64-bit file seek	用大偏移重新定位文件	br 的 Windows 偏移分支	附录 A；br L275
`asm`	inline assembly	在高级语言内嵌入机器指令模板	ASM_SHLD 分支可选	附录 A；br L25
`continue`	continue statement	跳到当前循环下一次迭代	跳过未选中的边段	附录 A；br L454
`exit`	process termination	退出整个进程	br 打不开输出文件时退出	附录 A；br L559
`find`	lookup by key	返回对应迭代器，找不到等于 end	只在有事件的时间步处理魔法	附录 A；br L671
`for (;;)`	unbounded for loop	省略循环条件，靠显式控制流退出	队列空时返回	附录 A；br L595
`fscanf`	formatted stream scan	按格式从文件流读数据	br 注释提及，主入口读事件用 fgets 和 sscanf	附录 A；br L293
`fseeko`	POSIX file seek	用 off_t 类型偏移定位	br 的非 Windows 路径	附录 A；br L277
`int&`	reference to integer	为已有整数提供可修改的别名	read_forest_region 写回尺寸	附录 A；br L295
`memory_order_acq_rel`	acquire-release order	合并获取与释放的读改写顺序	累计多人到站	附录 A；v8 L60
`off_t`	file offset type	表示文件偏移的类型	非 Windows 大文件位置转换	附录 A；br L277
`operator ()`	call operator	函数对象接受参数的调用形式	read_int 和 flush 加括号才执行	附录 A
`shld`	double-precision left shift	从相邻字补位的双字移位	拼左右相邻边界	附录 A；br L25
`sizeof`	size of object or type	给出对象或类型所占字节数	控制读取事件行的缓冲上限	附录 A；br L299
`stdin`	standard input	默认输入流	v1 重定向到输入路径	附录 A；v1 L26
`stdout`	standard output	默认输出流	v1 重定向到结果路径	附录 A；v1 L31
逻辑或	logical OR	有一边真就为真	识别不在数字范围的字符	附录 A

> 一句话记住：查到定义后，带着末列的位置回到代码里验一次。

APPENDIX B

附录 B　复杂度与性能账本

这一章你会得到什么：给定边长、步数和实际参加工作的核数，能算出容量、逻辑工作量和时间预算，也知道哪里必须停下来要实测数据。

B.1　先讲记账单位

设 n 为边长，t 为步数，W=⌈n/64⌉，P 为单进程实际工作线程数；Z=(n+2)W 是作者位图中一张面的字数。下表统计主要状态空间，输入文本、事件记录、输出缓冲另记。时间复杂度（time complexity，规模增大时操作次数的增长方式）和空间复杂度（space complexity，活跃存储随规模增长的方式）不是秒数或内存监控截图。按字计的一轮 nW 与按格计的 n^2 在固定位宽机器上的渐近阶仍相同，差的是每次操作干多少格。

版	每步转移的工作与额外工作	主要状态空间，不含文本	实测与可推算的上限
v1	n^2 个格判断，加两次大图复制量级；回春逐格	两个 $(n+2)^2$ 的 int 网格，行容器额外计	未附全程实测；无并行队伍，不能预算成 64 核
v2	同为 n^2 判断，不再两遍整图复制；回春逐格	两张二维 int，预分配；新增整文件及 4 MiB 输出缓冲	未附；省复制不等于总时长减半
v3	n^2 个字节状态判断；回春逐格	$2(n+2)^2$ 字节，另有整文件与输出缓冲	未附；相对 v2 状态有效字节按 4 字节 int 约为四分之一
v4	扫 Z 字造 fire，再用 nW 个字更新	5×8Z 字节＝两张旧面、两张新面、一张火图	未附；每字含至多 64 格，不能写成整题 64 倍
v5	与 v4 同，两遍各自建并行区	同 v4；另计线程运行库	未附；均匀且无限带宽时，并行部分理想可缩至约 1/P
v6	两阶段仍需串接，团队包住 t 步	同 v4；另计团队与同步	未附；少进并行区，不代表少造火图
v7	约 nW 字更新，取火约同量；段首重复取邻行，上界另计 O(PW)	4×8Z + 3×8WP 字节	未附；相对 v6 少一张全图火，不能仅据此给出秒数
v8	与 v7 同核心；回春按线程行段并行，仍需会合	同 v7，读毕尝试释放文本向量容量	未附；有 MPI 时是一个工作进程，不能乘上 64 个进程
br	若本步 A_s 块活跃，每块 S×64 行、每行 S 字组：约 $A_s \cdot 64S^2$ 个字组；最坏全图	每个参与进程按当前分配持整幅两代两位图，另有链接、活动队列、标记、分批读入和局部行窗	仅有 1.36 s 建线程探针注释，无整题日志；活跃比例未知，不能报总加速比

全程应对每步求和。对 v1–v3，若第 s 步回春面积为 $H_s \times C_s$，它还须访问 $H_s C_s$ 格。对 v4–v8，横跨字数 $J_s=\lfloor ey/64 \rfloor-\lfloor sy/64 \rfloor+1$，要处理 $H_s J_s$ 个字，点惊雷则为常量次访问。br 回春还要裁剪所

涉及的块。全图例外还要计一轮读入、一轮输出，不能把一个时间步的工作直接当成整题时间。固定 P、S 不意味着它们在工程预算里可以忽略。

> 一句话记住：先写“几格、几字、几块”，再谈每秒做多少。

B.2 一张纸算完内存

二进制容量单位：1 KiB=1024 字节，1 MiB=1024 KiB，1 GiB=1024 MiB。取 n=16384，则 W=256，Z=4,194,816 字。一张带上下填充的位面用 33,558,528 字节，即 32 MiB 加 4 KiB。四张面约 128 MiB，第五张再加约 32 MiB。若 P=16，三行窗口合计 3×8×256×16=98,304 字节，即 96 KiB。两者之差是容量数量级，不代表每步都从主存传了同样多；芯片缓存能复用一部分数据。

按 LF、一位数字与一个分隔符计，大图格子文本约 512 MiB，还没算头行和事件。v2–v7 一次读完整文件，不能只看 v7 的 128 MiB 状态面就把它当全进程峰值。br 每批缓冲约 (2n+4)×1024 字节，即约 32 MiB；若多个进程都分配、都读，这部分也会重复。br 传给 halo 的四个空向量在当前入口没有解包；不能把一个并未实例填满的通信边界空间算成这条路径已发生的消耗。

> 一句话记住：两位可以表示一格，不等于全程序每格只占两位。

B.3 速度能推到哪一步

公式 T(P)≈T串行+T可分/P+T等待+T额外搬运 是分析模型，不是拟合结果。假设某个虚构工作流的串行部分占 10%，可平分部分占 90%，且没有等待，用 16 核的理想加速为 1/(0.1+0.9/16)=6.4。这个例子只解释为什么“16 核”不是“16 倍”；材料没有给任何版本的 10% 比例。若工作单元原本已是 64 格的一字，线程分摊的是若干个字，不是把既有的处理速率再无条件乘 64。

满分预算 68 秒与基本分预算 110 秒，按全部 64 核分别约为 8.11 和 13.11 核纳秒/逻辑格；按一个 16 核节点而不跨节点分别约为 2.03 和 3.28。项目没有提供同步的编译选项、线程绑定、输入指纹、多个重复样本与正确性记录，所以不能给九个实现排速度名次。br 第 520 行注明创建线程探针实测 1.36 秒；没有原始输入和探针运行日志，只能标“源码作者记录的局部实测”，更不能把它从未知的总耗时直接减去当新成绩。

若将来拿到记录，先比同一输入的最终森林，再把读、事件、转移、输出的计时边界逐项对齐。以同一机器同一输入的基准中位耗时 T_0 除候选中位耗时 T_1 才是可核对的加速比；用题目时限除自己的耗时仅代表离目标还有多远。

> 一句话记住：时限给预算，源码给操作数，真实秒数只能由有条件记录的实验给出。

APPENDIX C

附录 C　按拼法查语法与设施

这一章你会得到什么：看见陌生符号时不用猜它在哪一页。用 PDF 或浏览器搜索左列的拼法；章节定位不随打印、字号和页码变化。中文概念可翻附录 A；这一页另外收录全部代码拼法，并把只有中文名字的运算按实际符号列出。

符号与数字

拼法	章节定位	一句话
`!`	1.2	把真假取反
`!=`	第2章 第 26–55 行	比较值不相等
`#define`	4.2	定义预处理宏
`#else`	4.2	条件编译未选前支时改选此支
`#endif`	4.2	结束条件编译区段
`#if`	4.2	按预处理条件保留一段源码
`#ifdef`	附录 A	若宏已定义才编译相应段
`#ifndef`	附录 A	宏未定义才保留此段
`#include`	第2章 第 1–25 行	在预处理时引入头文件声明
`#pragma`	1.5	向编译器提供实现或并行指示
`#pragma omp barrier`	第2章 第 189–221 行	要求团队全部到达同步点
`#pragma omp for`	1.6	在已有团队内分摊循环
`#pragma omp parallel`	1.5	同队线程各执行结构块
`#pragma omp parallel for`	1.5	同时建立团队和分派循环
`#pragma omp single`	1.6	由团队任一线程执行，通常末尾等待
`#pragma once`	5.6	同一编译单元只处理该头一次
`%`	1.2	求整数除法的余数
`&`	1.2	看上下文区分取地址、引用声明与按位交集
`&&`	1.2	两边都为真，左边假时不求右边
`&=`	第2章 第 141–168 行	用按位交集更新原值
`*`	第 0 章	区分指针声明、读写所指对象与乘法
`++`	1.1	数值加一或指针移到下一元素
`+=`	第2章 第 85–110 行	把计算量加到已有变量

拼法	章节定位	一句话
->	4.4	通过指针访问对象成员
-fiopenmp	4.1	启用 Intel 实现的一种编译选项
-fopenmp	4.1	启用 OpenMP 指令的一种编译选项
-machinefile	附录 A	指定参与主机清单
-np	4.1	向启动器指定进程数量
-o	4.1	指定生成文件名
-O3	4.1	请求较高级别优化
-xHost	4.1	为编译主机处理器指令集优化
. / ->	附录 A	前者访问对象成员，后者通过指针访问
0ULL	1.4	具有指定宽度类别的无符号零
::	1.1	在指定作用域查找名字
<<	1.1	依操作数决定移位或流输出
==	1.1	比较两个值是否相等
>>	1.1	依操作数决定移位或流输入
?:	第 2 章 L149	依条件择一求值。
[&]	第2章 第 26–55 行	匿名函数按引用使用外部局部量
[]	第2章 第 1–25 行	依上下文表示下标或匿名函数开头
[]{}	附录 A	为固定数组提供空初始化列表
[i]	第 2 章 L76–79	按元素编号定位，不自动保证下标有效。
__builtin_popcountll	5.9	数无符号长长整数中置位数
__has_include	4.6	预处理时检查某头文件能否找到
_fseeki64	附录 A	用大偏移重新定位文件
_ftelli64	5.7	查询大文件位置
_mm_pause	4.2	忙等时的处理器提示
_shld	5.2	拼接本字和邻字的边界位。
{ }	第 2 章 L10–15	限定代码块、初始化列表或类型定义；要看上下文。
|	第 2 章 L193	合并多个方向的火。
|=	第 2 章 L78	只把选中的位置位。
||	第 2 章 L50	只需一侧为真；与按位或不同。
~	1.4	每一位独立取反

> 一句话记住：同一拼法要结合操作数类型和上下文读。

A–F

拼法	章节定位	一句话
argc	第2章 第 1 - 25 行	命令行参数数目，含程序名
argv	1.2	保存各个参数字符串指针的数组视图
asm	附录 A	在高级语言内嵌入机器指令模板
ASM_SHLD	5.2	选择双字移位汇编形式
assign	5.9	用给定范围或数量的元素替换内容
atoi	4.3	从字符串解析整数，无法转换时得到 0
atomic	4.2	为特定对象提供不可分割的并发操作
auto	第2章 第 1 - 25 行	令编译器从初始化推断类型
auto&	4.3	由表达式推导对象类型并保留引用
begin()	第2章 第 169 - 188 行	指向区间第一个元素
bool	5.3	真或假
bool*	5.3	指向布尔标记数组
byte	1.3	由八位组成的字节，在本题平台成立
cerr	第2章 第 1 - 25 行	C++ 标准错误流
char	1.2	容纳一个字符代码的类型
char**	第2章 第 1 - 25 行	指向字符串指针的指针
cin	1.1	C++ 标准字符输入流
class	4.2	把状态与方法封装为类型
clear	4.5	移除元素而不承诺释放容量
const	1.2	限制通过这个名字修改值
const ui*	5.6	通过此指针只读字内容
constexpr	5.2	可用于常量表达式的声明方式
continue	附录 A	跳到当前循环下一次迭代
count()	第2章 第 1 - 25 行	取得以当前单位计量的数值
cout	1.1	C++ 标准字符输出流
data()	1.2	返回连续元素区首地址
defined	4.2	判断指定宏是否存在
deque	5.3	支持从两端操作的顺序容器
double	0.1	双精度浮点类型
duration	第2章 第 1 - 25 行	带计数类型和单位的时间间隔

拼法	章节定位	一句话
else	1.1	前一条件不成立的分支
emplace_back	4.3	在容器末端直接构造元素
empty()	5.5	检查容器是否为空
end()	第2章 第 169 – 188 行	指向区间末端之后
exchange	5.6	原子换新值、返回旧值
exit	附录 A	退出整个进程
explicit	1.3	禁止利用该构造函数做隐式转换
false	5.6	布尔假值
fclose	1.2	关闭文件流并报告成功或失败
fetch_add	4.2	原子地加值并返回旧值
fgets	D.2	最多读容量减一的字符并终止字符串
FILE	第2章 第 26 – 55 行	C 文件流的类型
fill	第2章 第 1 – 25 行	把半开区间中的每个元素赋给同一值
find	附录 A	返回对应迭代器，找不到等于 end
FIRE_HAS_MPI	4.6	源码内部声明的条件编译标记
FIRE_NO_MAIN	附录 A	预处理时可排除入口等代码
FIRE_THREADS	4.3	v8 读取的环境变量
fopen	1.2	按路径与模式开流，失败返回空指针
for	1.1	依次执行初始化、条件判断、循环体与更新
for (;;)	附录 A	省略循环条件，靠显式控制流退出
fprintf	1.2	按格式向指定流写文本
fread	1.2	读入指定数目和大小的项，返回完整项数
freopen	1.1	按路径和模式重接既有流，失败返回空
front	5.10	取得队列首元素
fscanf	附录 A	按格式从文件流读数据
fseek	第2章 第 26 – 55 行	以起点、当前位置或末端为基准移动文件位置
fseeko	附录 A	用 off_t 类型偏移定位
ftell	第2章 第 26 – 55 行	返回当前文件位置，失败可返回负数
ftello	5.7	用 off_t 报告当前位置
fwrite	第2章 第 222 – 261 行	写入多项并返回成功写出的完整项数

> 一句话记住：同一拼法要结合操作数类型和上下文读。

G–M

拼法	章节定位	一句话
getenv	4.3	按名取环境字符串，不存在则返回空指针
GiB	1.1	2^{30} 字节
goto	5.6	跳转到当前函数中的指定标号
hardware_concurrency	4.3	给出并行能力提示，可能为零
I_MPI_PIN	附录 A	配置进程绑定的环境变量
I_MPI_PIN_DOMAIN	4.3	配置绑定域的环境变量
if	1.1	条件为真才进入分支
inline	5.2	函数说明符，不保证展开
int	1.1	带符号普通整数类型
int main(int argc, char** argv)	第2章 第 1–25 行	参数个数加参数字符串指针
int&	附录 A	为已有整数提供可修改的别名
int64_t	5.7	恰为 64 位的带符号整数
join	4.3	等待一个可汇合线程执行结束
KiB	1.7	2^{10} 字节
lambda	第2章 第 1–25 行	在使用位置构造可调用对象
load	4.2	取一次原子对象当前值
lock()	5.6	尝试独占临界区，未获时等待
lock_guard	5.6	构造时加锁、离开作用域时解锁
long	第2章 第 26–55 行	带符号整型，宽度依平台
main	第2章 第 1–25 行	程序入口函数
max	4.4	返回较大的候选值
memory_order_acq_rel	附录 A	合并获取与释放的读改写顺序
memory_order_acquire	4.2	获取对方已发布的工作
memory_order_relaxed	4.2	只提供本对象的原子操作，不单独发布其他数据
memory_order_release	4.2	把此前的修改随发布操作交给读者
memset	第2章 第 222–261 行	按字节填充一段存储
MiB	1.3	2^{20} 字节
milli	第2章 第 1–25 行	表示千分之一的单位比例

拼法	章节定位	一句话
min	第2章 第 1–25 行	返回较小的候选值
MPI	0.4	跨进程传递消息的标准接口
MPI_Abort	附录 A	要求异常终止通信组中的任务
MPI_Bcast	附录 A	将一份数据从根进程广播到全组
MPI_Comm_rank	4.6	把本进程在通信组内的编号写入输出参数
MPI_Comm_size	5.11	取通信组参与进程数
MPI_COMM_WORLD	4.6	通常包含本次启动全部参与进程的通信组
MPI_Finalize	4.6	完成 MPI 环境收尾，是集体操作
MPI_Gatherv	附录 A	按各进程数量与偏移收集
MPI_Init	4.6	初始化进程的 MPI 环境
MPI_INT	附录 A	消息元素按 C 的 int 描述
MPI_Irecv	附录 A	登记非阻塞接收并给出请求
MPI_Isend	附录 A	启动非阻塞发送并给出请求
MPI_Recv	5.9	阻塞接收接口
MPI_Request	附录 A	标识一个异步操作的请求
MPI_Scatterv	附录 A	按每进程给定数量与偏移分发数据
MPI_Send	5.9	阻塞发送接口
MPI_STATUS_IGNORE	附录 A	允许调用不返回状态结构
MPI_Wait	附录 A	等请求完成
MPI_Wtime	附录 A	返回用于量经过时间的秒数
mpiicpx	4.1	面向 C++ 的 MPI 编译驱动
mpirun	4.1	启动多个进程
mutex	5.6	使临界区互斥进入的设施

> 一句话记住：同一拼法要结合操作数类型和上下文读。

N–S

拼法	章节定位	一句话
namespace	第2章 第 1–25 行	给名字划分范围
now()	第2章 第 1–25 行	返回该时钟当前时刻
nowait	第2章 第 111–140 行	在允许的构造末尾取消隐式等待
nullptr	5.5	不指向对象的指针值

拼法	章节定位	一句话
off_t	附录 A	表示文件偏移的类型
omp_get_num_threads	第2章 第 111 – 140 行	无参数，取当前团队人数
omp_get_thread_num	第2章 第 111 – 140 行	无参数，取团队内零基编号
OpenMP	0.4	共享内存并行指令与运行库
operator ()	附录 A	函数对象接受参数的调用形式
pop_front	5.10	移除首元素，不返回取出的值
private	4.2	仅类自身及获权者可直接使用
process_tasks	5.10	领本步块、处理并提名下一批。
public	4.2	允许外部使用成员
push_back	5.3	在尾端添加元素
reserve	第2章 第 85 – 110 行	预留容量而不创建新元素
resize	5.9	改变已有元素数量
return	1.2	结束当前函数并可带回值
schedule(static)	1.5	提前给团队分配循环迭代
SEEK_END	第2章 第 26 – 55 行	以文件末端定位的常量
SEEK_SET	第2章 第 26 – 55 行	以文件开头定位的常量
shld	附录 A	从相邻字补位的双字移位
shrink_to_fit	4.5	请求容量缩至接近现有大小，不强制执行
size()	第2章 第 222 – 261 行	返回已存在的元素个数
size_t	1.3	能表示对象大小的无符号整数类型
sizeof	附录 A	给出对象或类型所占字节数
SpinBarrier::wait	4.2、5.10	到齐后才进入新世代。
SpinLock	5.6	等待时反复测试的互斥锁
sscanf	D.2	从字符缓冲按格式提取字段并返回成功数
static	1.5	视位置表示静态存储期或内部链接
static_assert	5.2	条件不满足则拒绝编译
static_cast	1.3	显式执行编译期可检查的类型转换
static_cast<int>	5.8	把值转换到 int
std::	1.1	从标准库命名空间查找名字
std::max	5.8	比较两个参数取较大者

拼法	章节定位	一句话
std::milli	第2章 第 1 - 25 行	一千分之一秒为时长周期
std::min	第2章 第 111 - 140 行	比较两个参数取较小者
stderr	1.2	C 的标准错误流
stdin	附录 A	默认输入流
stdout	附录 A	默认输出流
steady_clock	第2章 第 1 - 25 行	用来量经过时长的单调时钟
store	4.2	把新值发布到原子对象
stride	1.3	相邻两行对应位置的地址间距
struct	第2章 第 85 - 110 行	将相关字段组合成一种类型
swap	1.2	交换两个对象或容器的内容归属

> 一句话记住：同一拼法要结合操作数类型和上下文读。

T–Z

拼法	章节定位	一句话
template	5.2	按编译期参数生成具体实现
thread	0.4	代表一条标准 C++ 线程
tile_edge	5.5	按方向返回相邻边缘数据指针。
time_point	第2章 第 1 - 25 行	某种时钟上的一个时刻
true	5.6	布尔真值
uint64_t	1.4	恰为 64 位的无符号整数
uint8_t	1.3	恰为 8 位的无符号整数
ULL	附录 A	标明无符号长长整数字面量
unlock()	5.6	释放互斥
unordered_map	5.11	按键查关联值的散列表
unsigned	1.3	无符号整数
using	第2章 第 1 - 25 行	为一种类型定义别名
vector	1.1	管理一段连续可变长存储的容器
void	4.2	函数不返回数据值
while	1.2	只要条件成立就重复执行

> 一句话记住：从拼法到源码，再从源码回到那一格森林。

APPENDIX D

附录 D　哪些地方还不能照注释下结论

这一章你会得到什么：把可以由源码直接证实的差异、依赖合法输入的写法、必须上机器才能回答的问题分开。书中不改任何原文件。

D.1　题面、脚本和任务说明的分歧

证据	看见的事实	本书怎么处理
v7 L221 – 261；br L676 – 716	任务说明的 221 与 676 都早于文件末行。v7 还没写输出，br 还没完成工作队和收尾。	按真实 261、716 行讲。
compare.py L7 – 29	文本模式逐行 split，再比每个字段字符串，不是逐字节。zip 在第一份文件较长而第二份刚耗尽时，可能预先消费第一份多出的一行，后面的 readline 就漏掉这行。	按 n 行 n 项输出；验书时另用独立小例检查比对器，不把这处宽容当成格式要求。
intro.md L48、52、57；gencase.py L72 – 85	题面先写 m 个事件，后文使用未另定义的 k；生成器头行和实际写出的都是事件总数。	书中统一写 m，遵守升序且每步最多一个。
PROMPT §0、§11	结构表列十章 Markdown，自检又要十一个。	保留十个指定文件，加一份独立阅读导引 README.md；进度与自检另计。
br L630 – 637、706 – 714；v8 L79 – 90	只有 v8 的可选 MPI 路径显式排除非零 rank；br 没有相同分支。	不替两份不同作者的实现调和成一个故事。
br L174 – 186、533 – 549、663 – 704	当前池用自旋锁领取任务、自旋栅栏等阶段，没用条件变量。	分开说明旧 node_step 与 main 实际调用的 process_tasks。

> 一句话记住：作者注释、任务摘要和可执行路径不一致时，先追路径。

D.2　正确读懂合法输入之外的前提

位置	证据与影响
v1 L79；v2 L98；v7 L131	晚到的条目可以在 <= 的版本被消费，但游标不会越过队首的未来条目；这些写法不是排序器。
v2 L28 – 34；v7 L32 – 38	v2 不检查短读，v7 检查短读，但没有检查 fseek/ftell 失败。返回负长度、极大文件或不能定位的文件不在其稳健范围内。
v7 L49 – 53、69 – 79、91 – 98	数字读取允许跳过非数字，到尾可能返回 0；逐格查表假设输入字符在 0..3。正常题面给出保证，任意外来文本没有同样保证。
v7 L60 – 61、116	使用 size_t 不会撤销先前的有符号计算；n+63、n+2 等仍先在 int 中计算。题目尺寸可以表示，不代表任意输入均安全。
v7 L230 – 244；v8 L305 – 316；br L561 – 575	写满后 q 等于 qend，计算 q+2 可能已越过可构造的一过尾指针；即使最终只用它作比较，也不是可移植的边界判断。用剩余空间的概念理解其意图。

位置	证据与影响
v7 L235、253 – 254；br L565、584 – 585	fwrite 的返回项数与 fclose 结果未查，正常退出不能排除后期写盘失败。
v7 L111 – 114、212 – 228	没有 MPI 的 rank 筛选；被外部启动多个进程时，各份并不协调同一输出路径。
v8 L31 – 35、79 – 89、103 – 115、300 – 304、333 – 335	检测头文件不等于验证链接；非零 rank 进入集体收尾，不保证立刻退进程。0 号有错误分支未配对 Finalize。
v8 L54、176 – 186、240、279 – 297	注释中的时延倍数和“归还给系统”不是可复验的保证。硬件并发提示不是绑定核数，两个阶段计时也偏向 0 号视角。
br L289 – 320、330 – 340、622 – 640	注释说四个象限，local_side 却改成 n，起始行列仍乘 rank 坐标。固定 16384 和块宽 1024 与读取尺寸共同决定下标；不能声称通用小规模或 64 rank 可运行。
br L315 – 340、633 – 637、649 – 656	按第一行的字节跨度和每格两字节定位；多空格、不同换行宽度或文件读取失败不能按同样方式解释。fopen、fgets、sscanf 的关键返回量也没有完整核查。
br L446 – 515、658 – 661、684 – 708	边缘打包器和还原器存在，主路径没有调用；输出无唯一写者。不能从 MPI_Init 推断已做跨进程边界交换或聚合。
br L1、517 – 531；intro.md L243 – 250	源码依赖独立 spin_lock.hpp，题面只收一个 cpp；原文件组合不能当作可直接提交的单文件。
std-main.cpp L120 – 131、155	参考实现访问 events[cur_event_id] 前没有显式检查游标仍小于事件数。规则解释以题面及合法事件的行为为准，不推定参考程序对所有边缘输入都稳健。

> 一句话记住：能在题目保证下解释一个程序，不等于已经证明它能处理所有文件与启动方式。

D.3 待验证的性能与平台问题

没有比赛容器、完整编译日志、原始计时记录，无法确认 v7 在现场具体是忽略指令后串行运行，还是缺少运行库符号而链接失败。v8 的自旋在指定绑定下是否合算、br 的活动队列在这组火势里有多少块能跳过，也没有实测证据。源码中的探针注释与题面预算各自保留，不以推算填补成绩空格。

编译器行为可对照 Intel 对 OpenMP 开关的说明；同步语义见 OpenMP 的 single 规范；进程结束的集体性质见 MPI Forum 的 Finalize 说明。链接供深入查阅，阅读本书无需联网。

> 一句话记住：原题规定“怎么评”，源文件显示“写了什么”，这两者之间还差一份真实运行记录。